



UNIVERSITY  
*of* SAN CARLOS  
SCIENTIA • VIRTUS • DEVOTIO



## **GATED LEARNING-PROGRESS EXPLORATION**

A Thesis Project  
Presented to the Faculty of the  
Department of Computer, Information Sciences, and Mathematics  
University of San Carlos

In Partial Fulfillment  
of the Requirements for the Degree  
**BACHELOR OF SCIENCE IN COMPUTER SCIENCE**

By  
**MIKAEL VINCENT B. TUMAMPOS**  
**ERVIN JOSHUA L. GUIRNELA**

**CHRISTINE F. PEÑA, MMATH**  
Faculty Adviser

May 2026

## DECLARATION OF AUTHORSHIP AND ORIGINALITY

This thesis, entitled *Gated Learning-Progress Exploration*, is the original work of the undersigned authors. All sources, prior work, and external materials used in the preparation of this thesis have been properly acknowledged where applicable. No portion of this thesis has been submitted for another degree or academic qualification at any institution.

---

MIKAEL VINCENT B. TUMAMPOS  
Author

---

ERVIN JOSHUA L. GUIRNELA  
Author

---

CHRISTINE F. PEÑA, MMATH  
Thesis Adviser

## **ACKNOWLEDGEMENTS**

This thesis is dedicated to all men and women who have contributed to science: to those whose names are preserved in books and archives, and to those whose careful work remains known chiefly through the knowledge it made possible. Their patience before uncertainty, discipline in the pursuit of evidence, and willingness to revise understanding in the presence of truth form the intellectual inheritance on which every scientific undertaking depends.

The completion of this work is indebted to that long and shared tradition. Its methods, questions, and standards were shaped by generations of researchers, teachers, engineers, and students who treated inquiry as both a responsibility and a service. May this study, in its limited scope, honor their example by adding a small and sincere contribution to the continuing work of understanding.

## ABSTRACT

Sparse, delayed, or deceptive rewards remain a persistent challenge in deep reinforcement learning because a policy may receive little task-relevant feedback before useful behavior is discovered. Intrinsic motivation addresses this limitation by augmenting the environment reward with auxiliary signals that encourage exploration, but common novelty- and prediction-error-based objectives can reward transitions that are surprising without being learnable or useful. This study investigates an intrinsic-reward method for reducing such unproductive curiosity while preserving the exploratory benefits of reward augmentation. The proposed method, *Gated Learning-Progress Exploration* (GLPE), combines two complementary transition-level signals: feature-space impact, measured by the change between consecutive latent observations, and region-local learning progress, measured by the decrease in forward-dynamics prediction error within an online partition of the learned latent space. The intrinsic reward is computed from normalized impact and learning-progress components and is optionally filtered by a region-specific gate that suppresses rewards in areas with persistently high prediction error but low learning progress. An ungated variant is also evaluated to isolate the effect of the base intrinsic score from the gate.

The method is implemented with a Proximal Policy Optimization backbone and evaluated on five Gymnasium control benchmarks: MountainCar-v0, BipedalWalker-v3, Ant-v5, HalfCheetah-v5, and Humanoid-v5. The experimental design compares GLPE and GLPE without gating against PPO, ICM, RND, RIDE, and RIAC under shared policy architectures, training budgets, and evaluation procedures. Across the benchmark suite, GLPE without gating is a consistent default that remains competitive with the strongest baseline methods under both sample-efficiency and wall-clock views. The gated variant is most useful in sparse-reward settings such as MountainCar-v0, where it achieves complete solved-threshold reliability while producing the strongest final return and step-AUC, but it can be conservative in dense-reward or high-variance tasks. Ablation results show that impact and learning progress contribute differently across environments, supporting their combined use as a robust exploration signal. The study contributes a formal formulation of GLPE, an empirical comparison against established intrinsic-motivation baselines, and an analysis of the tradeoff between targeted exploration control and computational overhead.



## TABLE OF CONTENTS

|                                                              |             |
|--------------------------------------------------------------|-------------|
| <b>DECLARATION OF AUTHORSHIP AND ORIGINALITY</b>             | <b>i</b>    |
| <b>ACKNOWLEDGEMENTS</b>                                      | <b>ii</b>   |
| <b>ABSTRACT</b>                                              | <b>iii</b>  |
| <b>TABLE OF CONTENTS</b>                                     | <b>iv</b>   |
| <b>LIST OF FIGURES</b>                                       | <b>viii</b> |
| <b>LIST OF TABLES</b>                                        | <b>ix</b>   |
| <b>1 INTRODUCTION</b>                                        | <b>1</b>    |
| 1.1 Background of the Study . . . . .                        | 1           |
| 1.2 Rationale . . . . .                                      | 3           |
| 1.3 Problem Statement . . . . .                              | 4           |
| 1.3.1 General Objective . . . . .                            | 5           |
| 1.3.2 Specific Objectives . . . . .                          | 5           |
| 1.4 Significance of the Study . . . . .                      | 5           |
| 1.5 Scope and Limitations . . . . .                          | 6           |
| 1.6 Organization of the Study . . . . .                      | 7           |
| <b>2 RELATED LITERATURE</b>                                  | <b>9</b>    |
| 2.1 Reinforcement Learning Foundations . . . . .             | 9           |
| 2.2 Policy Gradient and PPO Methods . . . . .                | 10          |
| 2.3 Intrinsic Motivation in Reinforcement Learning . . . . . | 12          |
| 2.4 Novelty-Based Exploration . . . . .                      | 13          |
| 2.5 Prediction-Error Curiosity . . . . .                     | 15          |
| 2.6 Impact-Driven Exploration . . . . .                      | 16          |
| 2.7 Learning-Progress Exploration . . . . .                  | 17          |
| 2.8 Unproductive Curiosity and Noisy Dynamics . . . . .      | 19          |
| 2.9 Research Gap . . . . .                                   | 20          |
| <b>3 THEORETICAL FRAMEWORK</b>                               | <b>22</b>   |
| 3.1 Problem Formulation . . . . .                            | 22          |

|          |                                               |           |
|----------|-----------------------------------------------|-----------|
| 3.2      | Reward Augmentation . . . . .                 | 23        |
| 3.3      | Latent Representation Learning . . . . .      | 24        |
| 3.4      | Forward and Inverse Dynamics Models . . . . . | 25        |
| 3.5      | Feature-Space Impact . . . . .                | 26        |
| 3.6      | Region-Local Learning Progress . . . . .      | 27        |
| 3.7      | Gated Learning-Progress Exploration . . . . . | 28        |
| 3.8      | Theoretical Assumptions . . . . .             | 30        |
| <b>4</b> | <b>METHODOLOGY</b>                            | <b>32</b> |
| 4.1      | Research Design . . . . .                     | 32        |
| 4.2      | Proposed Method . . . . .                     | 33        |
| 4.2.1    | Method Overview . . . . .                     | 33        |
| 4.2.2    | Latent Dynamics Module . . . . .              | 35        |
| 4.2.3    | Online Latent-Space Partitioning . . . . .    | 35        |
| 4.2.4    | Learning-Progress Estimation . . . . .        | 36        |
| 4.2.5    | Feature-Space Impact Estimation . . . . .     | 36        |
| 4.2.6    | Component-Wise Scale Stabilization . . . . .  | 37        |
| 4.2.7    | Base Intrinsic Reward . . . . .               | 37        |
| 4.2.8    | Ungated Variant . . . . .                     | 37        |
| 4.2.9    | Gated Variant . . . . .                       | 38        |
| 4.2.10   | Intrinsic Reward Scheduling . . . . .         | 38        |
| 4.3      | Reinforcement Learning Backbone . . . . .     | 39        |
| 4.3.1    | PPO Training Procedure . . . . .              | 39        |
| 4.3.2    | Policy and Value Networks . . . . .           | 40        |
| 4.3.3    | Reward Handling . . . . .                     | 40        |
| 4.4      | Baseline Methods . . . . .                    | 41        |
| 4.4.1    | PPO . . . . .                                 | 41        |
| 4.4.2    | ICM . . . . .                                 | 41        |
| 4.4.3    | RND . . . . .                                 | 41        |
| 4.4.4    | RIDE . . . . .                                | 42        |
| 4.4.5    | RIAC . . . . .                                | 42        |
| 4.5      | Experimental Environment . . . . .            | 42        |
| 4.5.1    | Benchmark Suite . . . . .                     | 42        |
| 4.5.2    | Training Budgets . . . . .                    | 43        |
| 4.5.3    | Random Seeds and Determinism . . . . .        | 44        |
| 4.6      | Evaluation Methodology . . . . .              | 44        |

|          |                                                      |           |
|----------|------------------------------------------------------|-----------|
| 4.6.1    | Offline Checkpoint Evaluation . . . . .              | 44        |
| 4.6.2    | Primary Performance Metric . . . . .                 | 44        |
| 4.6.3    | Sample Efficiency Metrics . . . . .                  | 45        |
| 4.6.4    | Reliability Metrics . . . . .                        | 45        |
| 4.6.5    | Wall-Clock Efficiency Metrics . . . . .              | 46        |
| 4.6.6    | Statistical Reporting . . . . .                      | 46        |
| 4.7      | Ablation Methodology . . . . .                       | 46        |
| 4.7.1    | Impact-Only Variant . . . . .                        | 46        |
| 4.7.2    | Learning-Progress-Only Variant . . . . .             | 47        |
| 4.7.3    | Gating Analysis . . . . .                            | 47        |
| 4.7.4    | Computational Analysis . . . . .                     | 47        |
| 4.8      | Project Management . . . . .                         | 48        |
| 4.8.1    | Development Process . . . . .                        | 48        |
| 4.8.2    | Costs . . . . .                                      | 48        |
| 4.8.3    | Timeline . . . . .                                   | 49        |
| 4.8.4    | Responsibilities . . . . .                           | 49        |
| 4.9      | Ethical and Reproducibility Considerations . . . . . | 49        |
| <b>5</b> | <b>RESULTS AND DISCUSSION</b>                        | <b>51</b> |
| 5.1      | Overview of Experimental Results . . . . .           | 51        |
| 5.2      | Learning Curves . . . . .                            | 52        |
| 5.2.1    | Sparse-Reward Environment Results . . . . .          | 52        |
| 5.2.2    | Continuous-Control Environment Results . . . . .     | 53        |
| 5.2.3    | Cross-Environment Observations . . . . .             | 54        |
| 5.3      | Step-AUC Analysis . . . . .                          | 54        |
| 5.4      | Final-Checkpoint Performance . . . . .               | 55        |
| 5.5      | Thresholded Reliability . . . . .                    | 56        |
| 5.5.1    | 25 Percent Threshold Results . . . . .               | 56        |
| 5.5.2    | 50 Percent Threshold Results . . . . .               | 58        |
| 5.5.3    | Solved-Threshold Results . . . . .                   | 58        |
| 5.6      | Ablation Results . . . . .                           | 59        |
| 5.6.1    | Impact-Only Performance . . . . .                    | 60        |
| 5.6.2    | Learning-Progress-Only Performance . . . . .         | 60        |
| 5.6.3    | Combined Reward Interpretation . . . . .             | 60        |
| 5.7      | Gating Behavior . . . . .                            | 61        |
| 5.7.1    | Gate Activation Patterns . . . . .                   | 61        |

|          |                                                             |           |
|----------|-------------------------------------------------------------|-----------|
| 5.7.2    | Effect in Sparse-Reward Settings . . . . .                  | 62        |
| 5.7.3    | Effect in Dense-Reward and High-Variance Settings . . . . . | 62        |
| 5.8      | Intrinsic Reward Dynamics . . . . .                         | 63        |
| 5.9      | Wall-Clock Efficiency . . . . .                             | 64        |
| 5.9.1    | Wall-Clock AUC . . . . .                                    | 64        |
| 5.9.2    | Runtime Breakdown . . . . .                                 | 65        |
| 5.9.3    | Computational Tradeoffs . . . . .                           | 65        |
| 5.10     | Discussion of Findings . . . . .                            | 67        |
| <b>6</b> | <b>CONCLUSION AND RECOMMENDATIONS</b>                       | <b>69</b> |
| 6.1      | Summary of the Study . . . . .                              | 69        |
| 6.2      | Conclusions . . . . .                                       | 71        |
| 6.3      | Contributions . . . . .                                     | 72        |
| 6.4      | Limitations . . . . .                                       | 74        |
| 6.5      | Recommendations for Future Work . . . . .                   | 75        |
|          | <b>BIBLIOGRAPHY</b>                                         | <b>78</b> |
|          | <b>APPENDICES</b>                                           | <b>80</b> |
|          | Technical Manuscript . . . . .                              | 80        |
|          | Peer Review Materials . . . . .                             | 88        |
|          | Presentation Certification . . . . .                        | 90        |

## LIST OF FIGURES

|     |                                            |    |
|-----|--------------------------------------------|----|
| 5.1 | Evaluation learning curves . . . . .       | 52 |
| 5.2 | Threshold reaching steps . . . . .         | 57 |
| 5.3 | GLPE ablation learning curves . . . . .    | 59 |
| 5.4 | GLPE gate decisions . . . . .              | 61 |
| 5.5 | Training reward decomposition . . . . .    | 63 |
| 5.6 | Wall-clock evaluation AUC . . . . .        | 65 |
| 5.7 | Runtime breakdown per PPO update . . . . . | 66 |

## LIST OF TABLES

|     |                                                |    |
|-----|------------------------------------------------|----|
| 4.1 | PPO hyperparameters . . . . .                  | 40 |
| 4.2 | Benchmark suite and training budgets . . . . . | 43 |
| 4.3 | GLPE hyperparameters . . . . .                 | 43 |
| 5.1 | Step-AUC by environment and method . . . . .   | 55 |
| 5.2 | Final-checkpoint returns . . . . .             | 55 |
| 5.3 | 25 percent threshold reliability . . . . .     | 57 |
| 5.4 | 50 percent threshold reliability . . . . .     | 58 |
| 5.5 | Solved-threshold reliability . . . . .         | 58 |
| 5.6 | GLPE component ablation returns . . . . .      | 59 |
| 5.7 | Gate-median microbenchmark . . . . .           | 66 |

# CHAPTER 1

## INTRODUCTION

### 1.1 Background of the Study

Reinforcement learning studies how an agent can learn a policy for sequential decision-making through interaction with an environment. At each time step, an agent observes the current state or observation, selects an action, receives a scalar reward, and transitions to a subsequent observation. The objective is to learn a policy that maximizes expected discounted return. In an episodic Markov decision process, this objective can be written as

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[ \sum_{t=0}^{T-1} \gamma^t r_t \right], \quad (1.1)$$

where  $\pi_\theta$  is a parameterized policy,  $\gamma \in [0, 1)$  is a discount factor,  $r_t$  is the reward at time  $t$ , and  $T$  is the episode horizon. Deep reinforcement learning extends this formulation by using neural networks to represent policies, value functions, and auxiliary predictive models, thereby enabling reinforcement learning in high-dimensional and continuous-control settings (Barto, 2012).

A persistent difficulty in reinforcement learning is exploration. When environment rewards are dense and well aligned with the task, reward feedback can guide policy improvement throughout training. However, many environments provide rewards that are sparse, delayed, or weakly informative during early interaction. In such settings, an agent may fail to encounter successful trajectories often enough to estimate useful gradients, even when the policy optimization algorithm is otherwise stable. This problem is especially important in control tasks where meaningful behaviors require extended action sequences before any large external reward is obtained.

Intrinsic motivation addresses this issue by augmenting the environment reward with an internally generated reward that encourages informative behavior even when external feedback is limited (Barto, 2012; Oudeyer & Kaplan, 2007; Oudeyer et al., 2007; Singh et al., 2004). The augmented reward is commonly expressed as

$$r_t = r_t^{\text{ext}} + \eta_t r_t^{\text{int}}, \quad (1.2)$$

where  $r_t^{\text{ext}}$  is the extrinsic reward supplied by the environment,  $r_t^{\text{int}}$  is an intrinsic reward computed from the agent's experience, and  $\eta_t$  controls the relative influence of intrinsic shaping. Intrinsic rewards can encourage novelty, information gain, prediction

improvement, or controllable change. Representative approaches include count-based exploration bonuses (Bellemare et al., 2016; Ostrovski et al., 2017; Tang et al., 2016), information-gain objectives (Houthoofd et al., 2016), prediction-error curiosity (Pathak et al., 2017; Stadie et al., 2015), Random Network Distillation (Burda, Edwards, Pathak, et al., 2018; Burda, Edwards, Storkey, & Klimov, 2018), episodic curiosity (Savinov et al., 2018), directed exploration schemes (Badia et al., 2020), and impact-driven exploration (Raileanu & Rocktäschel, 2020).

Although intrinsic rewards can improve sample efficiency, they can also introduce misleading optimization pressure. Unlike potential-based reward shaping, general intrinsic-reward augmentation is not guaranteed to preserve the optimal policy of the original task (Ng et al., 1999). Prediction-error and novelty bonuses may remain high in regions that are stochastic, difficult to model, or unrelated to task progress. This failure mode is often described as unproductive curiosity, and the noisy-TV example illustrates how an agent may repeatedly visit unpredictable but unhelpful states (Mavor-Parker et al., 2021). Consequently, exploration bonuses should not only encourage the agent to seek unfamiliar or surprising transitions; they should also distinguish between experience that supports learning and experience that remains persistently unlearnable.

Learning-progress exploration provides a useful perspective for this distinction. Instead of rewarding prediction error directly, a learning-progress signal rewards reductions in prediction error over time, thereby emphasizing regions where the agent’s predictive model is improving (Baranes & Oudeyer, 2009; Oudeyer & Kaplan, 2007; Schmidhuber, 1991). A transition is valuable under this view when it belongs to a part of the environment that is neither already mastered nor effectively random. However, practical use of learning progress in deep reinforcement learning requires mechanisms for estimating progress locally in a learned representation, stabilizing intrinsic-reward scale, and preventing high-error low-progress regions from dominating policy optimization.

This study investigates Gated Learning-Progress Exploration (GLPE), an intrinsic-reward method designed around these requirements. The method combines two complementary quantities: feature-space impact, which measures the magnitude of representation change between consecutive observations, and region-local learning progress, which measures improvement in a forward dynamics model within an online partition of latent space. The gated variant further suppresses intrinsic reward in regions where prediction error remains high while learning progress remains low. The resulting approach is evaluated as an exploration mechanism for Proximal Policy Opti-



mization (PPO) across discrete and continuous-control benchmarks, with comparisons against established intrinsic-reward baselines such as ICM, RND, RIDE, and RIAC (Baranes & Oudeyer, 2009; Burda, Edwards, Storkey, & Klimov, 2018; Pathak et al., 2017; Raileanu & Rocktäschel, 2020; Schulman et al., 2017).

## 1.2 Rationale

The central motivation for this study is the need for intrinsic rewards that remain useful under sparse, delayed, or unreliable external feedback without encouraging persistent interaction with unproductive regions of the environment. Many existing intrinsic-reward methods use prediction error, novelty, or state visitation as a proxy for exploration value. These proxies are effective when unfamiliar states are likely to be informative, but they can be insufficient when high intrinsic reward is caused by noise, stochastic transitions, or representational change that is not associated with task-relevant learning.

A prediction-error bonus, for example, rewards transitions that a learned model currently predicts poorly. This can guide the agent toward unfamiliar dynamics, but it does not by itself determine whether additional experience in the same region will reduce future error. If the error is high because the region is inherently stochastic or only weakly controllable, the agent may continue to receive intrinsic reward without acquiring useful competence. Similarly, novelty-oriented bonuses can favor broad coverage but may not prioritize transitions that change the agent’s ability to predict or control the environment.

Learning progress offers a more selective signal because it depends on temporal improvement rather than absolute surprise. If  $e_t$  denotes a forward-model prediction error, a simple progress measure can be viewed as the positive reduction between a longer-term and a shorter-term estimate of error:

$$\text{LP}_t = \max(0, \mu_t^{\text{long}} - \mu_t^{\text{short}}). \quad (1.3)$$

A high value indicates that recent prediction error has fallen below the longer-term baseline, suggesting that additional experience in the corresponding region has been productive. However, global learning-progress estimates are too coarse for environments with heterogeneous dynamics. Progress must be localized so that improvement in one part of the observation space does not mask stagnation in another.

Feature-space impact is also important because exploration should favor actions that produce meaningful changes in the learned representation. An agent that receives progress-based reward without regard to transition impact may overvalue small

changes in predictable regions. Conversely, an agent that receives impact reward without regard to learning progress may overvalue large but unlearnable changes. Combining both signals supports an intrinsic objective that rewards transitions that are both behaviorally consequential and associated with model improvement.

The proposed method is therefore motivated by three design principles. First, the intrinsic reward should be computed in a learned latent representation so that it can be applied to continuous observations without requiring hand-designed state abstractions. Second, learning progress should be estimated locally, using region-specific statistics, so that the method can distinguish productive and unproductive parts of the environment. Third, intrinsic shaping should be limited when evidence suggests persistent high error without corresponding progress. These principles lead to an intrinsic-reward family in which a base reward combines impact and learning progress, while an optional gate suppresses the reward in regions that satisfy a high-error low-progress criterion.

### **1.3 Problem Statement**

Sparse and delayed rewards make reinforcement learning difficult because the agent may not receive enough external feedback to discover useful behavior during early training. Intrinsic-reward methods address this problem by adding auxiliary exploration signals, but these signals can introduce their own failure modes. In particular, novelty- and prediction-error-based rewards may remain large in regions where the dynamics are noisy, difficult to predict, or not useful for solving the task. The technical problem addressed in this study is how to construct an intrinsic reward that promotes useful exploration while reducing the influence of unproductive curiosity.

The study focuses on the following research question:

How can feature-space impact and region-local learning progress be combined, and selectively gated, to improve reinforcement-learning exploration under sparse and continuous-control settings?

This question is investigated through the design and evaluation of a PPO-compatible intrinsic-reward method that estimates impact and learning progress from on-policy experience. The method is assessed against standard reinforcement-learning baselines and intrinsic-reward approaches using sample-efficiency, reliability, and wall-clock efficiency criteria.

### **1.3.1 General Objective**

The general objective of this study is to develop and evaluate a gated learning-progress intrinsic-reward method for reinforcement learning that encourages productive exploration by combining feature-space impact with localized model-improvement estimates.

### **1.3.2 Specific Objectives**

The specific objectives of the study are as follows:

1. Formulate an intrinsic-reward function that combines representation-space transition impact with region-local learning progress.
2. Develop an online latent-space partitioning procedure for maintaining local prediction-error statistics during policy training.
3. Define a gating mechanism that suppresses intrinsic reward in regions characterized by persistently high prediction error and low learning progress.
4. Integrate the proposed intrinsic reward with PPO using augmented rewards and scheduled intrinsic-reward scaling.
5. Compare the proposed method against PPO, ICM, RND, RIDE, and RIAC on a benchmark suite containing sparse-reward and continuous-control environments.
6. Evaluate performance using learning curves, final-checkpoint return, step-normalized area under the curve, thresholded reliability, and wall-clock efficiency.
7. Analyze ablations that isolate the roles of impact, learning progress, and gating in the resulting exploration behavior.

## **1.4 Significance of the Study**

This study contributes to reinforcement-learning research by examining an intrinsic-reward design that explicitly separates surprise from learning progress. The distinction is significant because high prediction error alone does not necessarily imply useful experience. By emphasizing local improvement in a dynamics model, the proposed method addresses a major limitation of curiosity-driven exploration: the tendency to reward transitions that are unpredictable but not productive.

The study is also significant for sparse-reward learning. In environments where successful behavior requires long action sequences before substantial external reward

is observed, an agent requires additional signals that make early learning possible. An intrinsic reward based on both impact and learning progress can guide the agent toward transitions that change the learned representation while still favoring regions where predictive competence is improving. This provides a principled alternative to using novelty or prediction error alone.

For continuous-control problems, the study demonstrates how region-local exploration statistics can be maintained in a learned latent space rather than in a manually discretized state space. This is important because continuous observations do not naturally support simple tabular visitation counts or fixed region definitions. An online partition of feature space allows the method to adapt its locality structure as training data are collected.

The evaluation framework further contributes by considering both sample efficiency and computational efficiency. Intrinsic rewards can improve learning per environment step while increasing the time required for each update. Reporting both step-normalized and wall-clock-normalized performance gives a more complete view of whether an exploration method is practically useful. The inclusion of ablations also clarifies how each component of the proposed method affects learning, reliability, and computational tradeoffs.

Finally, the study documents a complete reinforcement-learning research workflow: theoretical formulation, method design, experimental comparison, ablation analysis, and interpretation of results. The work is therefore relevant to research on exploration, intrinsic motivation, and empirical evaluation in deep reinforcement learning.

## **1.5 Scope and Limitations**

The scope of this study is limited to intrinsic-reward exploration for episodic reinforcement learning with PPO as the policy-optimization backbone. The proposed method is evaluated on five Gymnasium control benchmarks: MountainCar-v0, BipedalWalker-v3, Ant-v5, HalfCheetah-v5, and Humanoid-v5. These environments cover sparse-reward exploration and continuous-control locomotion, but they do not exhaust the range of possible reinforcement-learning domains. The study uses the default environment reward functions and termination conditions for these benchmarks.

The method operates on vector observations and learns a latent representation through auxiliary forward and inverse dynamics models. This study does not evaluate image-based observations, language-conditioned tasks, multi-agent environments, offline reinforcement learning, or model-based planning. The conclusions should there-

fore be interpreted as applying primarily to online, on-policy, vector-observation control tasks.

The proposed intrinsic reward is evaluated as a shaping signal rather than as a policy-invariant transformation of the original reward. Because general intrinsic reward augmentation can change the optimal policy, the study emphasizes empirical performance under controlled comparisons rather than formal guarantees of optimality preservation. The reward used for policy optimization has the form

$$r_t = r_t^{\text{ext}} + \eta_t \text{clip}(r_t^{\text{int}}, -r_{\max}, r_{\max}), \quad (1.4)$$

where the intrinsic coefficient, clipping range, and scheduling choices influence the resulting behavior.

The experimental comparison is limited to PPO, ICM, RND, RIDE, RIAC, and the proposed GLPE variants. All methods are evaluated using the same general reinforcement-learning backbone so that differences can be attributed primarily to the exploration objective. However, the results may depend on hyperparameter choices, benchmark selection, random seeds, and the computational setting used for training. The study reports multiple seeds and uncertainty-aware summaries where applicable, but it does not claim universal dominance over all possible exploration methods or training regimes.

The gated mechanism is designed to reduce unproductive curiosity by suppressing intrinsic reward in high-error low-progress regions. This behavior can support strong sparse-reward performance, but it can also become conservative in high-variance environments where temporary low progress does not necessarily imply that a region should be ignored. Consequently, gating is treated as a targeted exploration guardrail rather than as a universally beneficial component.

## 1.6 Organization of the Study

The study is organized into six chapters.

Chapter 1 introduces the reinforcement-learning exploration problem, motivates the use of intrinsic rewards based on impact and learning progress, states the research objectives, and defines the scope and limitations of the study.

Chapter 2 reviews related literature on reinforcement learning, PPO, intrinsic motivation, novelty-based exploration, prediction-error curiosity, impact-driven exploration, learning-progress exploration, and noisy dynamics. It also identifies the research gap

addressed by the proposed method.

Chapter 3 presents the theoretical framework. It formalizes the reinforcement-learning problem, reward augmentation, latent representation learning, forward and inverse dynamics models, feature-space impact, region-local learning progress, the gated exploration criterion, and the assumptions underlying the method.

Chapter 4 describes the methodology. It details the proposed GLPE method, the PPO training procedure, baseline methods, benchmark environments, evaluation metrics, ablation methodology, project-management considerations, and ethical and reproducibility concerns.

Chapter 5 presents and discusses the experimental results. It reports learning curves, step-AUC, final-checkpoint performance, thresholded reliability, ablation results, gating behavior, intrinsic-reward dynamics, and wall-clock efficiency. The chapter interprets the results in relation to the stated objectives and the design principles of the method.

Chapter 6 concludes the study by summarizing the main conclusions, identifying contributions and limitations, and recommending directions for future work.

## CHAPTER 2

### RELATED LITERATURE

#### 2.1 Reinforcement Learning Foundations

Reinforcement learning provides a mathematical framework for learning behavior through interaction with an environment. The standard formulation is an episodic Markov decision process (MDP), defined by an observation or state space  $\mathcal{S}$ , an action space  $\mathcal{A}$ , a transition distribution  $P(s_{t+1} \mid s_t, a_t)$ , a reward function  $r(s_t, a_t, s_{t+1})$ , a discount factor  $\gamma \in [0, 1)$ , and an episode horizon  $T$ . At each time step  $t$ , the agent observes  $s_t$ , samples or selects an action  $a_t$ , receives reward  $r_t$ , and transitions to  $s_{t+1}$ . The goal is to learn a policy  $\pi_\theta(a \mid s)$ , parameterized by  $\theta$ , that maximizes expected discounted return,

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[ \sum_{t=0}^{T-1} \gamma^t r_t \right]. \quad (2.1)$$

This interaction-based formulation makes reinforcement learning distinct from supervised learning because the data distribution depends on the policy being trained. Consequently, the agent must balance exploitation of currently rewarding actions with exploration of actions whose long-term consequences are uncertain.

Value functions summarize the long-term utility of states and actions. The state-value function of a policy is

$$V^\pi(s) = \mathbb{E}_\pi \left[ \sum_{k=0}^{T-t-1} \gamma^k r_{t+k} \mid s_t = s \right], \quad (2.2)$$

while the action-value function is

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[ \sum_{k=0}^{T-t-1} \gamma^k r_{t+k} \mid s_t = s, a_t = a \right]. \quad (2.3)$$

The difference  $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$  is the advantage function, which measures whether an action is better or worse than the policy's expected behavior at the same state. Advantage estimates are central in modern policy-gradient algorithms because they reduce variance while preserving the direction of policy improvement.

Deep reinforcement learning extends this framework by using neural networks to approximate policies, value functions, and auxiliary models. This approximation is necessary in continuous-control and high-dimensional settings where tabular represen-

tations are infeasible. However, function approximation also makes exploration more difficult. A neural policy may repeatedly visit a narrow subset of the state space if initial rewards favor locally stable behavior, and a value function may generalize poorly when the agent has not collected diverse experience. Sparse-reward environments make the problem more severe because most early transitions may have identical or nearly identical external rewards.

Reward design is therefore a central issue in reinforcement learning. Potential-based reward shaping can preserve optimal policies when it follows a restricted form, but arbitrary shaping terms may change the task objective (Ng et al., 1999). Intrinsic-motivation methods intentionally add such auxiliary shaping signals to improve exploration, especially when the original reward does not provide enough feedback during early training. These methods should therefore be evaluated not only by whether they increase the reward used for optimization, but also by whether they improve the extrinsic return of the learned policy.

The exploration problem addressed in this study lies at the intersection of sparse feedback, function approximation, and reward augmentation. An effective intrinsic reward should encourage the agent to collect informative transitions without permanently distracting policy optimization from the external task. This requirement motivates related work on novelty, prediction error, impact, and learning progress, each of which defines a different proxy for the value of experience.

## 2.2 Policy Gradient and PPO Methods

Policy-gradient methods optimize a parameterized policy directly by estimating the gradient of expected return with respect to the policy parameters. For a stochastic policy  $\pi_\theta(a \mid s)$ , the objective can be written as  $J(\theta) = \mathbb{E}_{\pi_\theta}[G_t]$ , where  $G_t$  denotes the discounted return from time  $t$ . The policy-gradient estimator uses sampled trajectories to increase the log-probability of actions that produce positive advantage estimates:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(a_t \mid s_t) A^{\pi_\theta}(s_t, a_t)]. \quad (2.4)$$

In practice,  $A^{\pi_\theta}$  is replaced by an empirical estimate computed from rewards and a learned value function. This direct optimization is well suited to continuous action spaces because the policy can represent a probability distribution over real-valued actions.

Generalized Advantage Estimation (GAE) improves the bias–variance tradeoff of



advantage computation by combining temporal-difference residuals over multiple horizons (Schulman et al., 2015). Let

$$\delta_t = r_t + \gamma V_\varphi(s_{t+1}) - V_\varphi(s_t) \quad (2.5)$$

be the one-step temporal-difference error for a value function  $V_\varphi$ . GAE forms the advantage estimate

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}, \quad (2.6)$$

where  $\lambda \in [0, 1]$  controls the interpolation between low-variance one-step estimates and higher-variance Monte Carlo returns. This mechanism is widely used in on-policy deep reinforcement learning because it stabilizes learning while allowing delayed rewards to influence earlier actions.

Proximal Policy Optimization (PPO) is a policy-gradient method designed to constrain each policy update so that optimization remains stable under minibatch stochastic gradient descent (Schulman et al., 2017). PPO uses the probability ratio

$$\varrho_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \quad (2.7)$$

between the new policy and the behavior policy that collected the data. The clipped surrogate objective is

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[ \min \left( \varrho_t(\theta) \hat{A}_t, \text{clip}(\varrho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (2.8)$$

where  $\epsilon$  limits the incentive to move the new policy too far from the old policy. The minimum prevents policy updates from obtaining large objective improvements by exceeding the clipping range in the direction favored by the advantage estimate.

PPO is commonly paired with a value-function loss and an entropy bonus. The value loss trains the critic to predict returns, while the entropy term encourages stochasticity and discourages premature policy collapse. The combined optimization problem is not a pure maximization of the clipped objective; it is a practical actor–critic training procedure that alternates between collecting on-policy rollouts and performing several optimization epochs over the collected batch. Because the data become stale after the policy changes, PPO remains fundamentally on-policy and typically requires repeated environment interaction.

The use of PPO is relevant for intrinsic motivation because reward augmentation

affects the advantage estimates used by the policy update. If the reward is changed to

$$r_t = r_t^{\text{ext}} + \eta_t r_t^{\text{int}}, \quad (2.9)$$

then the value targets and advantages reflect both task reward and intrinsic shaping. A well-designed intrinsic signal can therefore guide the policy toward useful experience before extrinsic rewards are frequent, but a misleading signal can also dominate the optimization process. PPO provides a stable backbone for comparing intrinsic rewards because the same policy optimizer can be used while varying only the exploration objective.

### 2.3 Intrinsic Motivation in Reinforcement Learning

Intrinsic motivation refers to internally generated objectives that encourage an agent to explore, learn, or acquire competence even when external reward is absent or insufficient. In reinforcement learning, intrinsic motivation is usually implemented by augmenting the environment reward with an auxiliary reward computed from the agent’s observations, actions, learned models, or memory (Barto, 2012; Oudeyer & Kaplan, 2007; Oudeyer et al., 2007; Singh et al., 2004). The augmented reward has the general form

$$r_t = r_t^{\text{ext}} + \eta_t r_t^{\text{int}}, \quad (2.10)$$

where  $r_t^{\text{ext}}$  is the task reward,  $r_t^{\text{int}}$  is the intrinsic reward, and  $\eta_t$  controls the strength of intrinsic shaping. The intrinsic term is not normally part of the original task definition; it is introduced to improve learning dynamics.

A central motivation for intrinsic reward is the sparse-reward problem. When successful behavior requires a long sequence of actions, random exploration may rarely discover reward, leaving policy-gradient estimates uninformative. An intrinsic reward can provide intermediate learning signals by rewarding experience that appears novel, surprising, controllable, or productive. The agent may then collect a more diverse set of trajectories and eventually encounter externally rewarding states. This role is especially important in environments where externally successful behavior is difficult to discover but easy to reinforce once discovered.

Intrinsic-motivation methods can be grouped by the proxy they use for exploration value. Novelty-based methods reward states or features that have been visited rarely. Information-gain methods reward transitions that reduce uncertainty in a learned model (Houthoofd et al., 2016). Prediction-error methods reward transitions that a predictive

model currently explains poorly (Pathak et al., 2017; Stadie et al., 2015). Learning-progress methods reward reductions in prediction error over time rather than the error itself (Baranes & Oudeyer, 2009; Oudeyer & Kaplan, 2007; Schmidhuber, 1991). Impact-driven methods reward actions that produce meaningful change in a learned representation (Raileanu & Rocktäschel, 2020). These categories are not mutually exclusive; a single method may combine several signals.

The theoretical status of intrinsic reward differs from policy-invariant reward shaping. Potential-based shaping has conditions under which the optimal policy is preserved, but general intrinsic-reward augmentation does not satisfy these conditions (Ng et al., 1999). An intrinsic reward may therefore change the effective optimization objective and produce behavior that is not optimal for the original task if the auxiliary signal remains influential after exploration is no longer needed. For this reason, empirical studies of intrinsic motivation typically report extrinsic evaluation return rather than the total shaped reward used during training.

A useful intrinsic reward should satisfy several practical requirements. First, it should produce a meaningful signal before external rewards are common. Second, it should scale across environments without requiring extensive task-specific tuning. Third, it should not reward unlearnable randomness indefinitely. Fourth, it should be compatible with the policy optimizer and observation representation used by the agent. These requirements are difficult to satisfy simultaneously because signals that are strong enough to guide early exploration may also be strong enough to misdirect the policy later in training.

The literature therefore motivates a move from simple surprise toward more selective measures of exploration value. A transition is not necessarily useful because it is rare or poorly predicted; it is useful when it helps the agent acquire predictive or control-relevant competence. The following sections review the major families of intrinsic rewards that inform this distinction.

## 2.4 Novelty-Based Exploration

Novelty-based exploration encourages the agent to visit states or features that have been encountered infrequently. In tabular settings, a simple exploration bonus can be defined from a visitation count  $N(s)$ , for example

$$r_t^{\text{int}} = \frac{\beta}{\sqrt{N(s_t)}}, \quad (2.11)$$

where  $\beta > 0$  controls the magnitude of the bonus. The reward is large when a state is new and decreases as the state is revisited. This mechanism promotes broad coverage of the state space and can help an agent discover sparse external rewards.

Direct state counting is not feasible in high-dimensional or continuous observation spaces because exact revisits are rare. Count-based exploration has therefore been extended through density models, hashing, and learned abstractions. Pseudo-count methods estimate how much a density model’s probability for an observation changes after training on that observation, thereby generalizing count-based bonuses beyond discrete tables (Bellemare et al., 2016). Hash-based approaches discretize learned or hand-designed features so that approximate counts can be maintained efficiently in deep reinforcement learning (Tang et al., 2016). Neural density models similarly estimate novelty by assigning low probability to unfamiliar observations (Ostrovski et al., 2017).

Novelty can also be estimated episodically rather than across the entire training history. Episodic curiosity mechanisms compare the current observation with observations already encountered in the same episode and reward states that are not easily reachable from recent memory (Savinov et al., 2018). Directed exploration methods can combine episodic and lifelong novelty signals so that agents continue seeking unfamiliar states while also avoiding repeated short-term behavior loops (Badia et al., 2020). Related self-supervised exploration methods have also used model disagreement as a signal for unfamiliar or informative transitions (Pathak et al., 2019). These methods recognize that exploration has multiple time scales: an agent should avoid immediate repetition within an episode, but it should also learn a long-term representation of what parts of the environment remain unfamiliar.

Novelty-based methods are attractive because they are conceptually simple and do not require the environment reward to be informative. They are particularly effective when task success is correlated with reaching rarely visited regions. However, novelty is an imperfect proxy for usefulness. A state can be novel but irrelevant to the task, and repeated visits to an initially novel region may stop receiving reward even if the agent has not yet learned a useful skill there. Conversely, a familiar region may still support important improvement if the agent is learning a precise control strategy.

Another limitation is that novelty depends strongly on the representation in which counts or densities are computed. A poor representation may treat irrelevant observation differences as novel or merge distinct control-relevant states into the same region. In continuous-control tasks, small changes in position or velocity can make exact observations appear unique even when they belong to the same behavioral regime. This can

cause novelty bonuses to remain high without reflecting genuine exploration progress.

Novelty-based exploration is therefore a useful foundation but not a complete solution to sparse-reward reinforcement learning. It addresses where the agent has not been, but it does not directly measure whether the agent is learning from the region, whether the transition is controllable, or whether the experience is likely to improve future task performance. These limitations motivate prediction-error, impact-driven, and learning-progress approaches, which attempt to define more informative exploration signals.

## 2.5 Prediction-Error Curiosity

Prediction-error curiosity rewards transitions that are difficult for a learned model to predict. The general assumption is that high prediction error indicates novelty, surprise, or incomplete understanding of the environment. If a model predicts the next representation  $\hat{z}_{t+1} = f_\psi(z_t, a_t)$  from the current representation  $z_t$  and action  $a_t$ , an intrinsic reward can be defined as

$$r_t^{\text{int}} = \|f_\psi(z_t, a_t) - z_{t+1}\|_2^2. \quad (2.12)$$

Early deep predictive approaches used this idea to encourage agents to seek transitions that improve their understanding of environment dynamics (Stadie et al., 2015).

The Intrinsic Curiosity Module (ICM) is a representative feature-space prediction-error method (Pathak et al., 2017). ICM learns an encoder  $\phi_\omega$  that maps observations to features  $z_t = \phi_\omega(o_t)$ , a forward model that predicts  $z_{t+1}$  from  $(z_t, a_t)$ , and an inverse model that predicts  $a_t$  from  $(z_t, z_{t+1})$ . The inverse model encourages the representation to retain information about controllable aspects of the transition, reducing sensitivity to observation details that do not depend on the agent’s action. The curiosity reward is then based on forward prediction error in this learned feature space rather than raw observation space.

Random Network Distillation (RND) provides another influential form of prediction-error curiosity (Burda, Edwards, Storkey, & Klimov, 2018). RND uses a fixed random target network  $h$  and a trainable predictor  $\hat{h}_\psi$ . The intrinsic reward is the prediction error

$$r_t^{\text{int}} = \|\hat{h}_\psi(o_t) - h(o_t)\|_2^2. \quad (2.13)$$

Because the target network is fixed, prediction error tends to be high for unfamiliar observations and decreases as the predictor trains on visited states. Large-scale studies

of curiosity-driven learning have shown that such prediction-error signals can support exploration across a range of environments (Burda, Edwards, Pathak, et al., 2018).

Prediction-error curiosity is powerful because it does not require explicit state counting and can be implemented with neural networks. It can discover sparse rewards by making unfamiliar observations attractive, and it can be combined with standard policy-gradient methods through reward augmentation. However, its main weakness is that prediction error conflates learnable novelty with irreducible uncertainty. A transition may remain difficult to predict because it is stochastic, partially observed, chaotic, or unrelated to the agent’s controllable behavior. In such cases, prediction error can remain high even after extensive experience.

This weakness is closely related to the noisy-TV problem. If an agent can repeatedly trigger observations that are unpredictable but behaviorally unproductive, a prediction-error bonus may reward this behavior indefinitely. Feature learning and inverse dynamics can reduce the problem by emphasizing controllable factors, but they do not eliminate all sources of unlearnable error. RND can also be affected by observation streams that maintain high predictor error for reasons unrelated to task progress.

Prediction-error methods therefore motivate the use of temporal improvement as a refinement of curiosity. High error alone answers the question of what the model currently fails to predict; it does not answer whether additional data are making the model better. Learning-progress methods address this distinction by rewarding reductions in error rather than the absolute error value.

## 2.6 Impact-Driven Exploration

Impact-driven exploration rewards actions that produce meaningful change in the environment or in a learned representation. The motivation is that useful exploration should not only reach unfamiliar observations, but should also involve transitions that the agent can influence. A simple feature-space impact measure is

$$I_t = \|\phi(o_{t+1}) - \phi(o_t)\|_2, \quad (2.14)$$

where  $\phi$  maps observations into a representation space. Large values indicate that the transition changed the representation substantially.

Rewarding Impact-Driven Exploration (RIDE) uses this principle to encourage agents to take actions that lead to significant feature-space changes (Raileanu & Rocktäschel, 2020). RIDE combines a learned representation with an impact reward

and an episodic state-count correction. The count correction reduces the reward for repeatedly producing the same transition pattern, while the impact term encourages behavior that changes the agent's represented state. This differs from pure novelty because the reward is tied to transition magnitude rather than only to whether a state has been visited rarely.

Impact rewards are useful in environments where the agent must learn to control movement, manipulate objects, or otherwise cause structured changes before reaching task reward. For example, in locomotion tasks, an agent that remains stationary may avoid penalties or unstable dynamics but will not learn useful movement. A feature-space impact term can provide pressure toward dynamic behavior even when extrinsic reward is initially weak. In sparse-reward tasks, impact can help the agent escape local regions of the state space by valuing transitions that produce substantial state changes.

However, impact alone is not sufficient to define productive exploration. A transition can have high impact while being harmful, random, or disconnected from task success. Large representational changes may occur because of falls, collisions, or other unstable behavior that does not improve competence. Furthermore, if the learned representation is poorly calibrated, the impact signal may emphasize dimensions that are easy to change but not useful for solving the task. Impact-driven exploration therefore benefits from being combined with additional criteria that indicate whether the resulting experience supports learning.

Impact and learning progress are complementary. Impact identifies transitions that change the representation, while learning progress identifies regions where prediction or understanding is improving. A combined intrinsic reward can prefer transitions that are both consequential and learnable. This distinction is important because an agent may otherwise learn to seek either low-impact predictable transitions that show small model improvement or high-impact transitions that remain unproductive. The literature on impact-driven exploration therefore provides a key component for exploration objectives that seek to connect behavioral change with model improvement.

## **2.7 Learning-Progress Exploration**

Learning-progress exploration rewards improvement in the agent's predictive or control model rather than rewarding novelty or prediction error directly. The idea appears in early work on curiosity and model-building controllers, where an agent is motivated by changes in its ability to predict the consequences of its actions (Schmidhuber, 1991). Under this view, experience is valuable when it reduces uncertainty or error. A region

that is already predictable is no longer interesting, while a region that remains unpredictable despite repeated experience is also less useful. The most valuable region is one where the agent is actively improving.

A simple learning-progress signal can be expressed as a positive decrease in prediction error,

$$LP_t = \max(0, \bar{e}_{\text{past}} - \bar{e}_{\text{recent}}), \quad (2.15)$$

where  $\bar{e}_{\text{past}}$  is a longer-term estimate of model error and  $\bar{e}_{\text{recent}}$  is a recent estimate. When recent error is lower than the longer-term baseline, the model appears to be improving. This formulation differs from prediction-error curiosity because a region with high but nondecreasing error receives little or no reward. It also differs from novelty because a frequently visited region can remain interesting if the agent is still learning from it.

Oudeyer and Kaplan describe intrinsic motivation systems that use competence progress and learning progress to guide autonomous development (Oudeyer & Kaplan, 2007; Oudeyer et al., 2007). Such systems divide the agent’s experience into regions or tasks and allocate exploration toward areas where progress is greatest. This creates an automatic curriculum: the agent first focuses on regions that are learnable under its current competence, then shifts attention as those regions become mastered. The curriculum property is especially relevant to reinforcement learning because complex behavior often requires learning intermediate skills before final task rewards become reachable.

R-IAC applies this principle through adaptive partitioning and region-specific progress estimates (Baranes & Oudeyer, 2009). The environment or goal space is partitioned into regions, and learning progress is computed locally from changes in prediction error. Regions with higher progress are sampled more often, while regions with low progress become less attractive. The locality of the estimate is important because different parts of an environment may have different dynamics. A global error estimate could hide improvement in one region behind stagnation in another.

Learning-progress methods directly address a major limitation of prediction-error curiosity, but they introduce practical challenges. First, progress estimates are non-stationary because the model and policy change during training. Second, progress must be computed over a suitable region structure; overly coarse regions mix unrelated dynamics, while overly fine regions lack enough data for stable estimates. Third, the magnitude of progress depends on the scale of the prediction error and may require normalization. Fourth, learning progress alone does not necessarily measure whether a transition produces meaningful behavioral change. A model can improve on small or



repetitive transitions that have limited value for exploration.

These challenges motivate learning-progress designs that operate in learned latent spaces, estimate progress locally, and combine progress with transition impact. Such designs retain the main advantage of learning progress—distinguishing learnable uncertainty from unproductive surprise—while making the signal more compatible with deep reinforcement learning.

## **2.8 Unproductive Curiosity and Noisy Dynamics**

Unproductive curiosity occurs when an intrinsic reward repeatedly attracts the agent toward experience that is surprising or novel but does not improve task performance or predictive competence. The most common example is the noisy-TV scenario, in which an agent can generate unpredictable observations that produce high curiosity reward without yielding useful control knowledge (Mavor-Parker et al., 2021). The problem illustrates a general limitation of intrinsic rewards based on novelty or prediction error: high uncertainty is not always reducible uncertainty.

Noisy dynamics can arise for several reasons. The environment may contain stochastic transitions, random visual features, contact instability, partial observability, or dynamics that are too complex for the current model class. In these cases, a learned predictor may maintain high error even after extensive training. If the intrinsic reward is proportional to prediction error, the agent receives a persistent incentive to revisit the region. This effect can be especially harmful in sparse-reward tasks because the intrinsic reward may dominate the early training signal before the agent has discovered reliable extrinsic reward.

Novelty-based rewards can also produce unproductive exploration. A density model or count estimator may treat irrelevant variation as new, causing the agent to chase observation differences that do not correspond to useful states. In continuous spaces, small but behaviorally insignificant differences can appear novel unless the representation abstracts them away. Episodic novelty mechanisms reduce immediate repetition, but they do not fully determine whether experience contributes to long-term competence.

Several design strategies can reduce unproductive curiosity. One strategy is to learn representations that focus on controllable aspects of the environment, as in inverse-dynamics-based curiosity (Pathak et al., 2017). Another is to estimate uncertainty types and avoid rewarding aleatoric uncertainty, which corresponds to irreducible randomness (Mavor-Parker et al., 2021). A third strategy is to use learning progress so

that only error reduction, rather than error itself, produces reward (Baranes & Oudeyer, 2009; Schmidhuber, 1991). A fourth strategy is to combine multiple signals, such as novelty and impact, so that a single failure mode does not dominate exploration.

Reward scheduling is also relevant. If intrinsic shaping remains strong throughout training, the learned policy may continue optimizing an auxiliary objective after the task reward is sufficient. Reducing the intrinsic coefficient over time can make curiosity act as an early exploration aid rather than a permanent substitute for the task objective. However, scheduling alone does not identify which specific regions are unproductive; it only reduces the global strength of the intrinsic term. A local mechanism can be more selective by suppressing reward only where the agent observes high error without learning progress.

The noisy-dynamics problem therefore motivates exploration objectives with explicit safeguards. An intrinsic reward should distinguish between learnable and unlearnable uncertainty, should avoid overvaluing uncontrollable observation changes, and should preserve the role of extrinsic reward in final policy performance. These considerations are central to the research gap addressed by combining region-local learning progress, feature-space impact, and gated suppression of high-error low-progress regions.

## **2.9 Research Gap**

The reviewed literature establishes several important principles for exploration in reinforcement learning. Novelty-based methods encourage state-space coverage and can guide agents toward sparse rewards, but they do not directly measure whether the agent is learning from the visited states. Prediction-error curiosity provides a neural and representation-based form of surprise, but it can reward stochastic or unlearnable regions. Impact-driven exploration encourages controllable change, but impact alone does not determine whether the resulting experience improves the agent’s model or policy. Learning-progress methods address the difference between surprise and improvement, but they require stable local estimates and an appropriate region structure in high-dimensional or continuous spaces.

A remaining gap is the need for an intrinsic-reward formulation that combines these complementary strengths while reducing their individual failure modes. In particular, an exploration signal for deep reinforcement learning should satisfy four requirements. First, it should operate in a learned representation so that it can be used with continuous observations and neural policies. Second, it should value transitions that produce meaningful feature-space change, since exploration should involve behaviorally con-

sequential action. Third, it should estimate learning progress locally, because different parts of the environment may become predictable at different rates. Fourth, it should suppress intrinsic reward in regions where prediction error remains high but learning progress remains low.

Existing methods address parts of this set of requirements but not the full combination. ICM and RND provide scalable prediction-error curiosity, but their intrinsic rewards are primarily driven by error magnitude (Burda, Edwards, Storkey, & Klimov, 2018; Pathak et al., 2017). RIDE introduces a feature-space impact signal and count-based modulation, but it does not directly use local model-improvement estimates as the main criterion for productive exploration (Raileanu & Rocktäschel, 2020). R-IAC uses region-local learning progress, but adapting this idea to deep reinforcement learning requires learned latent representations, stable scale handling, and integration with modern policy-gradient training (Baranes & Oudeyer, 2009). The noisy-TV literature further shows that high-error regions require explicit treatment when curiosity signals are used for policy optimization (Mavor-Parker et al., 2021).

This study addresses the gap by evaluating an intrinsic-reward family based on feature-space impact and region-local learning progress. The base reward combines the magnitude of latent transition change with an estimate of local model improvement. The gated variant further disables intrinsic shaping in regions that satisfy a high-error low-progress criterion, thereby treating unproductive curiosity as a local property rather than only as a global scheduling problem. This design follows the literature’s motivation for curiosity, impact, and progress, while focusing on their interaction within a PPO-based deep reinforcement-learning setting.

The research gap is therefore not the absence of intrinsic motivation methods, but the need for a selective intrinsic reward that distinguishes useful exploration from persistent surprise. The combination of impact, local progress, and gating clarifies how intrinsic rewards can be made more reliable under sparse rewards, noisy dynamics, and continuous-control learning.

## CHAPTER 3

### THEORETICAL FRAMEWORK

#### 3.1 Problem Formulation

The study is formulated within an episodic reinforcement-learning setting. An environment is represented as a Markov decision process with observation space  $\mathcal{O}$ , action space  $\mathcal{A}$ , transition distribution  $P(o_{t+1} \mid o_t, a_t)$ , extrinsic reward function  $r^{\text{ext}}(o_t, a_t, o_{t+1})$ , discount factor  $\gamma \in [0, 1)$ , and finite episode horizon  $H$ . At each time step  $t$ , the agent receives an observation  $o_t \in \mathcal{O}$ , samples an action  $a_t \sim \pi_\theta(\cdot \mid o_t)$  from a stochastic policy, receives an extrinsic reward  $r_t^{\text{ext}}$ , and transitions to  $o_{t+1}$ . The notation uses observations rather than privileged environment states because the exploration mechanism is defined only from information available to the agent.

The primary task objective is the maximization of expected discounted extrinsic return,

$$J_{\text{ext}}(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[ \sum_{t=0}^{H-1} \gamma^t r_t^{\text{ext}} \right], \quad (3.1)$$

where  $\tau = (o_0, a_0, r_0^{\text{ext}}, o_1, \dots)$  denotes a trajectory induced by the policy and environment. This objective defines the performance criterion of the task. Intrinsic rewards are introduced only as an auxiliary training signal for exploration and do not replace extrinsic return as the quantity used to evaluate learned behavior.

Sparse-reward and delayed-reward tasks make direct optimization of  $J_{\text{ext}}$  difficult because many early trajectories contain little information about useful behavior. The theoretical role of intrinsic motivation is to modify the training signal so that the policy visits informative parts of the observation–action space before extrinsic reward becomes frequent. The resulting optimization process is therefore governed by a shaped reward during training, while the success of the learned policy is judged by the original environment reward. This distinction is important because arbitrary reward shaping does not generally preserve optimal policies, unlike restricted potential-based shaping forms (Ng et al., 1999).

The central problem addressed by the framework is to define an intrinsic reward  $r_t^{\text{int}}$  that rewards productive exploration rather than persistent surprise. A productive transition is characterized by two properties. First, it should produce a meaningful change in a learned feature representation, indicating that the action affected the agent’s represented state. Second, it should occur in a region where the predictive model is

improving, indicating reducible uncertainty rather than irreducible noise. The following sections formalize these properties through latent dynamics, feature-space impact, region-local learning progress, and a gate that suppresses intrinsically rewarding but unproductive regions.

### 3.2 Reward Augmentation

During training, the policy optimizer receives a total reward formed by augmenting the environment reward with a clipped intrinsic term:

$$r_t = r_t^{\text{ext}} + \eta_t \text{clip}(r_t^{\text{int}}, -r_{\max}, r_{\max}), \quad (3.2)$$

where  $\eta_t \geq 0$  controls the strength of intrinsic shaping and  $r_{\max} > 0$  bounds the magnitude of the intrinsic contribution. The clipping operation prevents unusually large intrinsic values from dominating the advantage estimates used by the policy update. For environment-terminal transitions, the intrinsic term is set to zero so that episode termination itself is not treated as an intrinsically rewarding event.

The augmented training objective can be written as

$$J_{\text{aug}}(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[ \sum_{t=0}^{H-1} \gamma^t (r_t^{\text{ext}} + \eta_t \bar{r}_t^{\text{int}}) \right], \quad (3.3)$$

where  $\bar{r}_t^{\text{int}} = \text{clip}(r_t^{\text{int}}, -r_{\max}, r_{\max})$ . Policy-gradient updates such as PPO optimize this augmented objective through advantage estimates computed from  $r_t$  (Schulman et al., 2017). Consequently, the intrinsic term affects the data distribution collected by the policy, the value targets used by the critic, and the direction of policy improvement.

The coefficient  $\eta_t$  may be constant or scheduled over the course of training. A time-varying coefficient is useful when intrinsic rewards are intended to encourage early exploration while allowing the extrinsic task objective to dominate later learning. Let  $p_t \in [0, 1]$  denote normalized training progress. A generic schedule can be represented as  $\eta_t = \eta w(p_t)$ , where  $\eta$  is a base scale and  $w(p_t) \in [0, 1]$  is a nonincreasing weighting function. The theoretical framework only requires  $\eta_t$  to be nonnegative and bounded.

Reward augmentation introduces an explicit tradeoff. If  $\eta_t$  is too small, the intrinsic reward may not change exploration behavior in sparse-reward regions. If  $\eta_t$  is too large, the agent may optimize the auxiliary signal at the expense of the task reward. The subsequent components are therefore designed so that  $r_t^{\text{int}}$  is not merely large in novel or unpredictable states, but large primarily when the transition indicates controllable

feature-space change and local model improvement.

### 3.3 Latent Representation Learning

The intrinsic reward is computed in a learned latent space rather than directly in the observation space. Let

$$\phi_\omega : \mathcal{O} \rightarrow \mathbb{R}^d \quad (3.4)$$

be an encoder with parameters  $\omega$ , and define the latent representation of an observation as

$$z_t = \phi_\omega(o_t). \quad (3.5)$$

The dimension  $d$  is fixed during training. The representation  $z_t$  serves as the common space in which transition impact, prediction error, and local learning progress are measured.

Using a latent representation has two theoretical advantages. First, it allows the exploration signal to operate on compact features rather than on raw observations, which may include irrelevant or high-variance components. Second, it allows the representation to be shaped by predictive and inverse-dynamics objectives, so that the measured changes are more closely related to action-dependent structure. This follows the motivation of self-supervised curiosity methods, where learned features help focus prediction error on aspects of the environment that are affected by the agent’s behavior (Pathak et al., 2017).

The encoder is trained jointly with auxiliary dynamics models using transitions collected from the current policy. For a batch of transitions  $\mathcal{B}$ , the representation-learning objective is induced by the dynamics loss

$$\min_{\omega, \psi, \xi} \frac{1}{|\mathcal{B}|} \sum_{(o_t, a_t, o_{t+1}) \in \mathcal{B}} \mathcal{L}_{\text{dyn}}(t), \quad (3.6)$$

where  $\psi$  parameterizes the forward model and  $\xi$  parameterizes the inverse model. Because the batch distribution depends on  $\pi_\theta$ , the latent space changes as the policy discovers new regions of the environment. The exploration signal is therefore nonstationary by construction.

The framework does not assume that  $z_t$  is a sufficient statistic for the full environment state. Instead, it assumes that  $z_t$  is sufficiently informative for the auxiliary quantities used by the intrinsic reward. In particular, the feature difference  $z_{t+1} - z_t$  should reflect meaningful transition impact, and the forward prediction error in feature space

should reflect how well the local latent dynamics are being learned. These assumptions are weaker than full state reconstruction and are suitable for reinforcement-learning settings where only behaviorally relevant predictive structure is needed for exploration.

### 3.4 Forward and Inverse Dynamics Models

The latent dynamics module contains a forward predictor and an inverse predictor. The forward predictor

$$f_\psi : \mathbb{R}^d \times \mathcal{A} \rightarrow \mathbb{R}^d \quad (3.7)$$

receives the current latent representation and action, then predicts the next latent representation. The inverse predictor

$$g_\xi : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathcal{D}(\mathcal{A}) \quad (3.8)$$

receives consecutive latent representations and predicts a distribution over the action that caused the transition. Here  $\mathcal{D}(\mathcal{A})$  denotes an action-distribution family. For discrete action spaces, this family is a categorical distribution over actions. For continuous action spaces, the inverse model is represented as a diagonal Gaussian likelihood over real-valued actions.

The per-transition forward loss is the mean-squared prediction error in feature space,

$$\mathcal{L}_{\text{fwd}}(t) = \frac{1}{d} \|f_\psi(z_t, a_t) - z_{t+1}\|_2^2. \quad (3.9)$$

This quantity is also used as the transition-level prediction error,

$$e_t = \mathcal{L}_{\text{fwd}}(t). \quad (3.10)$$

The scalar  $e_t$  is not used directly as the final intrinsic reward, because high prediction error alone may correspond to stochastic or unlearnable dynamics. Instead, it is used to estimate learning progress inside local regions of the latent space.

The inverse loss depends on the action space. For a discrete action  $a_t$ , the inverse model produces logits over actions and is trained with cross-entropy,

$$\mathcal{L}_{\text{inv}}(t) = -\log g_\xi(a_t \mid z_t, z_{t+1}). \quad (3.11)$$

For a continuous action  $a_t \in \mathbb{R}^m$ , the inverse model produces a mean vector  $\mu_\xi(z_t, z_{t+1})$  and diagonal log-standard-deviation vector  $\ell_\xi(z_t, z_{t+1})$ . With  $\sigma_\xi = \exp(\ell_\xi)$ , the negative

log-likelihood is

$$\mathcal{L}_{\text{inv}}(t) = \frac{1}{2} \sum_{j=1}^m \left[ \frac{(a_{t,j} - \mu_{\xi,j})^2}{\sigma_{\xi,j}^2} + 2\ell_{\xi,j} + \log(2\pi) \right]. \quad (3.12)$$

The combined dynamics loss is

$$\mathcal{L}_{\text{dyn}}(t) = \beta_{\text{fwd}} \mathcal{L}_{\text{fwd}}(t) + \beta_{\text{inv}} \mathcal{L}_{\text{inv}}(t), \quad (3.13)$$

where  $\beta_{\text{fwd}} > 0$  and  $\beta_{\text{inv}} > 0$  are weighting coefficients. The forward term supplies a predictive error signal for learning-progress estimation, while the inverse term discourages representations that ignore action-relevant information. This combination supports an intrinsic reward based on model improvement without making raw prediction error the sole driver of exploration.

### 3.5 Feature-Space Impact

Feature-space impact measures how much a transition changes the learned latent representation. For transition  $(o_t, a_t, o_{t+1})$ , the impact term is defined as

$$I_t = \|z_{t+1} - z_t\|_2. \quad (3.14)$$

A transition with small  $I_t$  leaves the representation nearly unchanged, while a transition with large  $I_t$  moves the agent to a different region of latent space. This quantity is related to impact-driven exploration, where exploration is encouraged through meaningful changes in learned features rather than only through novelty counts (Raileanu & Rocktäschel, 2020).

Impact is not equivalent to reward, novelty, or prediction error. A transition can be novel but have little behavioral consequence, such as a small perturbation in an irrelevant observation component. A transition can also have high prediction error because of stochasticity while not producing a useful change in controllable state. The role of  $I_t$  is to favor actions that induce represented state change, thereby complementing the learning-progress signal defined from prediction-error reduction.

The use of a learned feature space is essential to the definition. If impact were computed directly in raw observation space, visually or numerically large differences could dominate the signal even when they are not relevant to control. Computing impact in  $\mathbb{R}^d$  instead makes the signal depend on the representation learned by the dynamics mod-



ule. The inverse-model component of the representation objective further encourages the feature space to retain information about action-dependent transitions.

Because the scale of  $I_t$  can vary across environments and throughout training, impact is treated as a component of a normalized intrinsic score rather than as a standalone reward. Let  $v_I$  be an exponential moving estimate of the second moment of  $I_t$ ,

$$v_I \leftarrow \beta_{\text{rms}} v_I + (1 - \beta_{\text{rms}}) I_t^2, \quad (3.15)$$

with  $0 < \beta_{\text{rms}} < 1$ . The normalized impact component is

$$\tilde{I}_t = \frac{I_t}{\sqrt{v_I + \varepsilon}}, \quad (3.16)$$

where  $\varepsilon > 0$  prevents division by zero. This normalization makes the impact term comparable to the learning-progress term in the combined intrinsic reward.

### 3.6 Region-Local Learning Progress

Learning progress measures improvement rather than surprise. The framework estimates learning progress locally because different parts of the latent space may become predictable at different rates. Let  $\mathcal{P}_t$  denote the current partition of the latent space into regions, and let

$$\rho_t = \rho(z_t) \in \mathcal{P}_t \quad (3.17)$$

be the region assigned to  $z_t$ . The partition is an online binary space partition over  $\mathbb{R}^d$  whose leaves define the current regions. As more points are observed, frequently visited leaves may be split so that local statistics can become more specific. This principle follows the learning-progress motivation of competence-based exploration and R-IAC, where regions are valued according to improvement rather than absolute error (Baranes & Oudeyer, 2009; Schmidhuber, 1991).

For each region  $r$ , two exponential moving averages of the forward prediction error are maintained: a long-horizon average  $\mu_r^{\text{long}}$  and a short-horizon average  $\mu_r^{\text{short}}$ . When a transition assigned to region  $r$  has prediction error  $e_t$ , the averages are updated as

$$\mu_r^{\text{long}} \leftarrow \beta_{\text{long}} \mu_r^{\text{long}} + (1 - \beta_{\text{long}}) e_t, \quad (3.18)$$

$$\mu_r^{\text{short}} \leftarrow \beta_{\text{short}} \mu_r^{\text{short}} + (1 - \beta_{\text{short}}) e_t, \quad (3.19)$$

where  $0 < \beta_{\text{short}} < \beta_{\text{long}} < 1$ . The short-horizon average responds more quickly to recent prediction errors, while the long-horizon average provides a slower baseline. On

the first visit to a region, both averages are initialized to the current prediction error.

The learning progress of a region is defined as the positive decrease from the long-horizon error estimate to the short-horizon error estimate:

$$\text{LP}(r) = \max(0, \mu_r^{\text{long}} - \mu_r^{\text{short}}). \quad (3.20)$$

For a transition at time  $t$ , the local learning-progress component is

$$\text{LP}_t = \text{LP}(\rho_t). \quad (3.21)$$

The value is high when recent prediction error in the region has fallen below its longer-term baseline, which indicates that the dynamics model is improving in that part of latent space.

This definition addresses a limitation of prediction-error curiosity. A region with persistently high error but no reduction has low learning progress, whereas a region with moderate error that is decreasing can have high learning progress. The signal therefore favors reducible uncertainty, not merely large error. However, learning progress alone may reward improvement on transitions that have little behavioral effect. For this reason, it is combined with feature-space impact.

As with impact, the scale of learning progress is normalized by a running second-moment estimate. Let

$$v_{\text{LP}} \leftarrow \beta_{\text{rms}} v_{\text{LP}} + (1 - \beta_{\text{rms}}) \text{LP}_t^2. \quad (3.22)$$

The normalized learning-progress component is

$$\widetilde{\text{LP}}_t = \frac{\text{LP}_t}{\sqrt{v_{\text{LP}} + \varepsilon}}. \quad (3.23)$$

Independent normalization of  $I_t$  and  $\text{LP}_t$  prevents one component from dominating solely because of scale differences.

### 3.7 Gated Learning-Progress Exploration

The base intrinsic score combines normalized feature-space impact and normalized region-local learning progress:

$$u_t = \alpha_{\text{impact}} \widetilde{I}_t + \alpha_{\text{LP}} \widetilde{\text{LP}}_t, \quad (3.24)$$

where  $\alpha_{\text{impact}} \geq 0$  and  $\alpha_{\text{LP}} \geq 0$  determine the relative contribution of the two components. The ungated form of the intrinsic reward is simply

$$r_t^{\text{int}} = u_t. \quad (3.25)$$

This form rewards transitions that both alter the learned representation and occur in regions with recent model improvement.

The gated form adds a binary region-specific variable  $g_r \in \{0, 1\}$  for each visited region  $r$ . The intrinsic reward becomes

$$r_t^{\text{int}} = g_{\rho_t} u_t. \quad (3.26)$$

A gate value of one allows the base intrinsic score to shape the policy update, while a gate value of zero suppresses intrinsic reward for transitions assigned to that region. The purpose of the gate is not to remove difficult regions generally, but to suppress regions that appear unproductive: high prediction error persists while local learning progress remains low.

Let  $\mathcal{R}_t$  be the set of regions that have received at least one transition. The gate uses robust global references computed from the current region statistics:

$$\text{LP}_{\text{med}} = \text{median}_{r \in \mathcal{R}_t} \text{LP}(r), \quad (3.27)$$

$$e_{\text{med}} = \text{median}_{r \in \mathcal{R}_t} \mu_r^{\text{short}}. \quad (3.28)$$

The learning-progress threshold is

$$\tau_{\text{LP}} = \kappa \text{LP}_{\text{med}}, \quad (3.29)$$

where  $\kappa > 0$  is a multiplier. The normalized error score for region  $r$  is

$$s_r = \frac{\mu_r^{\text{short}}}{e_{\text{med}} + \varepsilon}. \quad (3.30)$$

A region is classified as unproductive on a visit when

$$\text{LP}(r) < \tau_{\text{LP}} \quad \text{and} \quad s_r > \tau_s, \quad (3.31)$$

where  $\tau_s > 0$  is a high-error threshold.

The gate update includes persistence and hysteresis. Gating is applied only after at least  $R_{\text{min}}$  regions have been visited, so that the median reference values are meaning-

ful. If fewer than  $R_{\min}$  regions have statistics, all gates remain enabled. When gating is active and a region with  $g_r = 1$  satisfies the unproductive condition for  $K$  consecutive visits, the gate is set to zero. When  $g_r = 0$ , the gate is re-enabled only after the region satisfies

$$\text{LP}(r) > h\tau_{\text{LP}} \quad (3.32)$$

for two consecutive visits, with hysteresis multiplier  $h > 1$ . The stricter reactivation criterion reduces rapid switching caused by noise in the progress estimate.

The gated formulation treats unproductive curiosity as a local property. A single stochastic or poorly learnable region can be suppressed without reducing intrinsic motivation everywhere else. This differs from a purely global intrinsic-reward schedule, which can only reduce the overall strength of curiosity. The theoretical expectation is that the ungated reward provides a low-overhead exploration signal, while the gated reward is most useful when high-error low-progress regions would otherwise attract the policy repeatedly.

### 3.8 Theoretical Assumptions

The framework relies on several assumptions that define the intended interpretation of the intrinsic reward. These assumptions do not guarantee global optimality of the learned policy, but they specify the conditions under which feature-space impact, learning progress, and gating are expected to provide useful exploration guidance.

First, the environment is assumed to provide bounded episodic interaction. Extrinsic rewards, intrinsic rewards after clipping, and episode horizons are finite, so the discounted returns used by the policy optimizer are well defined. This assumption is necessary because the augmented reward in Equation 3.2 changes the value targets and advantages used during training.

Second, the learned representation is assumed to preserve action-relevant transition structure. The latent vector  $z_t$  need not reconstruct the observation or represent the full environment state, but distances in latent space should be meaningful enough for  $\|z_{t+1} - z_t\|_2$  to indicate transition impact. The inverse-dynamics objective supports this assumption by encouraging features that retain information about actions that caused observed transitions.

Third, the forward prediction error is assumed to be a usable proxy for local model competence. A decreasing forward error in a region is interpreted as evidence that the model is learning the local dynamics. The framework does not assume that all uncertainty is reducible. Instead, it explicitly allows regions with high error and low

progress, which are candidates for gate suppression.

Fourth, region-local statistics are assumed to receive enough samples to become meaningful. The online partition should be neither so coarse that unrelated dynamics are mixed nor so fine that most regions lack sufficient data. The capacity and maximum-depth parameters of the partition control this tradeoff. The requirement of a minimum number of visited regions before gating further prevents early median estimates from controlling exploration prematurely.

Fifth, the exponential moving averages are assumed to operate on a slower time scale than individual transition noise. The inequality  $\beta_{\text{short}} < \beta_{\text{long}}$  ensures that the short-horizon estimate reacts faster than the long-horizon estimate. Learning progress is therefore interpreted as a relative improvement signal, not as an absolute measure of environment difficulty.

Sixth, component-wise normalization is assumed to stabilize relative scaling without changing the qualitative meaning of the components. The normalized quantities  $\tilde{I}_t$  and  $\tilde{LP}_t$  are scale-adjusted versions of impact and learning progress. The weights  $\alpha_{\text{impact}}$  and  $\alpha_{\text{LP}}$  then determine the intended tradeoff rather than compensating for arbitrary units.

Finally, the framework assumes that intrinsic reward is a training aid rather than a separate task objective. The augmented objective may alter the policy optimization path and is not claimed to preserve the optimal policy of the original MDP in the strict reward-shaping sense. The method is therefore evaluated by whether the resulting policy achieves higher or more reliable extrinsic return, not by the magnitude of accumulated intrinsic reward.

## CHAPTER 4

### METHODOLOGY

#### 4.1 Research Design

This study used a quantitative experimental research design to evaluate intrinsic-reward methods for reinforcement learning. The experiment compared a proposed intrinsic-reward family, Gated Learning-Progress Exploration (GLPE), against standard and intrinsic-motivation baselines under a fixed Proximal Policy Optimization (PPO) training backbone. The independent variable was the exploration method used during training. The dependent variables were extrinsic episodic return, sample efficiency, threshold-reaching reliability, and wall-clock efficiency. All reported performance measurements were based on environment reward only; intrinsic reward was used only as a training signal.

The design consisted of three stages. First, the intrinsic reward was specified as a transition-level signal that combines feature-space impact with region-local learning progress. Second, all methods were trained on the same benchmark environments using matched PPO hyperparameters, environment-step budgets, and random seeds within each environment. Third, saved checkpoints were evaluated offline with deterministic action selection to separate policy evaluation from training-time stochasticity.

The study followed a controlled comparison protocol. For a fixed environment, the PPO architecture, value architecture, optimizer settings, discount factor, generalized advantage estimation parameter, and evaluation procedure were held constant across methods. The intrinsic-reward coefficient and clipping range were also held constant across intrinsic methods within the same environment. This control was necessary so that observed differences could be attributed primarily to the exploration signal rather than to changes in the underlying policy optimizer.

The proposed method was evaluated in two principal forms. The gated form, GLPE, suppresses intrinsic rewards in latent regions that exhibit high prediction error but low learning progress. The ungated form, GLPE (no gate), uses the same base intrinsic score without region suppression. Additional ablations removed either the feature-space impact component or the learning-progress component, allowing the contribution of each term to be isolated. A computational diagnostic variant was also used to measure the cost of repeatedly computing robust global medians for the gate.

The methodology emphasized reproducibility through fixed training budgets, explicit

seed sets, deterministic execution where supported, offline checkpoint evaluation, and aggregation across independent runs. Because reinforcement-learning outcomes can vary substantially across seeds, no single training trajectory was treated as sufficient evidence. Instead, conclusions were based on curves and summary statistics aggregated over all completed seeds for each method–environment pair.

## 4.2 Proposed Method

### 4.2.1 Method Overview

The proposed method constructs an intrinsic reward from two quantities computed in a learned latent space. The first quantity is feature-space impact, formalized in Section 3.5, which measures how much the latent representation changes between consecutive observations. The second quantity is region-local learning progress, formalized in Section 3.6, which measures whether a forward dynamics model is improving in the latent region currently visited by the agent. The two quantities are normalized separately and combined into a base intrinsic score. The gated variant multiplies this base score by the region-specific binary gate introduced in Section 3.7, suppressing regions that remain difficult to predict while showing little improvement.

Consistent with the reward-augmentation framework in Section 3.2, the augmented reward used for policy optimization on a transition  $(o_t, a_t, o_{t+1})$  is

$$r_t = r_t^{\text{ext}} + \eta_t \text{clip}(r_t^{\text{int}}, -r_{\max}, r_{\max}), \quad (4.1)$$

where  $r_t^{\text{ext}}$  is the environment reward,  $r_t^{\text{int}}$  is the intrinsic reward,  $\eta_t$  is the intrinsic-reward coefficient, and  $r_{\max}$  is the intrinsic clipping bound. Environment-terminal transitions receive zero intrinsic reward so that the auxiliary objective does not assign artificial value to termination itself.

Within each PPO iteration, intrinsic rewards are computed from the on-policy rollout before advantage estimation. The same rollout is then used to update the latent dynamics module. Thus, the intrinsic reward and the auxiliary predictive model are updated on the same stream of experience, while the policy is optimized using the total reward in Equation 4.1. Algorithm 1 summarizes this procedure at the level used by the training loop.

---

**Algorithm 1** High-level intrinsic-reward computation for the GLPE family within one PPO update.

---

```

1: Input: on-policy transitions  $\{(o_t, a_t, o_{t+1})\}_{t=1}^N$ ; terminal indicators; encoder  $\phi_\omega$ ; forward
   model  $f_\psi$ ; inverse model  $g_\xi$ ; region assignment function  $\rho$ ; region EMAs  $\mu_r^{\text{short}}, \mu_r^{\text{long}}$ ; RMS
   accumulators; optional gate states  $g_r$ 
2: Output: intrinsic rewards  $\{r_t^{\text{int}}\}_{t=1}^N$ ; updated region statistics; updated RMS statistics; up-
   dated gate states; updated dynamics parameters
3: for  $t = 1$  to  $N$  do
4:    $z_t \leftarrow \phi_\omega(o_t)$  and  $z_{t+1} \leftarrow \phi_\omega(o_{t+1})$ 
5:    $e_t \leftarrow d^{-1} \|f_\psi(z_t, a_t) - z_{t+1}\|_2^2$ 
6:   Assign  $z_t$  to region  $r \leftarrow \rho(z_t)$ , inserting the point and splitting a leaf when the partition
   criterion is met
7:   Update  $\mu_r^{\text{short}}$  and  $\mu_r^{\text{long}}$  using  $e_t$ 
8:    $\text{LP}_t \leftarrow \max(0, \mu_r^{\text{long}} - \mu_r^{\text{short}})$ 
9:    $I_t \leftarrow \|z_{t+1} - z_t\|_2$ 
10:  Update component RMS accumulators and compute  $\tilde{I}_t, \tilde{\text{LP}}_t$ 
11:   $u_t \leftarrow \alpha_{\text{impact}} \tilde{I}_t + \alpha_{\text{LP}} \tilde{\text{LP}}_t$ 
12:  if gating is enabled then
13:    Compute robust references  $\text{LP}_{\text{med}}$  and  $e_{\text{med}}$  across visited regions
14:    Update  $g_r$  using the low-progress, high-error persistence rule with hysteresis
15:     $r_t^{\text{int}} \leftarrow g_r u_t$ 
16:  else
17:     $r_t^{\text{int}} \leftarrow u_t$ 
18:  end if
19:  if transition  $t$  is environment-terminal then
20:     $r_t^{\text{int}} \leftarrow 0$ 
21:  end if
22: end for
23: Update  $(\omega, \psi, \xi)$  using the forward and inverse dynamics losses on the same rollout

```

---



### 4.2.2 Latent Dynamics Module

Let  $\phi_\omega$  denote an encoder that maps an observation to a latent vector,

$$z_t = \phi_\omega(o_t), \quad z_t \in \mathbb{R}^d. \quad (4.2)$$

For the vector-observation benchmarks used in this study, the encoder is a two-layer multilayer perceptron with 256 hidden units per layer and ReLU activations. The latent dimension is fixed at  $d = 128$  for all methods that require a learned dynamics representation.

The module contains a forward dynamics model  $f_\psi$  and an inverse dynamics model  $g_\xi$ . The forward model predicts the next latent vector from the current latent vector and the action:

$$\hat{z}_{t+1} = f_\psi(z_t, a_t). \quad (4.3)$$

The per-transition forward prediction error is

$$e_t = \frac{1}{d} \|f_\psi(z_t, a_t) - z_{t+1}\|_2^2. \quad (4.4)$$

The inverse model predicts the action from consecutive latent vectors. For discrete actions, the inverse objective is categorical cross-entropy. For continuous actions, the inverse model parameterizes a diagonal Gaussian likelihood with clamped log standard deviations.

The auxiliary dynamics loss is

$$\mathcal{L}_{\text{dyn}} = \beta_{\text{fwd}} \mathcal{L}_{\text{fwd}} + \beta_{\text{inv}} \mathcal{L}_{\text{inv}}, \quad (4.5)$$

where the forward and inverse losses are weighted equally in the experiments. The dynamics module is optimized with Adam at learning rate  $3 \times 10^{-4}$ , and its gradient norm is clipped to 5.0.

### 4.2.3 Online Latent-Space Partitioning

Learning progress is estimated locally by maintaining an online binary partition of the latent space. Each leaf of the partition tree corresponds to a region  $r$ . A latent vector  $z_t$  is assigned to a region by routing it from the root to a leaf according to stored split dimensions and thresholds.

Each splittable leaf stores assigned latent points until its capacity  $C$  is reached. If the leaf depth is below  $D_{\text{max}}$ , the leaf is split along the coordinate with the largest em-

pirical variance among the stored points. The split threshold is the median value on that coordinate, provided that the median produces a non-degenerate partition. One child retains the original region identifier and accumulated statistics; the other child receives a new identifier whose statistics are initialized on first visit. This procedure concentrates additional resolution in frequently visited parts of latent space while bounding the maximum tree depth.

#### 4.2.4 Learning-Progress Estimation

For each region  $r$ , two exponential moving averages of the forward prediction error are maintained: a long-horizon estimate  $\mu_r^{\text{long}}$  and a short-horizon estimate  $\mu_r^{\text{short}}$ . When a transition is assigned to region  $r$ , the updates are

$$\mu_r^{\text{long}} \leftarrow \beta_{\text{long}} \mu_r^{\text{long}} + (1 - \beta_{\text{long}}) e_t, \quad (4.6)$$

$$\mu_r^{\text{short}} \leftarrow \beta_{\text{short}} \mu_r^{\text{short}} + (1 - \beta_{\text{short}}) e_t. \quad (4.7)$$

The first visit to a region initializes both averages to the current prediction error. The experiments use  $\beta_{\text{long}} > \beta_{\text{short}}$ , making the long-horizon average slower to change than the short-horizon average.

The learning-progress value for region  $r$  is

$$\text{LP}(r) = \max(0, \mu_r^{\text{long}} - \mu_r^{\text{short}}). \quad (4.8)$$

For transition  $t$ , the learning-progress component is  $\text{LP}_t = \text{LP}(\rho_t)$ , where  $\rho_t$  is the region assigned to  $z_t$ . The positive part is used so that only reductions in recent prediction error are rewarded.

#### 4.2.5 Feature-Space Impact Estimation

The feature-space impact component is the Euclidean distance between consecutive latent vectors:

$$I_t = \|z_{t+1} - z_t\|_2. \quad (4.9)$$

This component rewards transitions that produce substantial change in the learned representation. It is action-conditioned indirectly because the next latent vector depends on the action selected by the policy and the environment transition.

#### 4.2.6 Component-Wise Scale Stabilization

The raw values of  $I_t$  and  $LP_t$  can differ in scale across environments and across training phases. To stabilize their contribution, each component is normalized by an independent running root-mean-square estimate. For a scalar component  $x_t$ , the second-moment accumulator is updated as

$$v_x \leftarrow \beta_{\text{rms}} v_x + (1 - \beta_{\text{rms}}) x_t^2, \quad (4.10)$$

with  $\beta_{\text{rms}} = 0.99$  and numerical constant  $\varepsilon = 10^{-8}$ . The normalized value is

$$\tilde{x}_t = \frac{x_t}{\sqrt{v_x + \varepsilon}}. \quad (4.11)$$

The method applies this normalization separately to impact and learning progress, producing  $\tilde{I}_t$  and  $\tilde{LP}_t$ .

#### 4.2.7 Base Intrinsic Reward

The shared base intrinsic score is

$$u_t = \alpha_{\text{impact}} \tilde{I}_t + \alpha_{\text{LP}} \tilde{LP}_t, \quad (4.12)$$

where  $\alpha_{\text{impact}} \geq 0$  and  $\alpha_{\text{LP}} \geq 0$  are component weights. The experiments fix  $\alpha_{\text{impact}} = 1.0$  and tune  $\alpha_{\text{LP}}$  by environment. Because the two components are normalized independently, the weights express the intended component tradeoff rather than compensating for arbitrary units.

#### 4.2.8 Ungated Variant

The ungated variant, denoted GLPE (no gate), directly uses the base score:

$$r_t^{\text{int}} = u_t. \quad (4.13)$$

This variant retains the local learning-progress statistics and feature-space impact term but does not suppress any region after the score has been computed. It therefore serves as both a competitive method and an ablation of the gating mechanism.

#### 4.2.9 Gated Variant

The gated variant assigns each visited region a binary gate  $g_r \in \{0, 1\}$ . Its intrinsic reward is

$$r_t^{\text{int}} = g_{\rho_t} u_t. \quad (4.14)$$

A gate value of one preserves the base intrinsic reward, whereas a gate value of zero suppresses it for the assigned region.

The gate is determined from robust global references across visited regions. Let  $\mathcal{R}$  be the set of regions with at least one assigned transition. The method computes

$$\text{LP}_{\text{med}} = \text{median}_{r \in \mathcal{R}} \text{LP}(r), \quad (4.15)$$

$$e_{\text{med}} = \text{median}_{r \in \mathcal{R}} \mu_r^{\text{short}}. \quad (4.16)$$

The learning-progress threshold is

$$\tau_{\text{LP}} = \kappa \text{LP}_{\text{med}}, \quad (4.17)$$

while the normalized error score for region  $r$  is

$$s_r = \frac{\mu_r^{\text{short}}}{e_{\text{med}} + \varepsilon}. \quad (4.18)$$

A region is classified as unproductive on a visit when

$$\text{LP}(r) < \tau_{\text{LP}} \quad \text{and} \quad s_r > \tau_s. \quad (4.19)$$

Gating is activated only after at least  $R_{\text{min}}$  regions have statistics. Before this condition is met, all gates remain enabled. When gating is active, a region with  $g_r = 1$  is disabled after  $K$  consecutive unproductive visits. A disabled region is re-enabled only after it satisfies  $\text{LP}(r) > h\tau_{\text{LP}}$  for two consecutive visits, where  $h > 1$  is a hysteresis multiplier. This persistence and hysteresis reduce rapid gate switching caused by noisy learning-progress estimates.

#### 4.2.10 Intrinsic Reward Scheduling

The GLPE family uses a cosine taper on the intrinsic coefficient. Let  $p \in [0, 1]$  denote progress as a fraction of the total environment-step budget. For taper start and end

fractions  $p_{\text{start}} < p_{\text{end}}$ , the schedule is

$$w(p) = \begin{cases} 1, & p \leq p_{\text{start}}, \\ \frac{1}{2} \left( 1 + \cos \left( \pi \frac{p - p_{\text{start}}}{p_{\text{end}} - p_{\text{start}}} \right) \right), & p_{\text{start}} < p < p_{\text{end}}, \\ 0, & p \geq p_{\text{end}}. \end{cases} \quad (4.20)$$

The effective coefficient is  $\eta_t = \eta w(p_t)$ . The schedule gives intrinsic exploration full strength early in training, gradually reduces the auxiliary pressure, and removes it near the end of training so that final policies are optimized primarily under environment reward.

### 4.3 Reinforcement Learning Backbone

#### 4.3.1 PPO Training Procedure

All methods used Proximal Policy Optimization as the policy-learning algorithm (Schulman et al., 2017). For each update, the current policy collected an on-policy batch of  $N = B \times T$  transitions from  $B$  vectorized environment instances over rollout horizon  $T$ . The total reward was formed by adding the scaled and clipped intrinsic reward when an intrinsic method was active. Generalized advantage estimation was then computed using the total reward (Schulman et al., 2015).

For a transition at time  $t$ , the temporal-difference residual used by GAE is

$$\delta_t = r_t + \gamma V_\nu(o_{t+1}) - V_\nu(o_t), \quad (4.21)$$

with standard terminal handling and timeout bootstrapping when a valid final observation is available. The advantage estimate is

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}, \quad (4.22)$$

where  $\gamma$  is the discount factor and  $\lambda$  is the GAE parameter. Advantages are normalized within each PPO update batch.

The clipped PPO policy objective is

$$\mathcal{L}_\pi(\theta) = -\mathbb{E}_t \left[ \min \left( \varrho_t(\theta) \hat{A}_t, \text{clip}(\varrho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (4.23)$$

where  $\epsilon$  is the PPO clipping range and the probability ratio is

$$\varrho_t(\theta) = \frac{\pi_\theta(a_t | o_t)}{\pi_{\theta_{\text{old}}}(a_t | o_t)}. \quad (4.24)$$

The value function is trained with a squared-error loss against the GAE value target, with optional value clipping in the MuJoCo tasks. Policy and value optimizers use Adam. Gradient norms are clipped to 1.0, and PPO epochs stop early within an update if the approximate KL divergence exceeds the environment-specific threshold.

| Environment      | LR                   | Epochs | Minibatches | Clip | $\lambda$ | Entropy | Value clip | KL stop |
|------------------|----------------------|--------|-------------|------|-----------|---------|------------|---------|
| MountainCar-v0   | $3.0 \times 10^{-4}$ | 10     | 16          | 0.20 | 0.95      | 0.00    | 0.00       | 0.06    |
| BipedalWalker-v3 | $5.0 \times 10^{-4}$ | 5      | 16          | 0.25 | 0.95      | 0.00    | 0.00       | 0.06    |
| Ant-v5           | $1.5 \times 10^{-4}$ | 15     | 64          | 0.20 | 0.95      | 0.00    | 0.20       | 0.04    |
| HalfCheetah-v5   | $3.0 \times 10^{-4}$ | 10     | 32          | 0.20 | 0.95      | 0.01    | 0.20       | 0.03    |
| Humanoid-v5      | $2.0 \times 10^{-4}$ | 5      | 32          | 0.20 | 0.97      | 0.01    | 0.20       | 0.03    |

Table 4.1: PPO hyperparameters used for all methods within each environment. The discount factor was  $\gamma = 0.99$  and the value-loss coefficient was 0.5 in all environments.

### 4.3.2 Policy and Value Networks

The policy and value function were represented by separate multilayer perceptrons. Each network used two hidden layers of width 256 with ReLU activations. For discrete-action environments, the policy produced categorical logits. For continuous-action environments, the policy produced a diagonal Gaussian distribution with a learned log standard deviation vector. When action bounds were finite, actions were mapped to the valid interval by a tanh squashing transformation followed by affine rescaling to the environment bounds.

The value network produced a scalar estimate  $V_\nu(o_t)$ . The policy and value networks did not share parameters. Vector observations were normalized using a running mean and variance estimated online from collected rollouts, and the normalized observations were used consistently by the policy, value function, and intrinsic modules.

### 4.3.3 Reward Handling

For intrinsic-reward methods, raw intrinsic values were transformed before being added to the environment reward. If an intrinsic module already produced normalized outputs, the normalized intrinsic reward was directly clipped and scaled. If an intrinsic module produced unnormalized outputs, a running RMS normalizer was applied before scaling.

The resulting training reward was

$$r_t = r_t^{\text{ext}} + \eta_t \text{clip}(r_{t,\text{norm}}^{\text{int}}, -r_{\text{max}}, r_{\text{max}}), \quad (4.25)$$

where  $r_{t,\text{norm}}^{\text{int}}$  denotes the intrinsic signal after the method-appropriate normalization step.

Intrinsic rewards were set to zero on environment-terminal transitions. For time-limit truncations, bootstrapping was used when a final observation was available. If a time-limit truncation did not provide a valid final observation, the transition was treated conservatively for bootstrapping and intrinsic reward was suppressed for that transition. This handling prevents invalid next observations from entering either intrinsic-reward computation or value-target estimation.

## 4.4 Baseline Methods

### 4.4.1 PPO

Vanilla PPO served as the extrinsic-reward baseline. It used the same policy architecture, value architecture, optimizer, rollout sizes, and evaluation protocol as all other methods, but the intrinsic coefficient was set to zero. This baseline measures the performance obtained without explicit intrinsic motivation.

### 4.4.2 ICM

The Intrinsic Curiosity Module (ICM) baseline used forward-model prediction error in a learned feature space as intrinsic reward (Pathak et al., 2017). The module contained the same general encoder, forward model, and inverse model structure used by the GLPE latent dynamics module. For transition  $(o_t, a_t, o_{t+1})$ , the intrinsic signal was the mean-squared prediction error between the predicted and encoded next latent vector. The inverse dynamics loss encouraged the representation to retain action-relevant transition information.

### 4.4.3 RND

Random Network Distillation (RND) used the prediction error between a trainable predictor network and a fixed randomly initialized target network as the intrinsic reward (Burda, Edwards, Pathak, et al., 2018; Burda, Edwards, Storkey, & Klimov, 2018). Both networks mapped observations to feature vectors of dimension 128 using two hidden

layers of width 256. Only the predictor was updated during training. High prediction error indicated observations that remained unfamiliar under the predictor, while repeated exposure reduced the intrinsic reward.

#### 4.4.4 RIDE

Rewarding Impact-Driven Exploration (RIDE) rewarded the magnitude of feature-space change between consecutive observations (Raileanu & Rocktäschel, 2020). The impact term was

$$\|\phi(o_{t+1}) - \phi(o_t)\|_2. \quad (4.26)$$

During rollout collection, this signal was divided by an episodic visitation count obtained by discretizing the learned feature vector with bin size 0.25. The episodic count was reset when an environment instance began a new episode. The RIDE encoder was trained with the same forward and inverse dynamics losses used by the ICM-style latent module.

#### 4.4.5 RIAC

The RIAC baseline used region-local learning progress as the intrinsic signal (Baranes & Oudeyer, 2009). It maintained an adaptive latent-space partition and exponential moving averages of forward prediction error within each region. The intrinsic reward was the normalized positive difference between the long-horizon and short-horizon error estimates. Unlike GLPE, RIAC did not include a feature-space impact term and did not use the high-error low-progress gate. It therefore isolated the contribution of learning progress without impact weighting or explicit unproductive-region suppression.

### 4.5 Experimental Environment

#### 4.5.1 Benchmark Suite

The benchmark suite consisted of five standard reinforcement-learning control environments: MountainCar-v0, BipedalWalker-v3, Ant-v5, HalfCheetah-v5, and Humanoid-v5. The suite was selected to cover both sparse-reward and continuous-control settings. MountainCar-v0 provides a sparse and delayed success signal in a low-dimensional discrete-action task. BipedalWalker-v3 requires coordinated continuous control and contains failure states. The three MuJoCo locomotion tasks



provide higher-dimensional continuous-control settings with denser reward feedback and different morphology complexities.

The default reward functions and termination conditions were used in all environments. No domain randomization was applied. No additional frame skipping was introduced beyond the environment configuration. Training used synchronous vectorized environments, with the number of parallel instances selected to obtain a nominal PPO batch size of  $N = 16,384$  transitions per update.

| Environment      | $B$ | $T$   | $N$    | Total steps | Seeds |
|------------------|-----|-------|--------|-------------|-------|
| MountainCar-v0   | 16  | 1,024 | 16,384 | 3,000,000   | 10    |
| BipedalWalker-v3 | 8   | 2,048 | 16,384 | 7,000,000   | 8     |
| Ant-v5           | 8   | 2,048 | 16,384 | 15,000,000  | 8     |
| HalfCheetah-v5   | 8   | 2,048 | 16,384 | 15,000,000  | 8     |
| Humanoid-v5      | 4   | 4,096 | 16,384 | 30,000,000  | 5     |

Table 4.2: Benchmark suite and training budgets.  $B$  is the number of parallel environment instances,  $T$  is the rollout horizon per instance, and  $N = B \times T$  is the nominal PPO update batch size.

#### 4.5.2 Training Budgets

The total environment-step budgets in Table 4.2 were chosen to extend beyond the early learning phase. This allowed the evaluation to measure both rapid initial improvement and long-horizon stability. The final PPO update in a run was allowed to contain fewer than  $N$  transitions if the remaining step budget was smaller than a full rollout batch.

The GLPE hyperparameters used for each environment are shown in Table 4.3. The same intrinsic scaling coefficient  $\eta$  and intrinsic clipping range  $r_{\max}$  were used by all intrinsic methods within a given environment. This kept the reward-scale comparison consistent across ICM, RND, RIDE, RIAC, GLPE, and GLPE (no gate).

| Environment      | $\eta$ | $r_{\max}$ | $\alpha_{\text{LP}}$ | $p_{\text{start}}$ | $p_{\text{end}}$ | $C$ | $D_{\max}$ | $\beta_{\text{long}}$ | $\beta_{\text{short}}$ | $\kappa$ | $\tau_s$ | $K$ | $R_{\min}$ |
|------------------|--------|------------|----------------------|--------------------|------------------|-----|------------|-----------------------|------------------------|----------|----------|-----|------------|
| MountainCar-v0   | 0.05   | 4.0        | 0.5                  | 0.05               | 0.80             | 128 | 10         | 0.995                 | 0.90                   | 0.010    | 2.0      | 5   | 8          |
| BipedalWalker-v3 | 0.08   | 4.0        | 0.5                  | 0.12               | 0.75             | 200 | 12         | 0.995                 | 0.90                   | 0.010    | 2.0      | 5   | 16         |
| Ant-v5           | 0.06   | 4.0        | 0.6                  | 0.10               | 0.70             | 256 | 12         | 0.997                 | 0.92                   | 0.012    | 2.5      | 8   | 64         |
| HalfCheetah-v5   | 0.04   | 5.0        | 0.5                  | 0.05               | 0.75             | 256 | 12         | 0.995                 | 0.90                   | 0.010    | 2.0      | 5   | 32         |
| Humanoid-v5      | 0.06   | 5.0        | 0.5                  | 0.05               | 0.80             | 256 | 12         | 0.998                 | 0.92                   | 0.010    | 2.0      | 5   | 32         |

Table 4.3: GLPE hyperparameters by environment. The impact weight was fixed at  $\alpha_{\text{impact}} = 1.0$ , and the hysteresis multiplier was fixed at  $h = 2.0$ . GLPE (no gate) used the same settings but disabled the gate.

### 4.5.3 Random Seeds and Determinism

Each method was evaluated using the same seed set within each environment. The seed counts are shown in Table 4.2. The experiments enabled deterministic execution where supported by the software stack and environment backend. The purpose of fixed seed sets was not to remove randomness from reinforcement learning, but to ensure that method comparisons were paired as closely as practical across independent training runs.

## 4.6 Evaluation Methodology

### 4.6.1 Offline Checkpoint Evaluation

Training performance was evaluated from saved checkpoints rather than from online training rollouts. A checkpoint was saved at step 0, at additional warmup points early in training, at regular intervals equal to 10% of the environment’s total step budget, and at the final step. Each evaluated checkpoint was run for 20 episodes in a separate evaluation environment. The policy used deterministic action selection: categorical mode for discrete policies and distribution mode for continuous policies. Episode seeds for evaluation were fixed functions of the corresponding training seed, making checkpoint comparisons repeatable.

Only extrinsic environment return was used for evaluation. Intrinsic reward was excluded from all evaluation metrics because the objective of the study is to improve task performance, not to maximize the auxiliary reward itself.

### 4.6.2 Primary Performance Metric

The primary performance metric was undiscounted episodic return from the environment:

$$G = \sum_{t=0}^{T_{\text{ep}}-1} r_t^{\text{ext}}, \quad (4.27)$$

where  $T_{\text{ep}}$  is the episode length. For each checkpoint and seed, the mean return over 20 evaluation episodes was recorded. For each method–environment pair, mean learning curves were obtained by averaging checkpoint returns across training seeds.

### 4.6.3 Sample Efficiency Metrics

Sample efficiency was measured by the relationship between evaluation return and cumulative environment steps. In addition to plotting learning curves, the study used step-AUC as a scalar summary. Let  $\bar{G}(s_i)$  denote the mean evaluation return at checkpoint step  $s_i$ , with checkpoints indexed as  $s_0, \dots, s_K$ . For an environment-step budget  $S = s_K - s_0$ , step-AUC was computed by trapezoidal integration and normalized by the budget:

$$\text{AUC}_{\text{step}} = \frac{1}{S} \sum_{i=1}^K \frac{\bar{G}(s_i) + \bar{G}(s_{i-1})}{2} (s_i - s_{i-1}). \quad (4.28)$$

Higher step-AUC indicates either faster early learning, higher sustained return, or both.

### 4.6.4 Reliability Metrics

Reliability was evaluated using threshold-reaching statistics across seeds. For each environment, a solved-return threshold  $G_{\text{solve}}$  was specified as follows:

$$\begin{aligned} \text{MountainCar-v0: } & -110, & \text{BipedalWalker-v3: } & 300, \\ \text{Ant-v5: } & 5,000, & \text{HalfCheetah-v5: } & 6,000, \\ \text{Humanoid-v5: } & 3,000. \end{aligned} \quad (4.29)$$

For each seed, the first checkpoint crossing a threshold was identified. When the threshold was crossed between two checkpoints, the crossing step was estimated by linear interpolation between the adjacent checkpoint returns.

The study reported reach counts and median steps-to-threshold at 25%, 50%, and 100% of the solved threshold. For positive solved thresholds, the fractional threshold was  $qG_{\text{solve}}$  for  $q \in \{0.25, 0.50, 1.00\}$ . For MountainCar-v0, whose solved threshold is negative and whose poor-return regime is near  $-200$ , the monotonic fractional thresholds were computed as

$$G_q = G_{\text{solve}}(2 - q). \quad (4.30)$$

This gives thresholds that become easier as  $q$  decreases while preserving the direction that higher return is better. Medians were computed over successful seeds only, and the reach count was reported alongside each median to avoid hiding failures.

#### 4.6.5 Wall-Clock Efficiency Metrics

Wall-clock efficiency was measured from end-to-end training time. For each PPO iteration, time was recorded for environment interaction, policy inference, intrinsic-reward computation, intrinsic-module updates, advantage computation, and PPO optimization. Cumulative wall-clock time was then associated with each checkpoint.

The wall-clock AUC metric integrated mean evaluation return against cumulative training time. For each environment, the common wall-clock budget was defined as the smallest final training time among the methods being compared. Learning curves were integrated only up to this common budget, preventing slower methods from being rewarded by extrapolation beyond the measured time available to faster methods.

#### 4.6.6 Statistical Reporting

Results were aggregated across independent training seeds. Learning curves report the mean across seeds at each checkpoint. Scalar summaries such as final-checkpoint return and AUC were reported with 95% nonparametric bootstrap confidence intervals over seeds when interval estimates were required. The bootstrap procedure resampled seeds with replacement and recomputed the statistic on each resample.

For thresholded reliability, the principal quantities were reach count and median steps-to-threshold. Reach count measures robustness across seeds, while the median crossing step measures speed among the successful runs. These two quantities were reported together because a method that reaches a threshold quickly in only a small number of seeds is less reliable than a method that reaches it consistently.

### 4.7 Ablation Methodology

#### 4.7.1 Impact-Only Variant

The impact-only ablation removed the learning-progress component from the GLPE base score by setting

$$\alpha_{LP} = 0, \quad \alpha_{\text{impact}} > 0. \quad (4.31)$$

The resulting intrinsic reward was proportional to normalized feature-space impact, with the same latent encoder, forward and inverse dynamics module, reward scaling, clipping, PPO backbone, and evaluation procedure used by GLPE. This ablation tested whether encouraging large representation changes was sufficient without explicitly measuring model improvement.

### 4.7.2 Learning-Progress-Only Variant

The learning-progress-only ablation removed the feature-space impact component by setting

$$\alpha_{\text{impact}} = 0, \quad \alpha_{\text{LP}} > 0. \quad (4.32)$$

The reward was then based only on normalized region-local learning progress. This ablation tested whether local reductions in forward-model error were sufficient without separately rewarding the magnitude of latent transition change. The online partition, EMA updates, and component normalization procedure were otherwise unchanged.

### 4.7.3 Gating Analysis

The gating analysis compared GLPE against GLPE (no gate) under matched hyperparameters. Both variants used the same impact term, learning-progress term, component normalization, latent-space partition, and intrinsic coefficient schedule. The only difference was whether the binary region gate was active. This comparison isolated the effect of suppressing high-error low-progress regions.

Gate behavior was analyzed using the fraction of visited regions whose gates were disabled:

$$\text{GateRate} = \frac{|\{r \in \mathcal{R} : g_r = 0\}|}{|\mathcal{R}|}. \quad (4.33)$$

The gate-rate metric was examined together with learning curves and intrinsic-reward statistics. A high gate rate indicates that many visited regions were classified as unproductive under the current thresholds, whereas a low gate rate indicates that most regions continued to contribute intrinsic reward.

### 4.7.4 Computational Analysis

The computational analysis measured the overhead introduced by intrinsic-reward methods relative to vanilla PPO. Per-iteration timing was decomposed into rollout interaction, policy inference, intrinsic computation, intrinsic-module update, advantage computation, and PPO optimization. This decomposition allowed the analysis to distinguish overhead caused by predictive auxiliary models from overhead caused specifically by GLPE’s region statistics and gating operations.

For the gated method, an additional diagnostic compared gate median computation with a cached-median variant. The standard gate recomputed robust global medians from current region statistics whenever they were needed. The cached variant reused

the most recently computed medians for a fixed number of region-statistic updates before refreshing them. This diagnostic did not change the definition of the main GLPE method; it quantified whether robust threshold computation was a significant runtime cost.

The wall-clock analysis used both direct timing breakdowns and wall-clock AUC. Direct timing identified where time was spent during training, while wall-clock AUC measured the combined effect of learning speed and computational overhead on achieved return per unit time.

## **4.8 Project Management**

### **4.8.1 Development Process**

The study followed an iterative development process rather than a strictly linear sequence of tasks. The initial proposal was kept sufficiently broad to allow the research direction, experimental design, and technical emphasis to be refined as preliminary results clarified the behavior of the proposed exploration method. Across successive cycles, the method specification, experimental procedure, analysis strategy, and thesis manuscript were revised to maintain consistency between the implemented study and its academic presentation.

Development and experiment execution preceded the final writing of the thesis manuscript. This ordering allowed the final methodology and discussion to be based on completed experiments rather than on planned procedures alone. The final manuscript was then written to consolidate the theoretical framing, experimental protocol, results, and interpretation into a standalone thesis.

### **4.8.2 Costs**

The primary direct expense of the study was cloud computing for reinforcement-learning experiments. The total cloud compute cost was PHP 41,266.87. This amount includes preliminary experiments, exploratory tests, development runs, and the final experimental runs.

The compute cost reflects the iterative nature of the project. Early runs were used to verify training behavior, evaluate feasibility, and refine the experimental configuration. Later runs were used for the main comparisons, ablations, checkpoint evaluations, and timing measurements reported in the thesis. No separate hardware procurement cost was incurred for the experimental component.

### **4.8.3 Timeline**

The study was conducted from February 2025 to May 2026. The timeline involved overlapping activities, including proposal preparation, method refinement, implementation, experiment execution, analysis, writing, and presentation preparation. These activities were not strictly separated because results from one stage often informed revisions to another stage.

The early phase from February 2025 focused on defining the research problem, surveying relevant literature, and preparing a proposal that allowed subsequent refinement of the technical direction. Main development and final experimental work began around December 2025. The main experimental runs were completed in December 2025, after which the remaining work focused on organizing the results, completing the analysis, writing the thesis manuscript, and preparing presentation materials. Some intervals involved reduced activity because of academic and scheduling constraints, but the overall work continued through iterative refinement until completion in May 2026.

### **4.8.4 Responsibilities**

Mikael Vincent Tumampos handled the primary research work, including the formulation of the method, implementation, experiment execution, result analysis, thesis writing, and preparation of the technical presentation. Ervin Joshua Guirnela supported the completion of the thesis by assisting in the preparation and organization of presentation materials, coordinating academic and institutional requirements, and helping present the study in a form suitable for review and defense. Christine Peña served as the thesis adviser and provided guidance on institutional requirements, thesis structure, academic presentation, and compliance with undergraduate thesis standards.

## **4.9 Ethical and Reproducibility Considerations**

This study used simulated reinforcement-learning benchmarks and did not involve human participants, personal data, or deployment in a real-world control setting. The primary ethical concern is therefore indirect: reinforcement-learning algorithms developed in simulation can later be transferred to domains where unsafe exploration, reward misspecification, or unreliable evaluation may cause harm. For this reason, the study evaluated policies only in benchmark environments and interpreted intrinsic reward as a training aid rather than as an objective suitable for unrestricted deployment.

The method changes the reward optimized during training by adding an auxiliary

intrinsic component. Such augmentation can alter the policy search trajectory and is not guaranteed to preserve the optimal policy of the original environment reward. The evaluation therefore used extrinsic return only, ensuring that reported performance reflects the task objective rather than the magnitude of curiosity-driven reward. Intrinsic reward was also clipped, scheduled, and suppressed on terminal or invalid transitions to reduce avoidable reward-scale artifacts.

Reproducibility was addressed through fixed environment-step budgets, explicit random seed sets, deterministic execution where supported, and offline checkpoint evaluation with fixed evaluation episode seeds. The same PPO hyperparameters and network architectures were used across methods within each environment. The same intrinsic scaling coefficient and clipping range were used for all intrinsic methods within a given environment, reducing the risk that differences were caused by reward-scale tuning rather than by the intrinsic signal itself.

The reporting protocol emphasized aggregate behavior rather than isolated successful runs. Learning curves were averaged across seeds, scalar metrics were summarized with bootstrap confidence intervals where appropriate, and thresholded reliability reported both reach counts and median steps among successful seeds. This is important because reinforcement-learning experiments can exhibit substantial variance across seeds.

The computational analysis also contributes to reproducibility by reporting wall-clock efficiency and runtime decomposition, not only sample efficiency. An intrinsic method that improves return per environment step but requires substantially more computation may be less practical under a fixed time budget. Including both step-based and time-based metrics provides a more complete account of the tradeoffs associated with the proposed exploration method.



## CHAPTER 5

### RESULTS AND DISCUSSION

#### 5.1 Overview of Experimental Results

This chapter presents the empirical results of the proposed Gated Learning-Progress Exploration (GLPE) method and its ungated variant against PPO and intrinsic-reward baselines. All results are reported using the deterministic offline evaluation protocol defined in Section 4.6.1 and are measured with extrinsic environment return only. Intrinsic rewards therefore affect the optimization trajectory during training but are not included in the reported evaluation return.

The evaluation covers the five environments listed in Table 4.2: MountainCar-v0, BipedalWalker-v3, Ant-v5, HalfCheetah-v5, and Humanoid-v5. The set includes sparse-reward exploration, continuous-control locomotion, dense reward shaping, and high-variance dynamics. The principal comparison is between GLPE, GLPE without gating, PPO without intrinsic reward, and the ICM, RND, RIDE, and RIAC intrinsic-reward baselines. The analysis also includes the component ablations defined in Section 4.7, which retain only feature-space impact or only learning progress.

The main empirical finding is that GLPE is most advantageous in sparse-reward exploration, while the ungated variant is generally more competitive in dense-reward and high-variance settings. On MountainCar-v0, GLPE reaches the solved threshold in all seeds and achieves the strongest final return among the compared methods. On BipedalWalker-v3, Ant-v5, and HalfCheetah-v5, the GLPE family remains competitive, but the strongest baseline depends on the environment and metric. On Humanoid-v5, all methods exhibit wide between-seed variation, and the confidence intervals are too broad for a stable ranking based only on mean return.

The results support three interpretations. First, combining feature-space impact with learning progress is useful when a task requires directed exploration before extrinsic rewards become informative. Second, the gate functions as a conservative guardrail rather than a general performance enhancer; it can suppress unproductive intrinsic reward but may also reduce useful exploration when the task already provides dense feedback. Third, intrinsic-reward methods must be evaluated jointly by return, sample efficiency, reliability, and wall-clock cost because a method that improves one dimension may be weaker on another.

## 5.2 Learning Curves

Figure 5.1 shows the deterministic evaluation learning curves for GLPE, GLPE without gating, PPO, and the four intrinsic-reward baselines. Each curve reports mean extrinsic return over evaluation checkpoints, while the plotted seed markers indicate the spread of independently trained policies. The curves are the primary evidence for learning dynamics because they show whether a method improves early, plateaus below a target, collapses after initial progress, or continues improving throughout the training budget.

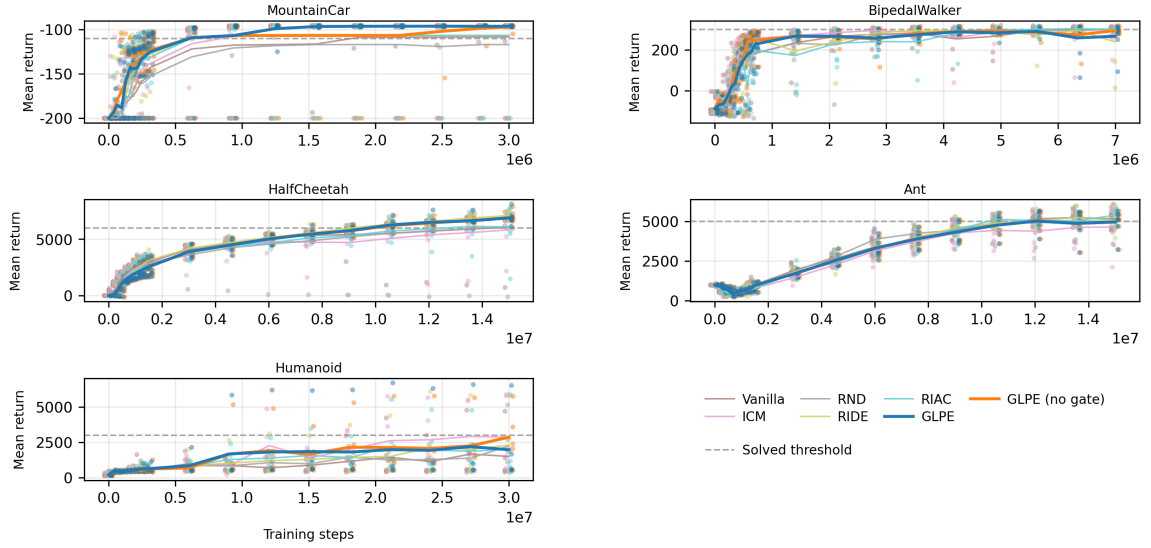


Figure 5.1: Evaluation learning curves for GLPE and baseline methods. Each panel plots mean extrinsic return against environment steps. Markers denote per-seed checkpoint means from deterministic offline evaluation. Horizontal dashed lines indicate the task-specific solved thresholds used in the threshold analyses.

### 5.2.1 Sparse-Reward Environment Results

The sparse-reward task shows the clearest advantage for the proposed method. In MountainCar-v0, the agent must discover a sequence of energy-building actions before the goal reward becomes attainable. GLPE reaches the solved threshold reliably and stabilizes near the top of the observed return range. Its final mean return is  $-96.2$  with a 95% bootstrap interval of  $[-96.5, -95.9]$ , while GLPE without gating obtains  $-97.1$  with interval  $[-99.0, -96.0]$ . The best baseline final mean is RIAC at  $-106.6$ , whose interval  $[-127.5, -95.9]$  indicates that some seeds solve the task while others remain substantially worse.

The shape of the MountainCar-v0 curves indicates that the combined impact-learning-progress signal helps the policy discover useful transitions more consistently than prediction-error or impact-only signals. Prediction-error bonuses can remain large in regions that are difficult to model, and impact rewards can favor motion without necessarily identifying progress toward the goal. GLPE instead rewards latent change when local predictive competence is improving, which is aligned with the requirement to discover controllable transitions that make later reward possible.

### 5.2.2 Continuous-Control Environment Results

The continuous-control environments produce a more mixed pattern. On BipedalWalker-v3, GLPE without gating learns rapidly and ends close to the solved threshold, obtaining a final mean return of 294.4. PPO without intrinsic reward has the highest final mean among the primary baselines at 302.2, while ICM and RND are also competitive. The result indicates that intrinsic rewards are not uniformly beneficial when the environment already provides enough shaped feedback to guide locomotion learning.

On Ant-v5, the GLPE variants have nearly identical learning curves and final returns because the gate has little visible effect on the aggregate trajectory. Both GLPE and GLPE without gating obtain a final mean return of 4,961. RIAC has the strongest final mean among the baselines at 5,384. The gap suggests that, in this dense locomotion setting, the proposed intrinsic signal is useful but does not dominate the information already available from the extrinsic reward and other exploration mechanisms.

On HalfCheetah-v5, GLPE and GLPE without gating achieve high final returns of 6,889 and 6,908, respectively. RIDE reaches the highest final mean at 7,106. The learning curves show that most methods improve steadily, which is consistent with the dense velocity-based reward structure of the task. In such environments, the principal challenge is not discovering any reward signal but maintaining stable policy improvement over long horizons.

Humanoid-v5 is the most variable task in the suite. GLPE without gating obtains a final mean return of 2,858, close to ICM at 2,874, but the uncertainty intervals for all methods are wide. GLPE with gating has a lower mean of 1,970, indicating that gate suppression can be too conservative when the high-dimensional dynamics produce noisy progress estimates. The large seed-to-seed variation prevents strong conclusions based solely on the final mean.

### 5.2.3 Cross-Environment Observations

Across environments, the proposed method is strongest when the task requires exploration before reward becomes informative. This is most evident in MountainCar-v0, where GLPE solves all seeds and separates clearly from the baselines. In dense locomotion tasks, the benefit of GLPE is more modest because reward gradients already provide frequent feedback. The ungated variant is often comparable to or better than the gated variant in these settings, which suggests that the gate should be viewed as protection against unproductive curiosity rather than as a universally beneficial addition.

The curves also show that final performance and learning speed can disagree. For example, a method may reach moderate returns early but fail to continue improving toward the solved threshold, while another method may learn more slowly but eventually reach higher returns. For this reason, Sections 5.3–5.9 report step-AUC, final-checkpoint return, threshold reliability, ablations, and wall-clock efficiency rather than relying on a single curve-based interpretation.

## 5.3 Step-AUC Analysis

Using the sample-efficiency metric defined in Section 4.6.3, step-AUC measures average evaluation return over the full training budget. For a method with mean evaluation return  $\bar{G}(s_i)$  at checkpoint step  $s_i$ , the normalized step-AUC is computed as

$$\text{AUC}_{\text{step}} = \frac{1}{s_K - s_0} \sum_{i=1}^K \frac{\bar{G}(s_i) + \bar{G}(s_{i-1})}{2} (s_i - s_{i-1}), \quad (5.1)$$

where  $s_0 = 0$  and  $s_K$  is the final evaluation step. This metric rewards methods that learn early and maintain high return, rather than methods that improve only at the final checkpoint.

Table 5.1 reports step-AUC for GLPE, GLPE without gating, and the strongest baseline per environment. The best baseline is selected independently for each environment from PPO, ICM, RND, RIDE, and RIAC.

GLPE has the highest step-AUC on MountainCar-v0, which is consistent with its rapid and reliable discovery of the sparse-reward solution. On BipedalWalker-v3, GLPE without gating is close to ICM, differing by about 2.1 return points in mean step-AUC. On Ant-v5 and HalfCheetah-v5, the strongest baselines have a moderate advantage, but the GLPE variants remain within the same broad performance range. On Humanoid-v5, ICM has the highest mean step-AUC, but the confidence intervals for GLPE, GLPE

| Environment      | GLPE                    | GLPE (no gate)          | Best baseline                 |
|------------------|-------------------------|-------------------------|-------------------------------|
| MountainCar-v0   | -107.0 [-114.8, -101.8] | -111.3 [-128.7, -101.4] | RIDE: -113.7 [-134.2, -101.9] |
| BipedalWalker-v3 | 249.0 [223.9, 270.7]    | 255.2 [230.0, 275.6]    | ICM: 257.3 [229.4, 278.3]     |
| Ant-v5           | 3,402 [3,067, 3,708]    | 3,402 [3,067, 3,707]    | RND: 3,565 [3,202, 3,876]     |
| HalfCheetah-v5   | 5,005 [4,708, 5,316]    | 4,998 [4,700, 5,296]    | RIDE: 5,208 [5,019, 5,410]    |
| Humanoid-v5      | 1,583 [551, 3,360]      | 1,665 [769, 2,921]      | ICM: 1,781 [829, 2,797]       |

Table 5.1: Step-AUC of deterministic evaluation return versus environment steps. Values are normalized by the corresponding environment step budget. Brackets show approximate 95% bootstrap uncertainty intervals obtained from the checkpoint confidence bands. Higher values are better.

without gating, and ICM overlap substantially.

The step-AUC results indicate that GLPE is not merely a final-checkpoint effect in the sparse-reward task. Its advantage on MountainCar-v0 is accumulated throughout training. In contrast, the smaller gaps in the locomotion environments show that the proposed intrinsic reward improves neither all training phases nor all task families uniformly. This supports evaluating GLPE as a targeted exploration method rather than as a replacement for all intrinsic-reward approaches.

## 5.4 Final-Checkpoint Performance

Final-checkpoint return measures the quality of the policy at the end of the fixed training budget. It complements step-AUC by emphasizing asymptotic performance within the allotted interactions. Table 5.2 reports final deterministic evaluation returns for GLPE, GLPE without gating, and the strongest baseline in each environment.

| Environment      | GLPE                 | GLPE (no gate)       | Best baseline                |
|------------------|----------------------|----------------------|------------------------------|
| MountainCar-v0   | -96.2 [-96.5, -95.9] | -97.1 [-99.0, -96.0] | RIAC: -106.6 [-127.5, -95.9] |
| BipedalWalker-v3 | 266.5 [214.0, 301.3] | 294.4 [285.9, 303.7] | PPO: 302.2 [291.2, 309.4]    |
| Ant-v5           | 4,961 [4,421, 5,359] | 4,961 [4,436, 5,349] | RIAC: 5,384 [5,118, 5,625]   |
| HalfCheetah-v5   | 6,889 [6,565, 7,285] | 6,908 [6,548, 7,280] | RIDE: 7,106 [6,901, 7,299]   |
| Humanoid-v5      | 1,970 [546, 4,355]   | 2,858 [1,428, 4,454] | ICM: 2,874 [770, 4,956]      |

Table 5.2: Final-checkpoint mean extrinsic return under deterministic offline evaluation. Brackets show 95% bootstrap confidence intervals over seeds. Higher values are better.

The final-checkpoint results reinforce the environment-dependent character of the method. GLPE is the strongest method on MountainCar-v0, where the return interval is narrow and close to the maximum observed solution quality. The reliability of this result is important because MountainCar-v0 contains limited extrinsic feedback until the agent has already learned an effective control strategy.

In BipedalWalker-v3, the ungated variant is much closer to the strongest baseline than gated GLPE. The difference suggests that gate suppression can remove useful intrinsic shaping when the environment’s dynamics are learnable but the progress signal fluctuates near the threshold. The same pattern appears more strongly in Humanoid-v5, where GLPE without gating is close to ICM while gated GLPE is lower.

In Ant-v5 and HalfCheetah-v5, the GLPE family achieves high returns but does not exceed the strongest baseline. These tasks provide dense reward components, so exploration bonuses mainly influence the path of optimization rather than enabling reward discovery from near-zero feedback. The results therefore suggest that GLPE is most appropriate when the exploration signal must compensate for sparse or delayed extrinsic feedback.

## 5.5 Thresholded Reliability

Average return can hide failures when only some seeds succeed. Thresholded reliability, defined in Section 4.6.4, therefore measures how many seeds reach a target return and how quickly successful seeds reach it. For a return threshold  $G_{\text{thr}}$ , the first reaching step is estimated by linear interpolation between the two adjacent evaluation checkpoints that bracket  $G_{\text{thr}}$ . If a seed never reaches the threshold by the final checkpoint, it is counted as unsuccessful and is excluded from the median reaching-step calculation.

Figure 5.2 summarizes threshold-reaching behavior at 25%, 50%, and 100% of the task-specific solved threshold. For positive solved thresholds, the intermediate threshold is a direct fraction of the solved value. For MountainCar-v0, where higher returns are less negative, the intermediate thresholds are mapped monotonically from the minimum-return side so that 25% and 50% represent progressively stronger partial performance.

### 5.5.1 25 Percent Threshold Results

The 25% threshold measures early acquisition of nontrivial behavior. Table 5.3 reports the number of successful seeds and the median number of environment steps, in millions, among those successful seeds.

At this low threshold, most environments are solved by most methods in the sense of reaching partial competence. GLPE and GLPE without gating reach the threshold in all seeds for four of the five environments. The exception is Humanoid-v5, where GLPE reaches the threshold in three of five seeds and GLPE without gating reaches it in four

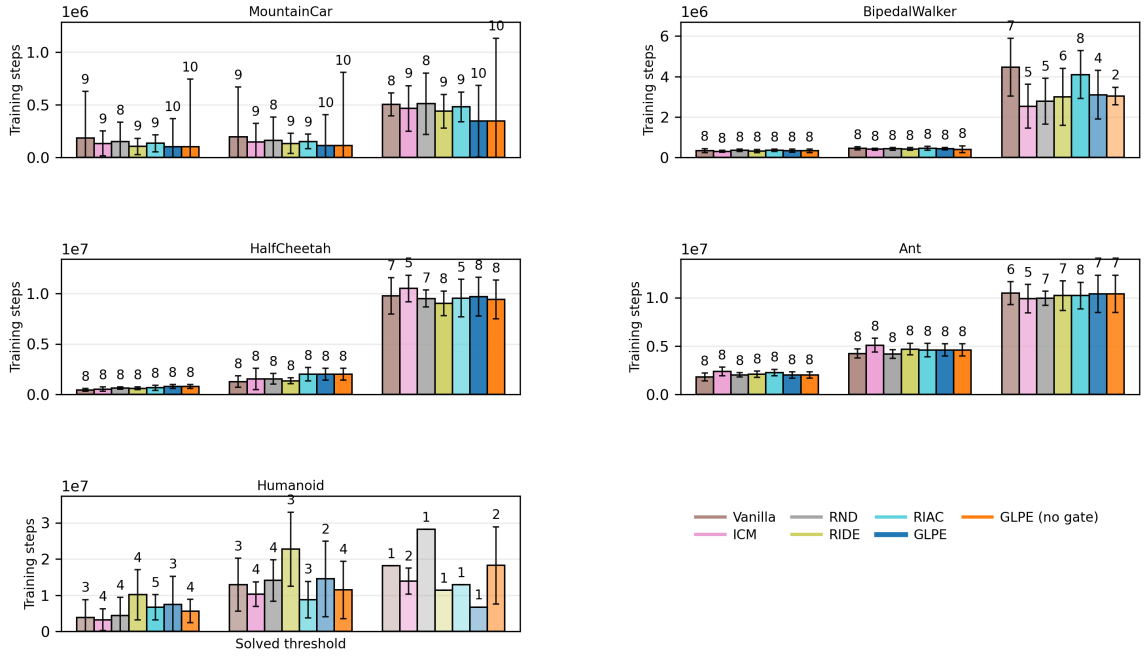


Figure 5.2: Steps to reach fixed return thresholds. Bars show the median number of environment steps among successful seeds. Error bars show standard deviation across successful seeds. Bar opacity and the numeric labels indicate the fraction and count of seeds that reached each threshold. Lower bars with higher opacity indicate faster and more reliable learning.

| Environment      | GLPE          | GLPE (no gate) | Most reliable baseline |
|------------------|---------------|----------------|------------------------|
| MountainCar-v0   | 10/10; 0.10 M | 10/10; 0.10 M  | RIDE: 9/10; 0.11 M     |
| BipedalWalker-v3 | 8/8; 0.33 M   | 8/8; 0.33 M    | ICM: 8/8; 0.30 M       |
| Ant-v5           | 8/8; 2.01 M   | 8/8; 2.01 M    | PPO: 8/8; 1.82 M       |
| HalfCheetah-v5   | 8/8; 0.79 M   | 8/8; 0.79 M    | PPO: 8/8; 0.46 M       |
| Humanoid-v5      | 3/5; 7.47 M   | 4/5; 5.64 M    | RIAC: 5/5; 6.67 M      |

Table 5.3: Reliability and median reaching steps at 25% of the solved threshold. Each entry reports successful seeds over total seeds and median steps to first reach the target.

of five. This early-threshold view shows that GLPE does not prevent basic learning in the dense tasks, even when it is not the fastest method.

### 5.5.2 50 Percent Threshold Results

The 50% threshold is a stricter measure of competent intermediate performance. Table 5.4 shows that GLPE without gating reaches this threshold in every seed on four tasks and in four of five seeds on Humanoid-v5.

| Environment      | GLPE          | GLPE (no gate) | Most reliable baseline |
|------------------|---------------|----------------|------------------------|
| MountainCar-v0   | 10/10; 0.12 M | 10/10; 0.11 M  | RIDE: 9/10; 0.13 M     |
| BipedalWalker-v3 | 8/8; 0.44 M   | 8/8; 0.41 M    | ICM: 8/8; 0.41 M       |
| Ant-v5           | 8/8; 4.61 M   | 8/8; 4.61 M    | RND: 8/8; 4.18 M       |
| HalfCheetah-v5   | 8/8; 2.01 M   | 8/8; 2.01 M    | PPO: 8/8; 1.28 M       |
| Humanoid-v5      | 2/5; 14.51 M  | 4/5; 11.47 M   | ICM: 4/5; 10.32 M      |

Table 5.4: Reliability and median reaching steps at 50% of the solved threshold. Each entry reports successful seeds over total seeds and median steps to first reach the target.

The 50% results show that GLPE is reliable on the simpler sparse-reward task and on the lower-dimensional continuous-control tasks. The main degradation occurs in Humanoid-v5, where high-dimensional dynamics and substantial variance make the gate less reliable. The ungated variant matches the best baseline reach count in Humanoid-v5, although ICM reaches the threshold earlier among successful seeds.

### 5.5.3 Solved-Threshold Results

The solved threshold measures whether a method reaches the conventional task-level target by the end of training. Table 5.5 reports the solved-threshold reliability.

| Environment      | GLPE          | GLPE (no gate) | Most reliable baseline |
|------------------|---------------|----------------|------------------------|
| MountainCar-v0   | 10/10; 0.35 M | 10/10; 0.35 M  | RIDE: 9/10; 0.44 M     |
| BipedalWalker-v3 | 4/8; 3.10 M   | 2/8; 3.04 M    | RIAC: 8/8; 4.09 M      |
| Ant-v5           | 7/8; 10.39 M  | 7/8; 10.39 M   | RIAC: 8/8; 10.22 M     |
| HalfCheetah-v5   | 8/8; 9.68 M   | 8/8; 9.42 M    | RIDE: 8/8; 9.03 M      |
| Humanoid-v5      | 1/5; 6.73 M   | 2/5; 18.23 M   | ICM: 2/5; 13.93 M      |

Table 5.5: Reliability and median reaching steps at the solved threshold. Each entry reports successful seeds over total seeds and median steps to first reach the target.

The solved-threshold results reveal where partial competence does not translate into full task completion. GLPE solves all MountainCar-v0 seeds and all HalfCheetah-v5 seeds, and it solves seven of eight Ant-v5 seeds. However, it solves only half of



the BipedalWalker-v3 seeds and one of five Humanoid-v5 seeds. In BipedalWalker-v3, many runs approach the solved threshold but remain just below it, making the solved count sensitive to the fixed cutoff. In Humanoid-v5, the gap is more substantive because the learned policies vary widely across seeds.

## 5.6 Ablation Results

Ablation experiments isolate the contribution of the two components in the base GLPE reward. The impact-only and learning-progress-only variants follow the definitions in Sections 4.7.1 and 4.7.2, respectively. Both ablations retain the same PPO backbone and evaluation protocol. Figure 5.3 shows the learning curves for the ablation methods.

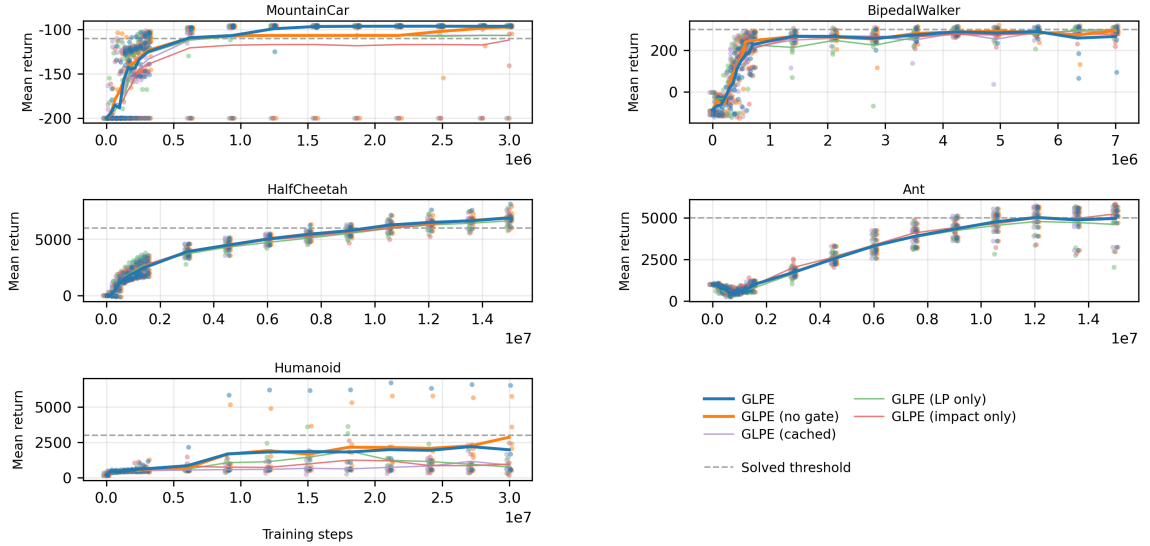


Figure 5.3: Evaluation learning curves for GLPE component ablations. The impact-only variant removes the learning-progress term, and the learning-progress-only variant removes the impact term. Curves report deterministic extrinsic evaluation return.

Table 5.6 reports final-checkpoint returns for the full method and the two single-component ablations.

| Environment      | GLPE                 | Impact-only            | Learning-progress-only |
|------------------|----------------------|------------------------|------------------------|
| MountainCar-v0   | -96.2 [-96.5, -95.9] | -111.8 [-137.0, -96.2] | -106.6 [-127.4, -96.0] |
| BipedalWalker-v3 | 266.5 [214.0, 301.3] | 277.5 [256.0, 296.5]   | 298.8 [285.5, 308.1]   |
| Ant-v5           | 4,961 [4,421, 5,359] | 5,233 [4,544, 5,663]   | 4,599 [3,617, 5,401]   |

Table 5.6: Final-checkpoint extrinsic return for GLPE and component ablations on representative environments. Brackets show 95% bootstrap confidence intervals over seeds. Higher values are better.

### 5.6.1 Impact-Only Performance

The impact-only variant performs competitively in dense locomotion but is weaker in the sparse-reward setting. On Ant-v5, impact-only reaches a final mean return of 5,233, exceeding the full GLPE mean of 4,961. This result indicates that feature-space displacement alone can provide a useful auxiliary signal when the environment already supplies dense extrinsic reward. In such a setting, encouraging controllable state change may complement the extrinsic locomotion objective without requiring local prediction-error improvement to be estimated accurately.

On MountainCar-v0, impact-only reaches  $-111.8$ , substantially below full GLPE. The wide interval  $[-137.0, -96.2]$  indicates inconsistent behavior across seeds. This supports the interpretation that impact without learning progress can reward motion that is energetic but not necessarily productive for discovering the goal-reaching trajectory.

### 5.6.2 Learning-Progress-Only Performance

The learning-progress-only variant performs best among the ablations on BipedalWalker-v3, reaching a final mean return of 298.8. This suggests that local model improvement can be informative in a task where the agent must coordinate contact-rich dynamics and where moderate exploration is useful. The result also indicates that impact can sometimes add noise when the relevant improvement signal is already captured by changes in model competence.

On Ant-v5, learning-progress-only is lower than both full GLPE and impact-only. This outcome is consistent with the possibility that local progress estimates become noisy in higher-dimensional continuous locomotion, especially when extrinsic rewards already guide behavior. On MountainCar-v0, learning-progress-only improves over impact-only but remains below full GLPE, showing that progress alone is not sufficient to fully explain the sparse-reward result.

### 5.6.3 Combined Reward Interpretation

The combined reward is most clearly justified by MountainCar-v0, where neither single-component ablation matches the full method. Feature-space impact identifies transitions that change the learned representation, while learning progress identifies regions where the predictive model is improving. The product of these ideas, implemented as a weighted sum after separate scale normalization, favors transitions that are both behaviorally consequential and locally learnable.

The ablation results also show that the best intrinsic component is task-dependent. Impact-only is strong on Ant-v5, learning-progress-only is strong on BipedalWalker-v3, and the combined method is strongest on MountainCar-v0. This pattern motivates the combined score as a robust default for heterogeneous tasks, while also showing that future work may benefit from adaptive component weighting rather than fixed coefficients.

## 5.7 Gating Behavior

As defined in Section 4.2.9, the gate is designed to suppress intrinsic reward only in latent regions with persistently high prediction error and low learning progress. It is therefore a local mechanism rather than a global decay schedule. Figure 5.4 visualizes the final-checkpoint trajectories of the gated method, with time steps colored according to whether intrinsic shaping is active.

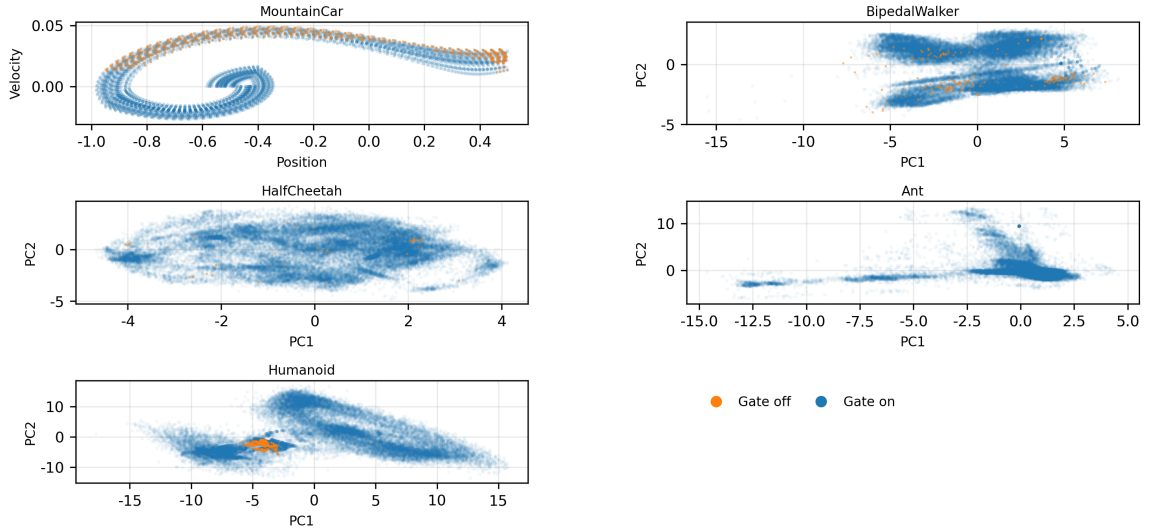


Figure 5.4: State-space view of GLPE gate decisions at final checkpoints. For MountainCar-v0, points show raw position and velocity. For the other environments, z-scored observations are projected onto the first two principal components. Gate-on points receive the base intrinsic reward, while gate-off points have the intrinsic reward suppressed.

### 5.7.1 Gate Activation Patterns

The gate-off points occupy limited regions of the visited state distribution. This pattern is consistent with the intended design: the gate should not eliminate intrinsic motivation

everywhere, but should selectively remove it where the region statistics indicate low progress despite high error. The visualization shows that gate suppression is spatially structured, which suggests that the online latent partition identifies localized regions rather than applying a uniform penalty to all transitions.

Because the gate uses medians across visited regions, the activation pattern depends on the distribution of the agent’s own experience. A region is evaluated relative to other regions currently known to the agent, not relative to an externally fixed difficulty scale. This makes the gate adaptive, but it also means that noisy early experience can influence later suppression decisions if a region receives enough visits to satisfy the persistence rule.

### **5.7.2 Effect in Sparse-Reward Settings**

In MountainCar-v0, the gated method achieves complete solved-threshold reliability and the strongest final performance. Both GLPE and GLPE without gating solve all seeds, but GLPE obtains the stronger step-AUC and the best final mean return among all compared methods. This suggests that gating can remove distracting intrinsic reward while preserving the exploration of controllable state changes needed to reach the goal.

The sparse-reward result is aligned with the gate’s purpose. When extrinsic reward is uninformative for much of training, the agent is vulnerable to intrinsic-reward loops. A local suppression rule can prevent the policy from overvaluing regions that remain difficult to predict but do not yield learning progress. The outcome in MountainCar-v0 shows that this mechanism can improve the usefulness of intrinsic reward in a low-feedback environment.

### **5.7.3 Effect in Dense-Reward and High-Variance Settings**

The gate is less beneficial in dense-reward and high-variance tasks. On BipedalWalker-v3, GLPE without gating has a higher final mean than gated GLPE. On Humanoid-v5, the difference is larger: GLPE without gating reaches a final mean close to ICM, while gated GLPE has a lower mean and lower 50% threshold reliability. These results indicate that suppressing intrinsic reward can be harmful when the progress statistics are noisy or when difficult regions still contain useful behavior.

In Ant-v5 and HalfCheetah-v5, the gated and ungated variants are similar in aggregate learning curves. This suggests that the gate may have limited influence when dense extrinsic rewards dominate the optimization signal. Overall, the empirical evi-

dence supports using gating as a targeted safeguard for unproductive curiosity, not as a default improvement for every continuous-control task.

## 5.8 Intrinsic Reward Dynamics

The intrinsic reward is intended to shape exploration during training rather than to define the final task objective. For this reason, the evaluation protocol in Section 4.6 reports only extrinsic return, while the training diagnostics examine how much intrinsic reward is added to the PPO update. Figure 5.5 decomposes rollout rewards into extrinsic and intrinsic components.

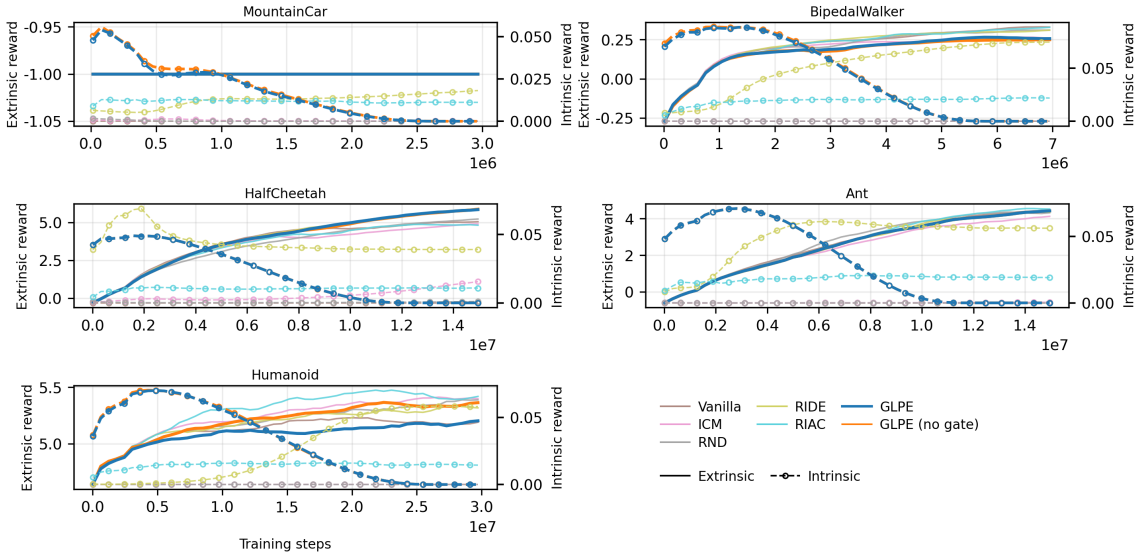


Figure 5.5: Training-time rollout reward decomposition. Solid lines show mean extrinsic reward per environment step collected during PPO rollouts. Dashed lines show the intrinsic shaping component after scaling and clipping. Evaluation metrics reported in Sections 5.2–5.9 use extrinsic return only.

The diagnostic curves show that GLPE and GLPE without gating apply intrinsic shaping primarily as a transient auxiliary signal. The intrinsic coefficient is tapered during training, so the contribution of intrinsic reward decreases as the policy approaches the later stages of the interaction budget. This schedule reduces the chance that the final policy is optimized mainly for auxiliary novelty or progress rather than environment return.

The reward decomposition also clarifies why training rewards cannot be compared directly with evaluation returns. During training, the policy update uses the sum of extrinsic reward and scaled intrinsic reward. During evaluation, only the environment

return is measured. Consequently, a large intrinsic component can accelerate learning if it guides useful exploration, but it can also degrade performance if it encourages behavior that is not aligned with extrinsic return.

For prediction-error methods, intrinsic reward may remain large in regions that are difficult to predict, even when those regions no longer provide useful learning opportunities. GLPE addresses this issue through local learning-progress estimation and optional gating. The observed tapering and gate suppression are therefore complementary mechanisms: tapering reduces the global strength of intrinsic shaping over time, while gating suppresses specific regions that appear unproductive before the global coefficient has decayed.

## **5.9 Wall-Clock Efficiency**

Sample efficiency does not fully describe practical cost. Intrinsic-reward methods add auxiliary computation for encoders, prediction models, statistics, and reward construction. A method can therefore improve return per environment step while reducing return per unit of wall-clock time. This section analyzes the wall-clock metrics defined in Section 4.6.5 and the computational diagnostics defined in Section 4.7.4 to measure this tradeoff.

### **5.9.1 Wall-Clock AUC**

Wall-clock AUC measures evaluation return as a function of elapsed training time rather than environment steps. For each environment, curves are truncated to a common time horizon equal to the minimum final runtime among the compared methods, avoiding extrapolation beyond measured training time. Figure 5.6 reports the resulting wall-clock AUC.

The wall-clock view preserves the main qualitative finding from the step-based analysis but makes overhead more visible. GLPE without gating remains close to the strongest baseline in several environments because it uses the same base intrinsic score without the extra cost of gate thresholding. The gated method is less favorable under wall-clock AUC when the performance gain from suppression does not compensate for the additional computation. This is particularly relevant in dense-control settings, where the extrinsic reward already provides a strong learning signal.

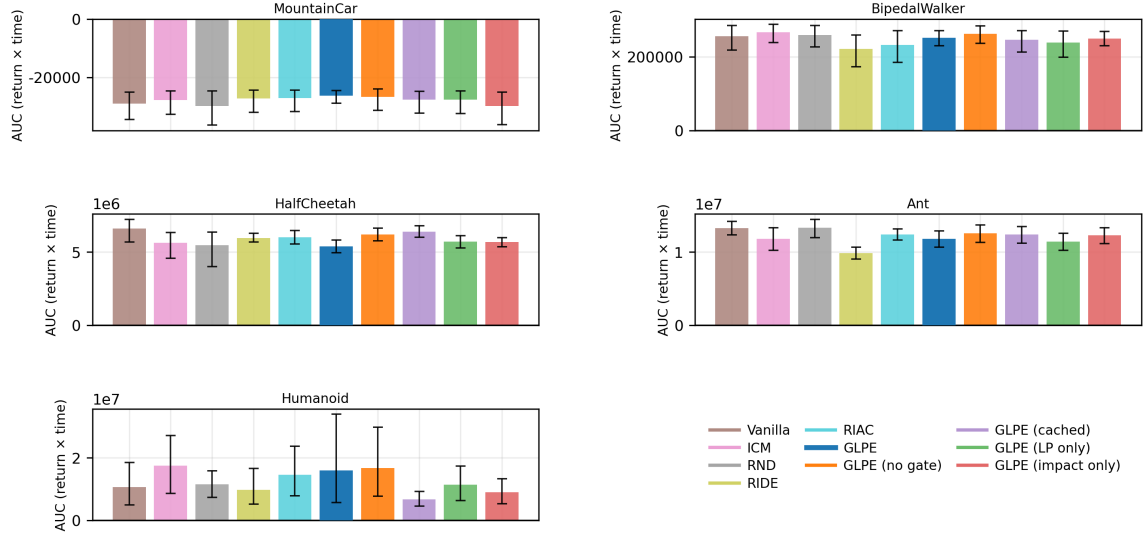


Figure 5.6: Wall-clock AUC of deterministic evaluation return. Each task uses a common runtime horizon so that slower methods are not credited for unobserved future performance. Higher values indicate better return accumulated per unit of training time.

## 5.9.2 Runtime Breakdown

Figure 5.7 decomposes training time per PPO update into environment interaction, policy optimization, intrinsic computation, and related components. The stacked bars show that the relative cost of intrinsic reward depends strongly on the environment.

In computationally expensive locomotion environments, environment stepping and PPO optimization account for much of the total runtime. Intrinsic overhead is therefore a smaller fraction of the full update cost. In inexpensive environments such as MountainCar-v0, the environment itself is cheap to simulate, so auxiliary computation becomes more visible in wall-clock comparisons. This distinction explains why a method can appear sample-efficient on a lightweight environment while still imposing a substantial runtime penalty.

## 5.9.3 Computational Tradeoffs

The most distinctive additional cost in gated GLPE is the computation of robust global medians used to decide whether a local region should be suppressed. A microbenchmark of the gate-median computation shows the cost of recomputing these medians every update relative to using cached medians. Table 5.7 reports the measured throughput.

The benchmark shows that the gate can be made substantially faster by caching

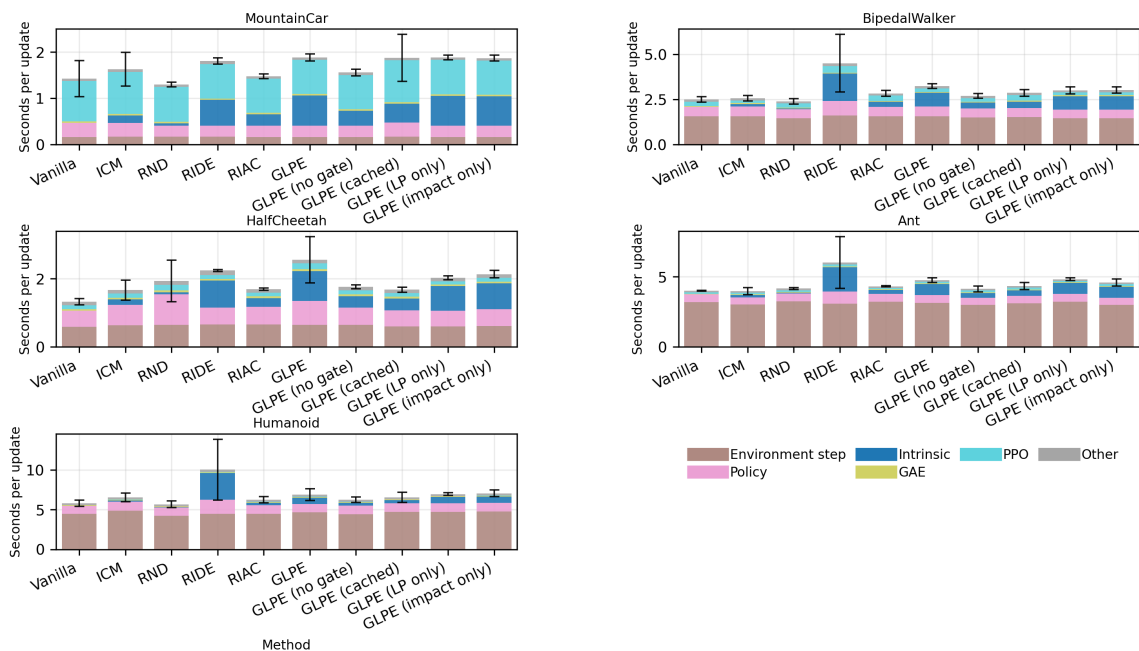


Figure 5.7: Runtime breakdown per PPO update. Each stack reports component-wise median time from the final quarter of training updates, averaged across runs. Black error bars show 95% confidence intervals for total update time across runs.

| Configuration          | Throughput            | Relative speed |
|------------------------|-----------------------|----------------|
| Recompute every update | 20,877 transitions/s  | 1.00×          |
| Cache for 64 updates   | 100,173 transitions/s | 4.83×          |

Table 5.7: Gate-median microbenchmark. Throughput is the median across nine trials for the same workload. Caching medians improves throughput but can make gate decisions depend on slightly stale global references.



global median references. However, cached medians change the exact timing of gate decisions because thresholds are no longer recomputed at every update. For this reason, caching is best interpreted as an engineering optimization rather than as part of the core algorithmic comparison. The main tradeoff is between responsiveness and runtime: frequent median recomputation gives the gate current statistics, while caching improves throughput at the cost of stale thresholds.

### 5.10 Discussion of Findings

The experiments show that GLPE is most effective when the agent must discover useful behavior before extrinsic rewards provide sufficient guidance. This conclusion is supported by MountainCar-v0, where GLPE achieves the best final return, strongest step-AUC, and complete solved-threshold reliability. The result is consistent with the design of the intrinsic reward: feature-space impact encourages behaviorally meaningful transitions, while region-local learning progress favors transitions whose dynamics are becoming more predictable.

The results also show that gating is not universally beneficial. The gate improves the sparse-reward result but can reduce performance in dense-reward or high-variance environments. This is most visible in Humanoid-v5, where gated GLPE has lower final performance and lower threshold reliability than the ungated variant. A plausible explanation is that high-dimensional locomotion produces noisy local progress estimates, causing the gate to suppress regions that are difficult but still useful for learning. Thus, the empirical role of the gate is best described as a selective guardrail against unproductive curiosity, not as a general-purpose performance booster.

The ablation results indicate that the two reward components contribute differently across tasks. Impact-only exploration is competitive in Ant-v5, learning-progress-only exploration is strong in BipedalWalker-v3, and the full combination is necessary for the strongest MountainCar-v0 result. This pattern suggests that the fixed component weights used in the experiments are a reasonable general setting but are unlikely to be optimal for every environment. An adaptive weighting mechanism could potentially improve robustness by increasing the importance of learning progress in sparse-reward settings and increasing the importance of impact in dense-control settings.

The threshold analyses reveal that partial competence and solved performance should be distinguished. GLPE and GLPE without gating reach 25% and 50% thresholds reliably in most environments, but full solved-threshold reliability is more difficult in BipedalWalker-v3 and Humanoid-v5. This distinction is important because mean learn-

ing curves can make methods appear similar even when their probability of reaching a practical target differs across seeds. Reliability metrics therefore provide information that is not captured by mean return alone.

Wall-clock results show that computational overhead must be considered when evaluating intrinsic-reward methods. The gated method adds region statistics and robust threshold computations, and these costs matter most when environment simulation is inexpensive. The ungated variant often provides a better runtime-performance balance because it retains the combined impact–learning-progress reward without the additional gate computation. For practical use, the choice between gated and ungated GLPE should therefore depend on whether the expected benefit of suppressing unproductive regions is likely to exceed the added runtime cost.

Overall, the findings support the central premise of this study: learning progress can be combined with feature-space impact to produce an intrinsic reward that is useful for exploration, particularly in sparse-reward settings. At the same time, the results identify limitations in high-dimensional and dense-reward environments, where the gate may become conservative and where standard baselines remain competitive. The method should therefore be viewed as a contribution to targeted intrinsic motivation rather than as a universally dominant reinforcement-learning algorithm.

## CHAPTER 6

### CONCLUSION AND RECOMMENDATIONS

#### 6.1 Summary of the Study

This study addressed the exploration problem in reinforcement learning under sparse, delayed, and heterogeneous reward conditions. The central problem was that conventional intrinsic-reward signals can encourage exploration but may also reward unproductive behavior when novelty, surprise, or prediction error remains high in regions that are noisy, difficult to model, or weakly related to task progress. To address this issue, the study developed and evaluated Gated Learning-Progress Exploration (GLPE), an intrinsic-reward method that combines feature-space impact with region-local learning progress and optionally suppresses regions that exhibit persistently high prediction error with low improvement.

The proposed method was formulated for episodic Markov decision processes and integrated with Proximal Policy Optimization through reward augmentation. For each transition, an encoder maps observations to a latent representation, a forward dynamics model estimates the next latent state, and an inverse dynamics model regularizes the representation toward action-relevant features. Feature-space impact is measured as the latent displacement

$$I_t = \|z_{t+1} - z_t\|_2, \quad (6.1)$$

while local learning progress is computed from the difference between long- and short-horizon exponential moving averages of forward prediction error in the assigned latent region:

$$\text{LP}(r) = \max(0, \mu_r^{\text{long}} - \mu_r^{\text{short}}). \quad (6.2)$$

After component-wise scale normalization, these quantities form the base intrinsic score

$$u_t = \alpha_{\text{impact}} \tilde{I}_t + \alpha_{\text{LP}} \tilde{\text{LP}}_t. \quad (6.3)$$

The ungated variant uses this score directly, whereas the gated variant multiplies it by a region-specific binary gate that disables intrinsic reward in regions classified as high-error and low-progress. The resulting augmented reward used for policy optimization is

$$r_t = r_t^{\text{ext}} + \eta_t \text{clip}(r_t^{\text{int}}, -r_{\text{max}}, r_{\text{max}}), \quad (6.4)$$

with an intrinsic coefficient schedule that reduces auxiliary shaping later in training.

The empirical evaluation compared GLPE and GLPE without gating against PPO, ICM, RND, RIDE, and RIAC on five Gymnasium control benchmarks: MountainCar-v0, BipedalWalker-v3, Ant-v5, HalfCheetah-v5, and Humanoid-v5. These environments were selected to cover low-dimensional sparse-reward exploration, contact-rich locomotion, dense continuous-control reward structures, and high-dimensional high-variance dynamics. The evaluation used deterministic offline checkpoints and reported extrinsic return only, ensuring that performance reflected the environment objective rather than the magnitude of intrinsic reward. The analysis included learning curves, final-checkpoint return, step-normalized area under the curve, thresholded reliability, component ablations, gate visualizations, intrinsic reward dynamics, and wall-clock efficiency.

The results show that GLPE is most effective in sparse-reward exploration. On MountainCar-v0, the gated method achieved the strongest final return among the compared methods, the strongest step-AUC, and complete solved-threshold reliability across all seeds. This supports the hypothesis that combining impact and learning progress can identify transitions that are both behaviorally meaningful and locally learnable. The ungated method was generally more competitive in dense-reward and high-variance environments, especially where the gate became conservative or where extrinsic reward already supplied frequent guidance. In BipedalWalker-v3 and Humanoid-v5, GLPE without gating produced stronger final performance than the gated method, indicating that suppressing intrinsic reward can be harmful when local progress estimates are noisy or when difficult regions remain useful for control learning.

The ablation results further clarify the role of the two intrinsic components. In the sparse-reward setting, the full combination outperformed both the impact-only and learning-progress-only variants, showing that neither component alone was sufficient to reproduce the main result. In contrast, impact-only exploration was competitive on Ant-v5, and learning-progress-only exploration was strong on BipedalWalker-v3. These outcomes indicate that the usefulness of each component depends on the structure of the environment and that fixed component weights may not be optimal across all tasks.

The wall-clock analysis showed that intrinsic rewards must be evaluated not only by sample efficiency but also by computational cost. The ungated variant often achieved a more favorable runtime-performance balance because it retains the combined impact–learning-progress signal without the additional median computations and gate updates required by the gated method. The gated variant remains most justified when the benefit of suppressing unproductive curiosity compensates for the added overhead, as

observed most clearly in the sparse-reward environment. Overall, the study demonstrated that learning progress and feature-space impact can be combined into a practical intrinsic-reward family, but it also showed that gating should be treated as a selective guardrail rather than as a universally beneficial modification.

## 6.2 Conclusions

The study concludes that feature-space impact and region-local learning progress provide complementary information for intrinsic-reward exploration. Impact identifies transitions that cause meaningful change in the learned representation, while learning progress distinguishes regions where the predictive model is improving from regions that are already predictable or persistently difficult. When combined after independent scale normalization, the two components form an intrinsic signal that is more robust in sparse-reward exploration than either component alone. This conclusion is supported most directly by MountainCar-v0, where the combined method outperformed the single-component ablations and achieved reliable solved-threshold performance.

The second conclusion is that local learning progress is a useful corrective to prediction-error curiosity but is not sufficient as a complete exploration objective in all environments. A high prediction error can indicate either an informative opportunity or an unproductive region. By estimating progress as a local reduction in forward-model error, GLPE favors regions where additional experience appears to improve predictive competence. However, the learning-progress-only ablation did not dominate across the benchmark suite, which shows that model improvement alone can omit behaviorally important transitions when it is not paired with an impact term.

The third conclusion is that gating is valuable primarily as a targeted safeguard against unproductive curiosity. The gated method improved the sparse-reward result by suppressing intrinsic shaping in selected high-error low-progress regions, and gate visualizations indicated that suppression was localized rather than global. However, the same mechanism reduced performance in some dense-reward and high-variance settings. This pattern shows that a gate based on local progress statistics can be too conservative when difficult regions remain useful or when high-dimensional dynamics make progress estimates unstable. Therefore, gating should not be interpreted as a universally superior extension of the base reward. It is best understood as an optional guardrail whose value depends on the environment’s reward sparsity, stochasticity, and representation stability.

The fourth conclusion is that GLPE without gating is the stronger default when

computational cost and cross-environment robustness are considered together. The ungated variant preserves the principal technical contribution of the method: the combination of normalized feature-space impact and region-local learning progress. It avoids the additional overhead of robust gate thresholding and performed competitively under both step-based and wall-clock-normalized views. In environments with dense extrinsic feedback, the ungated variant often matched or exceeded the gated variant, suggesting that the base intrinsic reward already provides useful shaping without requiring suppression.

The fifth conclusion is that intrinsic-reward methods require multidimensional evaluation. Learning curves and final returns alone do not fully characterize exploration performance. Step-AUC identifies early and sustained learning, thresholded reliability distinguishes consistent success from isolated high-return seeds, ablations reveal component-level behavior, and wall-clock AUC accounts for computational overhead. The findings of this study would be less precise if evaluated only by final mean return, because some methods reached partial competence reliably without consistently reaching solved thresholds, while others imposed runtime costs that changed their practical value.

The final conclusion is that the proposed method improves exploration under the conditions for which it was designed but does not establish universal dominance over all intrinsic-reward approaches. Its strongest evidence appears in sparse-reward exploration, where the auxiliary signal must compensate for limited external feedback. In dense locomotion and high-variance control, standard baselines remain competitive, and the gate may suppress useful exploration. Consequently, GLPE should be viewed as a contribution to targeted intrinsic motivation: it provides a principled way to combine controllable representation change with local model improvement while exposing clear tradeoffs in gating and computational cost.

### **6.3 Contributions**

The first contribution of this study is the formulation of a learning-progress intrinsic-reward family that combines latent transition impact with local predictive improvement. The method does not reward prediction error directly. Instead, it rewards transitions in which the learned representation changes and the corresponding latent region shows evidence of decreasing forward-model error. This formulation addresses the limitation of curiosity signals that treat surprise as intrinsically valuable even when additional experience does not lead to improved prediction.

The second contribution is an online latent-space partitioning procedure for estimating local learning progress during on-policy reinforcement learning. The partition stores region-specific prediction-error statistics and increases resolution in visited portions of the latent space through bounded recursive splitting. This allows the method to distinguish heterogeneous dynamics without requiring a fixed hand-designed state abstraction. The use of short- and long-horizon exponential moving averages provides a compact estimate of whether predictive competence is improving within each region.

The third contribution is a region-wise gating mechanism for suppressing unproductive intrinsic reward. The gate compares each region’s learning progress and recent prediction error against robust references computed over visited regions, and it suppresses the base intrinsic score when a region remains high-error and low-progress for a sufficient duration. This mechanism operationalizes the distinction between learnable difficulty and unproductive unpredictability. The empirical results show that this guardrail can support strong sparse-reward performance while also revealing the conditions under which it may become overly conservative.

The fourth contribution is a PPO-compatible integration of the proposed intrinsic reward with explicit reward-scale controls. The method uses component-wise normalization, intrinsic clipping, terminal-transition suppression, and a cosine taper for the intrinsic coefficient. These design choices make the intrinsic reward usable as a transient training aid rather than as a permanent replacement for the task objective. The resulting procedure can be compared directly with established intrinsic-reward baselines under the same policy-optimization backbone.

The fifth contribution is an empirical evaluation that considers performance, reliability, ablation behavior, gate dynamics, and computational efficiency together. The benchmark suite covers both sparse-reward and continuous-control regimes, and the evaluation reports deterministic extrinsic return from offline checkpoints rather than training reward. The inclusion of step-AUC, final-checkpoint return, thresholded reliability, wall-clock AUC, and timing breakdowns provides a broad assessment of how intrinsic rewards affect both learning quality and training cost.

The sixth contribution is the characterization of when the proposed components are useful. The results show that the combined impact–learning-progress reward is most beneficial in sparse-reward exploration, that single components can be competitive in some dense-control settings, and that gating is advantageous only when suppression of unproductive regions outweighs its conservatism and overhead. This characterization is important because it frames the method as an environment-dependent exploration mechanism rather than as a universally dominant algorithm.

## 6.4 Limitations

The first limitation is the restricted environment scope. The experiments used five vector-observation control benchmarks and did not include image observations, language-conditioned environments, multi-agent settings, offline reinforcement learning, model-based planning, or real-world robotic deployment. The conclusions therefore apply most directly to online, on-policy, simulated control tasks with vector observations. Tasks with richer sensory input may require different representation-learning assumptions, larger auxiliary models, or different mechanisms for estimating local progress.

The second limitation is that the method was evaluated with PPO as the reinforcement-learning backbone. PPO provides a stable on-policy setting for controlled comparison, but the behavior of the intrinsic reward may differ under off-policy algorithms, replay buffers, distributed actors, or value-based methods. The local progress statistics in GLPE are computed from on-policy experience, and their interpretation may change if transitions are repeatedly sampled from a replay buffer or collected by many asynchronous policies.

The third limitation concerns reward shaping. The augmented reward used during training is not a potential-based transformation of the environment reward and is not guaranteed to preserve the optimal policy of the original task. Although evaluation uses extrinsic return only, the intrinsic component can still change the policy optimization trajectory and may favor behaviors that are not aligned with the final task objective. The intrinsic coefficient schedule and clipping range reduce this risk but do not eliminate it.

The fourth limitation is sensitivity to representation quality. Feature-space impact assumes that distances in the learned latent representation correspond to meaningful transition changes. Learning progress assumes that forward-model prediction error reflects local competence. If the encoder fails to preserve action-relevant structure, if the inverse model is insufficiently informative, or if the latent partition mixes unrelated dynamics, then both impact and progress can become unreliable. This limitation is especially relevant for high-dimensional environments where many different behaviors may map to nearby latent regions.

The fifth limitation is the fixed design of component weights and gate thresholds. The experiments used a fixed impact weight and environment-specific settings for the learning-progress weight and other hyperparameters. The ablation results show that different environments favor different components, suggesting that static weights cannot be optimal for all tasks. Similarly, the gate relies on median-based references,



persistence rules, and hysteresis parameters that may not adapt well to every reward structure or dynamics distribution.

The sixth limitation is the computational cost of intrinsic-reward computation. The method requires an auxiliary encoder, forward model, inverse model, online region statistics, component normalization, and optional gating. The overhead is especially visible in inexpensive environments where environment interaction is fast. The gated variant adds further cost through robust threshold computation and gate-state updates. Although caching can improve throughput, it introduces stale references and can alter gate decisions.

The seventh limitation is the number of random seeds and benchmark scale. The study reports multiple seeds and uncertainty-aware summaries, but high-variance environments such as Humanoid-v5 still exhibit wide confidence intervals. A larger seed count or a broader benchmark suite would provide stronger evidence about method rankings in difficult continuous-control settings. The current results are sufficient to identify qualitative behavior and tradeoffs, but they do not prove that small differences between methods are stable across all possible training configurations.

The final limitation is that the study analyzes unproductive curiosity through prediction-error and learning-progress statistics rather than through a formal guarantee. The gate is designed to suppress regions that empirically appear high-error and low-progress, but this classification can be incorrect. A region may temporarily show low progress while still being necessary for later task success, or it may appear learnable under the model while remaining irrelevant to extrinsic return. Thus, the method reduces a known failure mode of curiosity-driven exploration but does not fully solve the broader problem of aligning intrinsic objectives with task objectives.

## 6.5 Recommendations for Future Work

Future work should first investigate adaptive weighting between feature-space impact and learning progress. The ablation results indicate that the most useful component varies across environments: the combined reward is strongest in sparse-reward exploration, impact alone can be competitive in dense locomotion, and learning progress alone can be effective in contact-rich control. An adaptive mechanism could adjust  $\alpha_{\text{impact}}$  and  $\alpha_{\text{LP}}$  according to observed reward sparsity, progress stability, or the correlation between intrinsic reward and later extrinsic improvement. Such a mechanism may reduce the need for environment-specific tuning.

A second direction is to improve the gate so that it is less conservative in high-

variance environments. The current binary gate suppresses a region when local statistics indicate persistent high error and low progress. Future variants could use a continuous attenuation factor rather than a hard zero-one decision, uncertainty-aware thresholds, or a recovery rule that reactivates regions when later evidence indicates renewed progress. A softer gate could preserve protection against unproductive curiosity while reducing the risk of suppressing difficult but useful parts of the state space.

A third direction is to extend the method beyond vector observations. Image-based environments would require encoders that produce stable action-relevant latent spaces under visual variation, partial observability, and irrelevant background changes. The impact term and latent partition should be evaluated with convolutional or transformer-based encoders, and the progress signal should be tested under representations learned jointly from high-dimensional observations. This extension would clarify whether GLPE’s assumptions about latent distances and forward-model error hold in richer perception settings.

A fourth direction is to evaluate GLPE under other reinforcement-learning backbones. Off-policy actor-critic methods, replay-based algorithms, and distributed training may interact differently with learning-progress statistics because the data distribution is no longer purely on-policy. Future work should examine whether region-local EMAs should be updated from the most recent behavior policy, from replay samples, or from importance-weighted statistics. This would determine whether the intrinsic reward can be generalized beyond PPO while preserving the interpretation of local model improvement.

A fifth direction is to study more principled region construction. The current online partition uses bounded recursive splitting in latent space, which is simple and computationally controlled. Alternative region definitions could use clustering, density models, learned prototypes, successor features, or uncertainty-aware partitioning. These alternatives may produce regions that better align with local dynamics, especially in high-dimensional environments where axis-aligned splitting may not capture the geometry of behaviorally meaningful transitions.

A sixth direction is to reduce computational overhead. The wall-clock results show that the practical value of intrinsic rewards depends on training cost as well as sample efficiency. Future implementations could approximate robust gate references incrementally, update gates less frequently with error bounds, share computation between intrinsic modules and policy networks, or use lightweight learned summaries of region statistics. The objective should be to preserve the exploration signal while improving return per unit of wall-clock time.

A seventh direction is to expand the empirical evaluation. Additional sparse-reward tasks, stochastic environments, long-horizon navigation problems, and tasks with distractor dynamics would provide a stronger test of the method’s intended purpose. More seeds should be used in high-variance domains to separate true method differences from training noise. Evaluation should continue to include both step-based and wall-clock-normalized metrics, because computational overhead is an essential part of practical reinforcement-learning performance.

Finally, future work should investigate stronger links between intrinsic reward and downstream extrinsic value. Learning progress identifies where the model is improving, but improvement in prediction does not always imply improvement in task performance. A promising extension is to estimate whether intrinsically rewarded transitions increase the probability of later extrinsic return, threshold crossing, or value-function improvement. Such an extension could preserve the benefits of curiosity-driven exploration while more directly aligning intrinsic motivation with the external objective.

## BIBLIOGRAPHY

- Badia, A. P., Sprechmann, P., Vitvitskyi, A., Guo, D., Piot, B., Kapturowski, S., Tieleman, O., Arjovsky, M., Pritzel, A., Bolt, A., & Blundell, C. (2020). Never Give Up: Learning Directed Exploration Strategies. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.2002.06038>
- Baranes, A., & Oudeyer, P.-Y. (2009). R-IAC: Robust Intrinsically Motivated Exploration and Active Learning. *IEEE Transactions on Autonomous Mental Development*, 1(3), 155–169. <https://doi.org/10.1109/tamd.2009.2037513>
- Barto, A. G. (2012, November). *Intrinsic Motivation and Reinforcement Learning*. [https://doi.org/10.1007/978-3-642-32375-1\\_2](https://doi.org/10.1007/978-3-642-32375-1_2)
- Bellemare, M. G., Srinivasan, S., Ostrovski, G., Schaul, T., Saxton, D., & Munos, R. (2016). Unifying Count-Based Exploration and Intrinsic Motivation. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.1606.01868>
- Burda, Y., Edwards, H., Pathak, D., Storkey, A., Darrell, T., & Efros, A. A. (2018). Large-Scale Study of Curiosity-Driven Learning. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.1808.04355>
- Burda, Y., Edwards, H., Storkey, A., & Klimov, O. (2018). Exploration by Random Network Distillation. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.1810.12894>
- Houthooft, R., Chen, X., Duan, Y., Schulman, J., De Turck, F., & Abbeel, P. (2016). VIME: Variational Information Maximizing Exploration. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.1605.09674>
- Mavor-Parker, A. N., Young, K. A., Barry, C., & Griffin, L. D. (2021). How to Stay Curious While Avoiding Noisy TVs Using Aleatoric Uncertainty Estimation. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.2102.04399>
- Ng, A., Harada, D., & Russell, S. (1999). Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping, 278–287. <https://doi.org/10.5555/645528.657613>
- Ostrovski, G., Bellemare, M. G., van den Oord, A., & Munos, R. (2017). Count-Based Exploration with Neural Density Models. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.1703.01310>
- Oudeyer, P.-Y., & Kaplan, F. (2007). What Is Intrinsic Motivation? A Typology of Computational Approaches. *Frontiers in Neurorobotics*, 1, 6. <https://doi.org/10.3389/neuro.12.006.2007>
- Oudeyer, P.-Y., Kaplan, F., & Hafner, V. V. (2007). Intrinsic Motivation Systems for Autonomous Mental Development. *IEEE Transactions on Evolutionary Computation*, 11(2), 265–286. <https://doi.org/10.1109/tevc.2006.890271>
- Pathak, D., Agrawal, P., Efros, A., & Darrell, T. (2017). Curiosity-Driven Exploration by Self-Supervised Prediction. *arXiv*. <https://doi.org/10.48550/arXiv.1705.05363>
- Pathak, D., Gandhi, D., & Gupta, A. (2019). Self-Supervised Exploration via Disagreement. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.1906.04161>

- Raileanu, R., & Rocktäschel, T. (2020). RIDE: Rewarding Impact-Driven Exploration for Procedurally-Generated Environments. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.2002.12292>
- Savinov, N., Raichuk, A., Marinier, R., Vincent, D., Pollefeys, M., Lillicrap, T., & Gelly, S. (2018). Episodic Curiosity through Reachability. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.1810.02274>
- Schmidhuber, J. (1991, February). *A Possibility for Implementing Curiosity and Boredom in Model-Building Neural Controllers*. <https://doi.org/10.7551/mitpress/3115.003.0030>
- Schulman, J., Moritz, P., Levine, S., Jordan, M., & Abbeel, P. (2015). High-Dimensional Continuous Control Using Generalized Advantage Estimation. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.1506.02438>
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. *arXiv*. <https://doi.org/10.48550/arXiv.1707.06347>
- Singh, S., Barto, A., & Chentanez, N. (2004). Intrinsically Motivated Reinforcement Learning.
- Stadie, B. C., Levine, S., & Abbeel, P. (2015). Incentivizing Exploration in Reinforcement Learning with Deep Predictive Models. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.1507.00814>
- Tang, H., Houthooft, R., Foote, D., Stooke, A., Chen, X., Duan, Y., Schulman, J., De Turck, F., & Abbeel, P. (2016). #Exploration: a study of count-based exploration for deep reinforcement learning. *arXiv (Cornell University)*, 30, 2750–2759. <https://doi.org/10.48550/arxiv.1611.04717>

# APPENDICES

## Technical Manuscript

### Gated Learning-Progress Exploration

Mikael Vincent Tumamos  
University of San Carlos  
Cebu City, Philippines  
tumamos@mikaelvincent.dev

Ervin Joshua Guirnela  
University of San Carlos  
Cebu City, Philippines  
17100409@usc.edu.ph

Christine Peña  
University of San Carlos  
Cebu City, Philippines  
cpena@usc.edu.ph

#### Abstract

Exploration via intrinsic rewards can accelerate learning when extrinsic feedback is sparse or deceptive, yet many intrinsic objectives overemphasize novelty in regions of high uncertainty where learning is slow or unstable. We introduce *Gated Learning-Progress Exploration* (GLPE), a simple intrinsic-reward family that prioritizes transitions that (i) induce meaningful changes in a learned feature representation and (ii) occur in regions where a forward dynamics model is actively improving. GLPE estimates region-local learning progress using the discrepancy between short- and long-horizon exponential moving averages (EMAs) of prediction error within an online partition of feature space, and combines it with a feature-space impact term. An optional region-specific gate suppresses intrinsic rewards in regions where prediction error remains high but learning progress is low, providing a targeted guardrail against unproductive curiosity. We also study *GLPE (no gate)*, which uses the same base intrinsic score without any gating. We evaluate both variants with a common PPO backbone on five Gymnasium control benchmarks and compare against widely used intrinsic-reward baselines, reporting performance under both step- and wall-clock-normalized views. Across the suite, GLPE (no gate) matches the strongest baseline within uncertainty, while the full gated variant is most beneficial in sparse-reward regimes where ungated intrinsic rewards can destabilize training.

#### Keywords

reinforcement learning, intrinsic motivation, exploration, learning progress

#### 1 Introduction

Deep reinforcement learning agents can learn effectively when dense extrinsic rewards provide informative gradients throughout training; however, performance often degrades when rewards are sparse, delayed, or shaped in ways that do not guide early exploration. Intrinsic-motivation methods address this by augmenting the environment reward with an auxiliary signal that encourages exploration even when external feedback is absent or uninformative [3, 11, 12, 20].

Prior intrinsic objectives span several complementary perspectives. Count-based and density-model visitation bonuses encourage novelty and broad coverage [4, 10, 22]. Information-gain objectives seek transitions that reduce uncertainty in a learned model [7]. Prediction-error bonuses reward surprise under a forward model, including early deep predictive approaches [21] and feature-space curiosity methods such as ICM [13]. Random Network Distillation (RND) and related curiosity variants operationalize novelty through prediction error against a fixed target [5, 6], while episodic mechanisms and hybrid schemes further shape exploration using reachability memory, disagreement, or directed exploration strategies

[1, 14, 16]. Impact-driven rewards instead encourage controllable state change, providing a complementary signal when novelty alone is not task-aligned [15]. These intrinsic signals are typically applied as reward shaping during policy optimization; unlike potential-based shaping, they are not generally policy-invariant and can change the optimal policy, so they are assessed empirically [9].

Despite strong empirical results, many intrinsic objectives exhibit a recurring failure mode: *unproductive curiosity*. Novelty or prediction error can remain high in regions that are difficult to model—because dynamics are complex, stochastic, or only weakly coupled to task progress—causing the agent to spend substantial effort collecting experience that improves neither the intrinsic model nor the task policy. The “noisy TV” phenomenon is one concrete example of this pathology [8]. The issue is especially acute in sparse-reward settings, where the intrinsic signal can dominate early learning.

A complementary perspective is to reward *learning progress*: experience is valuable when it improves a predictive model over time rather than merely being surprising [2, 11, 17]. Learning progress can help distinguish regions that are learnable-but-unmastered from regions that are either already predictable or effectively noisy. Practical progress-based methods must still address how to localize progress in high-dimensional observations, how to couple it to exploration-relevant change, and how to avoid spending effort in regions where progress has stalled.

We propose *Gated Learning-Progress Exploration* (GLPE), a lightweight intrinsic-reward family that couples learning progress to representation change. GLPE measures *impact* as the magnitude of change in a learned feature representation between consecutive observations, and estimates *region-local learning progress* by tracking forward-model prediction error within an online partition of the feature space using short- and long-horizon exponential moving averages. The resulting base score favors transitions that both change the representation and occur in regions where the dynamics model is actively improving. To mitigate unproductive curiosity, GLPE optionally applies a simple, region-specific *gate* that suppresses intrinsic shaping in regions with persistently high prediction error but low learning progress, serving as a targeted guardrail rather than a global annealing heuristic. We also study *GLPE (no gate)*, which uses the same base score without gating and serves as a low-overhead default.

We evaluate both GLPE variants with a common PPO backbone on MOUNTAINCAR-v0, BIPEDALWALKER-v3, and three MuJoCo locomotion tasks (ANT-v5, HALFCHEETAH-v5, HUMANOID-v5), comparing against widely used intrinsic-reward baselines including ICM [13], RND [6], RIDE [15], and RIAC [2]. Across this suite, GLPE (no gate) remains consistently competitive under both step- and wall-clock-normalized views, while the gated variant is most helpful

in sparse-reward regimes where ungated shaping can destabilize training (Section 4).

## 2 Method

### 2.1 Problem setup

We consider an episodic Markov decision process with observations  $o_t \in \mathcal{O}$ , actions  $a_t \in \mathcal{A}$ , and extrinsic rewards  $r_t^{\text{ext}}$ . A stochastic policy  $\pi_\theta(a | o)$  is trained to maximize the expected discounted return.

We augment the extrinsic reward with an intrinsic term computed per transition,

$$r_t = r_t^{\text{ext}} + \eta_t \text{clip}(r_t^{\text{int}}, -r_{\max}, r_{\max}), \quad (1)$$

where  $\eta_t \geq 0$  controls the intrinsic-extrinsic trade-off and  $r_{\max} > 0$  bounds the magnitude of intrinsic shaping. For environment-terminal transitions, we set  $r_t^{\text{int}} = 0$  so that intrinsic shaping does not depend on episode termination.

We propose two intrinsic reward formulations within a shared framework: GLPE and GLPE (no gate). Both combine feature-space impact with region-local learning progress. GLPE additionally introduces a region-specific gate that suppresses intrinsically rewarding but unproductive experience. We refer to these two variants collectively as the *GLPE family*.

### 2.2 Latent dynamics model

Let  $\phi_o$  be a learned encoder producing a  $d$ -dimensional feature vector  $z_t = \phi_o(o_t)$ . We train (i) a forward dynamics predictor  $f_\phi$  that maps  $(z_t, a_t)$  to a prediction of  $z_{t+1}$ , and (ii) an inverse dynamics predictor  $g_\psi$  that maps  $(z_t, z_{t+1})$  to an action distribution. The inverse model is instantiated as a classifier for discrete actions and as a Gaussian likelihood model for continuous actions.

Training uses the combined loss

$$\mathcal{L}_{\text{dyn}}(t) = \beta_{\text{fwd}} \mathcal{L}_{\text{fwd}}(t) + \beta_{\text{inv}} \mathcal{L}_{\text{inv}}(t), \quad (2)$$

with weights  $\beta_{\text{fwd}}, \beta_{\text{inv}} > 0$ . The forward loss is the mean-squared error in feature space,

$$\mathcal{L}_{\text{fwd}}(t) = \frac{1}{d} \|f_\phi(z_t, a_t) - z_{t+1}\|_2^2, \quad (3)$$

and we define the per-transition prediction error

$$e_t = \mathcal{L}_{\text{fwd}}(t). \quad (4)$$

The intrinsic rewards below depend on  $(z_t, z_{t+1})$  and  $e_t$ , but do not require access to privileged environment state.

### 2.3 Online region partitioning in latent space

To localize learning progress, we maintain an online partition of the latent space into regions. Each region corresponds to a leaf in a binary space-partitioning tree over the  $d$ -dimensional feature space.

Given an embedding  $z_t$ , we assign it to a leaf region  $\rho_t$  by routing it through the tree. Each leaf maintains a visit counter and, while it remains splittable, a buffer of points used to choose a split dimension and threshold. Once a leaf has accumulated  $C$  assigned points and its depth is below  $D_{\max}$ , we split it by selecting the highest-variance coordinate that yields a non-degenerate partition and cutting at the median along that coordinate. One child retains the parent region

identifier (and its accumulated statistics); the other child receives a new identifier whose statistics are initialized on its first visit. This procedure yields a growing set of regions that progressively refines frequently visited parts of feature space.

We update region statistics (defined next) using the region assignment for the current embedding  $z_t$ . The capacity  $C$  and maximum depth  $D_{\max}$  control the granularity of the partition.

### 2.4 Region-local learning progress

For each region  $r$  we maintain two exponential moving averages (EMAs) of the forward prediction error: a long-horizon average  $\mu_r^{\text{long}}$  and a short-horizon average  $\mu_r^{\text{short}}$ . When a transition at time  $t$  is assigned to region  $\rho_t = r$ , we update

$$\mu_r^{\text{long}} \leftarrow \beta_{\text{long}} \mu_r^{\text{long}} + (1 - \beta_{\text{long}}) e_t, \quad (5)$$

$$\mu_r^{\text{short}} \leftarrow \beta_{\text{short}} \mu_r^{\text{short}} + (1 - \beta_{\text{short}}) e_t, \quad (6)$$

with  $0 < \beta_{\text{long}} < 1$ ,  $0 < \beta_{\text{short}} < 1$ , and typically  $\beta_{\text{long}} > \beta_{\text{short}}$  so that  $\mu_r^{\text{long}}$  changes more slowly. On the first visit to a newly created region identifier, we initialize  $\mu_r^{\text{long}} = \mu_r^{\text{short}} = e_t$ .

We define the current learning progress of a region  $r$  as the positive part of the difference between the long and short averages,

$$\text{LP}(r) = \max(0, \mu_r^{\text{long}} - \mu_r^{\text{short}}). \quad (7)$$

For each transition we use the region assignment  $\rho_t$  and set  $\text{LP}_t = \text{LP}(\rho_t)$ . This quantity is large when the short-term error has recently dropped below the longer-term baseline, indicating active model improvement in that region.

### 2.5 Impact in feature space

We also compute a feature-space impact term that encourages transitions that change the learned representation,

$$I_t = \|z_{t+1} - z_t\|_2. \quad (8)$$

Unlike count-based novelty, impact depends on the learned representation and can capture structured changes in high-dimensional observations.

### 2.6 Component-wise scale stabilization

The magnitudes of  $I_t$  and  $\text{LP}_t$  can vary across tasks and throughout training. To obtain a stable scale without task-specific tuning, we maintain running root-mean-square (RMS) normalizers for each component via an exponential moving average of squared values. For a scalar signal  $x_t$  we update an accumulator

$$v \leftarrow \beta_{\text{rms}} v + (1 - \beta_{\text{rms}}) x_t^2, \quad (9)$$

and normalize by  $\text{RMS}(x_t) = \sqrt{v + \epsilon}$  with a small  $\epsilon > 0$ .

Applying this independently to  $I_t$  and  $\text{LP}_t$  yields normalized components

$$\tilde{I}_t = \frac{I_t}{\text{RMS}(I_t)}, \quad \tilde{\text{LP}}_t = \frac{\text{LP}_t}{\text{RMS}(\text{LP}_t)}. \quad (10)$$

We then define a shared base intrinsic score

$$u_t = \alpha_{\text{impact}} \tilde{I}_t + \alpha_{\text{LP}} \tilde{\text{LP}}_t, \quad (11)$$

with nonnegative weights  $\alpha_{\text{impact}}, \alpha_{\text{LP}}$ . Both variants use the same  $u_t$  and differ only in whether it is gated.

## 2.7 GLPE (no gate)

GLPE (no gate) is a region-aware intrinsic reward that combines feature-space impact and region-local learning progress without any explicit suppression mechanism. The intrinsic reward is

$$r_t^{\text{int}} = u_t. \quad (12)$$

This variant preserves the locality and nonstationarity tracking provided by the region partition and EMAs, while avoiding additional gating hyperparameters.

## 2.8 GLPE: gated learning-progress exploration

GLPE augments the same base score  $u_t$  with a binary, region-specific gate that suppresses intrinsic rewards in regions that appear difficult to predict yet show little or no learning progress.

*Global robust thresholds.* At any time, let  $\mathcal{R}$  denote the set of regions that have been visited at least once, and let  $\text{LP}(r)$  denote the current learning progress in region  $r$ . We compute robust global reference values using medians across visited regions,

$$\begin{aligned} \text{LP}_{\text{med}} &= \text{median}_{r \in \mathcal{R}} \text{LP}(r), \\ e_{\text{med}} &= \text{median}_{r \in \mathcal{R}} \mu_r^{\text{short}}. \end{aligned} \quad (13)$$

We define a learning-progress threshold  $\tau_{\text{LP}} = \kappa \text{LP}_{\text{med}}$  with multiplier  $\kappa > 0$ , and a scale-free normalized error score

$$s_r = \frac{\mu_r^{\text{short}}}{e_{\text{med}} + \varepsilon}, \quad (14)$$

where  $\varepsilon > 0$  avoids division by zero. A region  $r$  is considered *unproductive* on a visit if it simultaneously has low learning progress and high normalized error,

$$\text{LP}(r) < \tau_{\text{LP}} \quad \text{and} \quad s_r > \tau_s, \quad (15)$$

with  $\tau_s > 0$  a fixed threshold.

*Gate dynamics.* Each region maintains a gate value  $g_r \in \{0, 1\}$  and two counters tracking consecutive unproductive and productive visits. Gating is only activated once at least  $R_{\text{min}}$  regions have been visited so that the medians are meaningful. When gating is active, the update uses persistence and hysteresis: (i) if  $g_r = 1$  and the region is unproductive for  $K$  consecutive visits, set  $g_r \leftarrow 0$ ; and (ii) if  $g_r = 0$ , re-enable the region when  $\text{LP}(r) > h \tau_{\text{LP}}$  for two consecutive visits, where  $h > 1$  is a hysteresis multiplier. If gating is inactive (fewer than  $R_{\text{min}}$  visited regions), we set  $g_r \leftarrow 1$  and reset the counters.

The intrinsic reward for GLPE is then

$$r_t^{\text{int}} = g_{\rho_t} u_t. \quad (16)$$

## 2.9 On-policy optimization and intrinsic scheduling

We optimize the policy with PPO using the augmented reward  $r_t$ . Training details are given in Section 3.

For the GLPE family, we anneal the strength of intrinsic shaping over the course of training using a cosine taper. Let  $p \in [0, 1]$  denote training progress as a fraction of total environment steps. Given a start fraction  $p_{\text{start}}$  and end fraction  $p_{\text{end}}$  with  $p_{\text{start}} < p_{\text{end}}$ ,

we define

$$w(p) = \begin{cases} 1, & p \leq p_{\text{start}}, \\ \frac{1}{2} \left( 1 + \cos \left( \pi \frac{p - p_{\text{start}}}{p_{\text{end}} - p_{\text{start}}} \right) \right), & p_{\text{start}} < p < p_{\text{end}}, \\ 0, & p \geq p_{\text{end}}, \end{cases} \quad (17)$$

and set  $\eta_t = \eta w(p_t)$  in the augmented reward. When  $p_{\text{start}}$  and  $p_{\text{end}}$  are omitted, we hold  $\eta_t = \eta$  fixed throughout training.

The latent dynamics model parameters  $(\omega, \psi, \xi)$  are updated using the same on-policy transitions that generate intrinsic rewards, while the region statistics, gates, and RMS normalizers are updated online during intrinsic computation. Figure 1 summarizes the intrinsic reward computation within one PPO update.

```

1: Input: on-policy transitions  $\{(o_t, a_t, o_{t+1})\}_{t=1}^N$ ; encoder  $\phi_\omega$ ; forward
   model  $f_\psi$ ; inverse model  $g_\xi$ ; region assignment function  $\rho$ ; region
   EMAs  $\mu_r^{\text{short}}, \mu_r^{\text{long}}$ ; RMS accumulators; gate states  $g_r$  (optional)
2: Output: intrinsic rewards  $\{r_t^{\text{int}}\}_{t=1}^N$ ; updated region statistics; updated
   intrinsic model parameters
3: for  $t = 1$  to  $N$  do
4:    $z_t \leftarrow \phi_\omega(o_t)$ ,  $z_{t+1} \leftarrow \phi_\omega(o_{t+1})$ 
5:    $e_t \leftarrow \frac{1}{\eta} \|f_\psi(z_t, a_t) - z_{t+1}\|_2^2$ 
6:   Assign region  $r \leftarrow \rho(z_t)$  (inserting  $z_t$  and splitting leaves when
   needed)
7:   Update region EMAs  $\mu_r^{\text{short}}, \mu_r^{\text{long}}$  with  $e_t$ 
8:    $\text{LP}_t \leftarrow \max(0, \mu_r^{\text{long}} - \mu_r^{\text{short}})$ 
9:    $I_t \leftarrow \|z_{t+1} - z_t\|_2$ 
10:  Normalize  $I_t$  and  $\text{LP}_t$  with running RMS to get  $\tilde{I}_t, \tilde{\text{LP}}_t$ 
11:   $u_t \leftarrow \alpha_{\text{impact}} \tilde{I}_t + \alpha_{\text{LP}} \tilde{\text{LP}}_t$ 
12:  if gating enabled then
13:    Compute robust global medians  $\text{LP}_{\text{med}}, e_{\text{med}}$  across visited re-
    gions
14:    Update gate  $g_r$  using persistence and hysteresis rules
15:     $r_t^{\text{int}} \leftarrow g_r u_t$ 
16:  else
17:     $r_t^{\text{int}} \leftarrow u_t$ 
18:  end if
19: end for
20: Update intrinsic model parameters  $(\omega, \psi, \xi)$  using  $\{(z_t, a_t, z_{t+1})\}$ 
   and supervised losses

```

Figure 1: High-level pseudocode for computing GLPE-family intrinsic rewards within a PPO update.

## 3 Experimental Setup

### 3.1 Tasks and environments

We evaluate exploration and control on five standard Gymnasium benchmarks spanning discrete and continuous control: MOUNTAINCAR-V0, BIPEDALWALKER-V3, and the MuJoCo locomotion tasks ANT-V5, HALF-CHEETAH-V5, and HUMANOID-V5. Across experiments, we use the default environment reward functions and termination conditions, with no domain randomization and no additional frame skipping. Training uses vectorized environments with  $B$  parallel instances and rollouts of length  $T$  per instance. We set  $(B, T)$  so that the nominal on-policy batch size per PPO update is

$$N = B \times T = 16,384 \quad (18)$$



transitions, varying  $(B, T)$  only to accommodate environment throughput and episode structure. To match the total step budget exactly, the final update may use a smaller batch when fewer than  $N$  steps remain. Table 1 summarizes the suite, interaction budgets, and parallelization settings. For each environment, we evaluate all methods using the same set of training seeds (Table 1) and enable deterministic execution where supported.

| Environment      | $B$ | $T$   | $N$    | Total steps | Seeds |
|------------------|-----|-------|--------|-------------|-------|
| MOUNTAINCAR-V0   | 16  | 1,024 | 16,384 | 3,000,000   | 10    |
| BIPEDALWALKER-V3 | 8   | 2,048 | 16,384 | 7,000,000   | 8     |
| ANT-V5           | 8   | 2,048 | 16,384 | 15,000,000  | 8     |
| HALFCHEETAH-V5   | 8   | 2,048 | 16,384 | 15,000,000  | 8     |
| HUMANOID-V5      | 4   | 4,096 | 16,384 | 30,000,000  | 5     |

**Table 1: Benchmark suite and training budgets.  $B$  is the number of parallel environment instances,  $T$  is the rollout horizon per instance, and  $N = B \times T$  is the number of transitions used in each PPO update. Each run is evaluated offline at saved checkpoints using 20 episodes (Section 3.4).**

*Training budgets beyond the early-learning regime.* The total environment-step budgets in Table 1 are intentionally generous so that runs extend beyond the initial improvement phase. This provides sufficient budget to assess long-horizon stability of exploration behavior and intrinsic shaping, including late-training drift, regressions, or oscillations.

### 3.2 Methods compared

We compare the two GLPE variants, GLPE and GLPE (no gate) (Section 2), against commonly used exploration baselines: Vanilla PPO, ICM, RND, RIDE, and RIAC. Vanilla PPO optimizes only the extrinsic environment reward. ICM uses forward-model prediction error in a learned feature space as an intrinsic signal, while RND uses the prediction error of a trainable predictor against a fixed random target network. RIDE rewards feature-space impact, down-weighted by episodic visitation counts obtained by discretizing features, and RIAC uses region-local learning progress computed from changes in forward-model error within an adaptive partition of feature space. All methods use the same PPO backbone and network architectures. Within each environment we keep PPO hyperparameters fixed across methods to isolate the effect of the intrinsic objective.

For intrinsic-reward methods, the policy is trained with the augmented reward defined in Section 2, combining the environment reward with a scaled and clipped intrinsic term. We use the same intrinsic scaling coefficient  $\eta$  and clipping range  $r_{\max}$  for all intrinsic methods within a given environment (Table 2), so differences arise from the intrinsic signal itself rather than from reward-scale tuning. GLPE (no gate) uses the same base intrinsic score as GLPE but disables the gating mechanism.

### 3.3 RL backbone and training protocol

All agents are trained with Proximal Policy Optimization (PPO; [19]) and generalized advantage estimation (GAE; [18]). The policy

and value function are parameterized by separate multilayer perceptrons with two hidden layers of width 256 and ReLU activations. For continuous-control tasks, the policy outputs a diagonal Gaussian distribution. Actions are squashed to the environment bounds when those bounds are finite.

Each training iteration proceeds as follows: (i) collect up to  $N = 16,384$  on-policy transitions by running the current policy in  $B$  vectorized environments for  $T$  steps; (ii) compute the intrinsic reward for each transition (when applicable), set intrinsic rewards to zero on environment-terminal transitions and on time-limit truncations without a final observation, and form the total reward used for advantage estimation; (iii) compute advantages and value targets with GAE, bootstrapping across time-limit truncations when a valid final observation is available and treating truncations without a final observation as terminal for bootstrapping purposes; and (iv) perform multiple epochs of PPO updates over shuffled minibatches of the collected batch. We use Adam optimizers for both the policy and value networks, normalize advantages within each update batch, and clip gradient norms to 1.0.

We normalize vector observations using a running mean and variance estimated online from collected rollouts. We apply the same normalization to the inputs of the policy, value, and intrinsic modules.

*Intrinsic reward computation and updates.* Trainable intrinsic models are updated once per PPO iteration using the same on-policy batch used for PPO. For methods whose intrinsic output is not internally normalized (e.g., ICM, RND, and RIDE), we apply a running RMS normalization to the raw intrinsic signal before scaling and clipping. For methods that produce a normalized intrinsic output by construction (RIAC and the GLPE family), we directly apply scaling and clipping. For GLPE and GLPE (no gate), we additionally anneal the intrinsic coefficient  $\eta_t$  using the cosine taper described in Section 2, with task-specific start and end fractions listed in Table 2.

### 3.4 Evaluation protocol

We evaluate performance using undiscounted episodic return from the environment (extrinsic reward only). We do not run evaluation

| Environment      | $\eta$ | $r_{\max}$ | $\alpha_{LP}$ | $p_{\text{start}}$ | $p_{\text{end}}$ | $C$ | $D_{\max}$ | $\beta_{\text{long}}$ | $\beta_{\text{short}}$ | $\kappa$ | $\tau_s$ | $K$ | $R_{\min}$ |
|------------------|--------|------------|---------------|--------------------|------------------|-----|------------|-----------------------|------------------------|----------|----------|-----|------------|
| MOUNTAINCAR-V0   | 0.05   | 4.0        | 0.5           | 0.05               | 0.80             | 128 | 10         | 0.995                 | 0.90                   | 0.010    | 2.0      | 5   | 8          |
| BIPEDALWALKER-V3 | 0.08   | 4.0        | 0.5           | 0.12               | 0.75             | 200 | 12         | 0.995                 | 0.90                   | 0.010    | 2.0      | 5   | 16         |
| ANT-V5           | 0.06   | 4.0        | 0.6           | 0.10               | 0.70             | 256 | 12         | 0.997                 | 0.92                   | 0.012    | 2.5      | 8   | 64         |
| HALFCHEETAH-V5   | 0.04   | 5.0        | 0.5           | 0.05               | 0.75             | 256 | 12         | 0.995                 | 0.90                   | 0.010    | 2.0      | 5   | 32         |
| HUMANOID-V5      | 0.06   | 5.0        | 0.5           | 0.05               | 0.80             | 256 | 12         | 0.998                 | 0.92                   | 0.010    | 2.0      | 5   | 32         |

**Table 2: GLPE hyperparameters by environment. We fix  $\alpha_{\text{impact}} = 1.0$  for all tasks.  $p_{\text{start}}$  and  $p_{\text{end}}$  define the cosine taper schedule for  $\eta_t$  as a function of training progress  $p$  (Section 2); this taper is used for GLPE and GLPE (no gate).  $C$  and  $D_{\max}$  are the region capacity and maximum depth for the online latent-space partition.  $\kappa$  sets the learning-progress threshold via  $\tau_{LP} = \kappa LP_{\text{med}}$ ,  $\tau_s$  is the normalized-error threshold,  $K$  is the number of consecutive unproductive visits required to gate off a region, and  $R_{\min}$  is the minimum number of visited regions required before gating is activated (Section 2). The hysteresis multiplier is fixed to  $h = 2.0$  for all tasks. GLPE (no gate) uses the same settings but does not apply gating.**

rollouts online during training; instead, we evaluate saved checkpoints offline. Each evaluated checkpoint is run for 20 episodes in a separate environment instance using deterministic action selection (distribution mode). Episodes are reset with fixed per-episode seeds derived from the training seed. For each environment and method, we run multiple independent training seeds (Table 1) and aggregate results across seeds.

**Checkpointing and logging.** During training we checkpoint the full agent state at step 0, at nine additional warmup points approximately evenly spaced within the first checkpoint interval, and then at a fixed checkpoint interval thereafter (set to 10% of the total step budget per environment), with a final checkpoint at the end of training. We additionally log scalar metrics every fixed number of environment steps (set to 2% of the total step budget).

### 3.5 Efficiency measurements

In addition to sample efficiency (learning as a function of environment steps), we track computational efficiency using wall-clock time. All timings are measured in the same execution setting used for training (CPU in our experiments) and therefore reflect end-to-end optimization overhead. For each PPO iteration we log per-component wall-clock time spent in environment interaction, policy inference, intrinsic computation, intrinsic-module updates, advantage computation, and PPO optimization. We use these logs to form cumulative training time.

When reporting scalar summaries of learning curves, we compute two AUC metrics via trapezoidal integration of mean return: step-AUC (return versus environment steps, divided by the environment’s step budget) and wall-clock AUC (return versus cumulative training time). For wall-clock AUC, we use a *common time budget* per environment defined as the minimum final wall-clock time attained among the compared methods. This ensures that summaries do not rely on extrapolation beyond the measured runtime of any method.

### 3.6 Implementation details and hyperparameters

Tables 3 and 2 list the PPO and GLPE hyperparameters used in all experiments. Unless specified in the tables, we use  $\gamma = 0.99$  and value-loss coefficient 0.5 for PPO.

| Environment      | LR                   | Epochs | Minib | Clip | $\lambda$ | Ent. | v-clip | KL stop |
|------------------|----------------------|--------|-------|------|-----------|------|--------|---------|
| MOUNTAINCAR-V0   | $3.0 \times 10^{-4}$ | 10     | 16    | 0.20 | 0.95      | 0.00 | 0.00   | 0.06    |
| BIPEDALWALKER-V3 | $5.0 \times 10^{-4}$ | 5      | 16    | 0.25 | 0.95      | 0.00 | 0.00   | 0.06    |
| ANT-V5           | $1.5 \times 10^{-4}$ | 15     | 64    | 0.20 | 0.95      | 0.00 | 0.20   | 0.04    |
| HALFCHEETAH-V5   | $3.0 \times 10^{-4}$ | 10     | 32    | 0.20 | 0.95      | 0.01 | 0.20   | 0.03    |
| HUMANOID-V5      | $2.0 \times 10^{-4}$ | 5      | 32    | 0.20 | 0.97      | 0.01 | 0.20   | 0.03    |

**Table 3: PPO hyperparameters by environment.** “Minib” is the number of minibatches per update (each epoch iterates once over all  $N$  samples). “Ent.” is the entropy regularization coefficient. “v-clip” is the value-function clipping range (0 disables value clipping). “KL stop” is the approximate KL threshold for early stopping within an update.

For methods that learn a latent dynamics model (ICM, RIDE, RIAC, and the GLPE family), the shared encoder produces 128-dimensional features and is implemented as a two-layer MLP (256

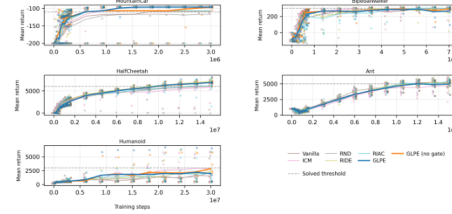
units per layer, ReLU) for these vector-observation tasks. Forward and inverse model losses are weighted equally. The model is trained with Adam at learning rate  $3 \times 10^{-4}$ , and intrinsic-model gradient norms are clipped to 5.0.

## 4 Results and Discussion

We compare GLPE and GLPE (no gate) against Vanilla PPO and four intrinsic-reward baselines (ICM, RND, RIDE, RIAC) on the five-task suite from Section 3. Unless otherwise stated, we report *extrinsic* evaluation return under the deterministic offline evaluation protocol (Section 3.4).

### 4.1 Learning curves and final performance

Figure 2 shows evaluation learning curves versus environment steps. Across tasks, GLPE (no gate) tracks the best-performing baseline closely and exhibits stable performance across seeds. The gated variant differs primarily in sparse-reward or exploration-sensitive regimes: it is most beneficial on MOUNTAINCAR-V0 and can be overly conservative on HUMANOID-V5, where between-seed variance dominates.



**Figure 2: Evaluation learning curves for GLPE and baseline intrinsic-reward methods.** Each panel plots mean *extrinsic* return (higher is better) versus environment steps for one task. Solid lines show the mean across seeds at each checkpoint; markers show the corresponding per-seed means (20 evaluation episodes per checkpoint). The horizontal dashed line is a fixed “solved” return threshold used for summary analyses. These curves summarize learning speed and stability across seeds; tight clustering of per-seed markers indicates robust performance.

Final-checkpoint performance is reflected by the rightmost points in Figure 2. Table 4 reports step-AUC, which rewards both early learning and sustained performance over the fixed interaction budget.

On MOUNTAINCAR-V0, GLPE attains the best final return and the strongest step-AUC among all compared methods, and both GLPE variants solve in all seeds under the solved-threshold criterion (Figure 2, 100% threshold). On BIPEDALWALKER-V3, GLPE (no gate) improves rapidly and remains competitive by step-AUC, but many runs plateau just below the solved threshold of 300, making thresholded success counts sensitive to small differences near the cutoff (Figure 3, 100% threshold). On the MuJoCo locomotion tasks, where extrinsic reward is comparatively dense, the two GLPE

variants behave similarly and remain within a modest gap of the strongest intrinsic baseline under both the learning-curve view (Figure 2) and the step-AUC summary (Table 4). HUMANOID-V5 exhibits particularly high variance, with wide confidence intervals across all methods, so method rankings are not stable under bootstrap uncertainty.

Some diagnostic plots include auxiliary GLPE variants (e.g., cached medians and single-component ablations) to isolate computational and signal contributions; we treat these as supporting analyses rather than primary baselines.

#### 4.2 Ablations: impact vs. learning progress

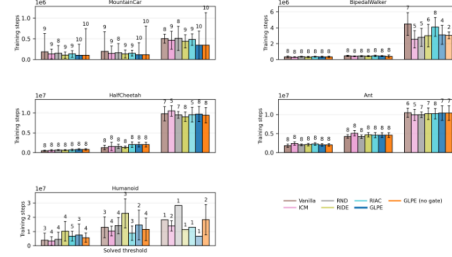
To probe which components drive performance in different regimes, Table 5 reports final returns for GLPE and two single-term ablations on representative tasks.

On sparse-reward MOUNTAINCAR-V0, combining impact and learning progress is critical: each single-term variant falls substantially short of full GLPE. In contrast, on dense-reward ANT-V5 the impact-only variant is competitive, and on near-threshold BIPEDALWALKER-V3 the LP-only variant comes closest to the solved cutoff. These patterns motivate the combined score as a robust default when the “right” intrinsic driver is task-dependent.

#### 4.3 Thresholded reliability and steps-to-threshold

Learning curves can obscure whether a method is merely slower or is failing entirely. We therefore evaluate reliability at fixed return

thresholds using steps-to-threshold statistics that report both time-to-threshold and reach rates across seeds (Figure 3). We use task-specific solved thresholds (MOUNTAINCAR-V0:  $-110$ , BIPEDALWALKER-V3:  $300$ , ANT-V5:  $5,000$ , HALFCHETAH-V5:  $6,000$ , HUMANOID-V5:  $3,000$ ) and additionally report results at 25% and 50% of these levels.



**Figure 3: Sample efficiency and reliability at fixed return thresholds.** For each task, bars report the median number of environment steps to first reach a target return level, estimated by linearly interpolating between evaluation checkpoints. The three x-axis positions correspond to 25%, 50%, and 100% of each task’s solved threshold (the dashed line in Figure 2); intermediate thresholds are computed using the same monotonic mapping across tasks. Error bars show the standard deviation in steps across seeds that reached the threshold. Bar opacity encodes the fraction of seeds that reached the threshold by the end of training, and the number above each bar is the count of successful seeds. Lower bars and higher opacities indicate faster and more reliable learning.

At the 50% threshold, GLPE (no gate) reaches the target in all seeds on four tasks and in 4/5 seeds on HUMANOID-V5, matching or exceeding the best baseline reach count on every environment (Figure 3, 50% threshold). The gap between 50% and 100% success is concentrated in high-variance or near-cutoff regimes: HUMANOID-V5 and BIPEDALWALKER-V3 show many runs that reach competent behavior without consistently clearing the full solved cutoff (Figure 3, 100% threshold). On MOUNTAINCAR-V0, by contrast, both GLPE variants reach the solved threshold quickly and reliably, and several baselines exhibit occasional failures that remain near the minimum return (Figure 2).

#### 4.4 Intrinsic shaping dynamics and gating behavior

The GLPE family is designed as a transient exploration aid rather than a permanent auxiliary objective. We therefore anneal the intrinsic coefficient  $\eta_t$  using the cosine taper defined in Section 2, down-weighting intrinsic shaping late in training. Figure 4 decomposes the training-time rollout reward into extrinsic and intrinsic components. Compared to baselines that maintain a nonzero intrinsic term throughout training, GLPE and GLPE (no gate) reduce the applied intrinsic contribution as training progresses.

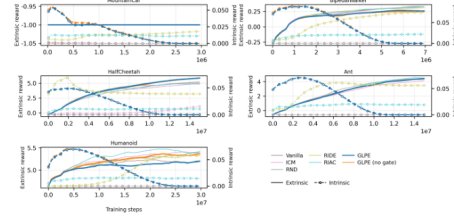
| Environment      | GLPE                    | GLPE (no gate)          | Best baseline                 |
|------------------|-------------------------|-------------------------|-------------------------------|
| MOUNTAINCAR-V0   | -107.0 [-114.8, -101.8] | -111.3 [-128.7, -101.4] | RIDE: -113.7 [-134.2, -101.9] |
| BIPEDALWALKER-V3 | 249.0 [223.9, 270.7]    | 255.2 [230.0, 275.6]    | ICM: 257.3 [229.4, 278.3]     |
| ANT-V5           | 3,402 [3,067, 3,708]    | 3,402 [3,067, 3,797]    | RND: 3,565 [3,202, 3,876]     |
| HALFCHETAH-V5    | 5,005 [4,708, 5,316]    | 4,998 [4,700, 5,296]    | RIDE: 5,208 [5,019, 5,410]    |
| HUMANOID-V5      | 1,583 [551, 3,360]      | 1,665 [769, 2,921]      | ICM: 1,781 [829, 2,797]       |

**Table 4: Step-AUC of deterministic evaluation return versus cumulative environment steps, computed by trapezoidal integration of the mean return-versus-steps curve over evaluation checkpoints and dividing by the environment’s step budget. This yields the average evaluation return over training, rewarding methods that learn quickly and sustain high returns. Intervals are obtained by integrating the per-checkpoint 95% bootstrap confidence bands over seeds and should be interpreted as an approximate uncertainty range for the integrated metric. Higher is better. The best baseline is selected (per environment) among Vanilla PPO, ICM, RND, RIDE, and RAC by highest mean step-AUC.**

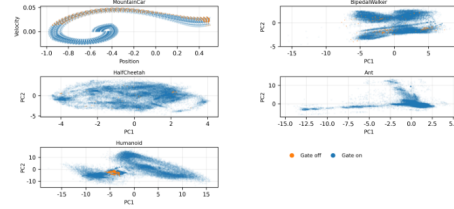
| Environment      | GLPE                 | Impact-only            | LP-only                |
|------------------|----------------------|------------------------|------------------------|
| MOUNTAINCAR-V0   | -96.2 [-96.5, -95.9] | -111.8 [-137.0, -96.2] | -106.6 [-127.4, -96.0] |
| BIPEDALWALKER-V3 | 266.5 [214.0, 301.3] | 277.5 [256.0, 296.5]   | 298.8 [285.5, 308.1]   |
| ANT-V5           | 4,961 [4,421, 5,359] | 5,235 [4,544, 5,663]   | 4,599 [3,617, 5,401]   |

**Table 5: Final-checkpoint mean extrinsic return for GLPE and component ablations. Impact-only sets  $\alpha_{LP}=0$  and LP-only sets  $\alpha_{Impact}=0$ . Brackets show 95% bootstrap confidence intervals over training seeds. Higher is better.**

To make the gate’s effect concrete, Figure 5 shows state-space projections of final-checkpoint trajectories with time steps colored by whether intrinsic shaping is active. Across environments, gate-off points occupy a small and typically localized subset of the visited distribution. This supports the interpretation of gating as a targeted guardrail: it suppresses shaping only in narrow regions that appear persistently high-error and low-progress, rather than globally weakening exploration.



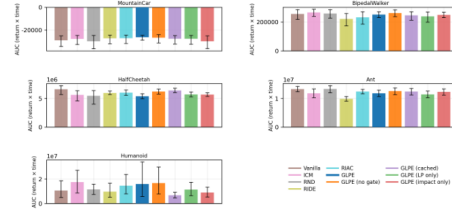
**Figure 4: Decomposition of rollout reward during training.** Solid lines show the mean extrinsic reward per environment step collected during PPO rollouts. Dashed lines show the intrinsic shaping component actually added during training, computed as the difference between the logged total reward and the extrinsic reward after each method’s scaling and clipping (GLPE additionally applies tapering). Intrinsic values use the right axis and extrinsic values use the left axis. This plot is a diagnostic for the optimization signal and complements the evaluation curves, which report extrinsic return without intrinsic shaping.



**Figure 5: State-space view of GLPE’s gating decisions.** Points correspond to states visited in trajectories from the final checkpoint (subsamped); for MOUNTAINCAR-V0 we plot raw position and velocity, and for the remaining environments we project z-scored observations onto the first two principal components. Blue points indicate time steps where the intrinsic term is active (gate on), while orange points indicate time steps where the gate disables intrinsic shaping (gate off). Across tasks, gate-off points occupy a small and typically localized subset of the visited distribution, indicating that gating suppresses intrinsic shaping only in narrow regions rather than globally.

#### 4.5 Wall-clock efficiency and computational overhead

Intrinsic rewards can change not only sample efficiency but also end-to-end training cost. Figure 6 reports wall-clock AUC of evaluation return under a common per-environment time horizon (Section 3.5), ensuring that slower methods are not advantaged by extrapolation beyond measured runtime. Across BIPEDALWALKER-V3, HALF-CHEETAH-V5, ANT-V5, and HUMANOID-V5, GLPE (no gate) remains close to the strongest baseline under this wall-clock view. The gated variant typically attains lower wall-clock AUC, consistent with the additional computation required by gating and robust statistics.



**Figure 6: Wall-clock AUC of evaluation performance.** For each task, bars report trapezoidal area under the curve (AUC) of the mean deterministic evaluation return as a function of cumulative training time (Section 3.5). To compare methods fairly when overhead differs, each task uses a common time horizon equal to the minimum final training time among the compared methods, and curves are truncated to this horizon (no extrapolation). Higher is better. Error bars are derived by integrating the per-checkpoint 95% confidence bounds on mean return.

Figure 7 decomposes per-update runtime into environment interaction, PPO optimization, and intrinsic components. On the more expensive MuJoCo tasks, environment stepping and PPO optimization dominate total time, making intrinsic overhead a secondary factor; on MOUNTAINCAR-V0, where environment interaction is inexpensive, intrinsic overhead plays a larger role in wall-clock comparisons.

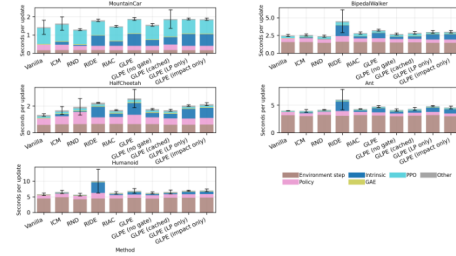
Recomputing the global medians used by GLPE’s gate to set robust thresholds is one identifiable cost of the gated variant. In a standalone microbenchmark of this operation, we report median throughput across nine trials for the same workload. Recomputing the medians every update (cache off,  $k=1$ ) yields 20,877 transitions/s (baseline, 1.00 $\times$ ), while recomputing every  $k=64$  updates and reusing cached values in between (cache on) yields 100,173 transitions/s (4.83 $\times$  speedup). Because cached medians are slightly stale and can change gating decisions when  $k > 1$ , we treat caching as an optional implementation choice rather than part of the core GLPE comparison.

## 5 Conclusion

Intrinsic-reward shaping can substantially accelerate learning when extrinsic feedback is sparse or delayed, yet novelty- or prediction-error-based bonuses can also concentrate exploration in regions where progress is slow, unstable, or effectively noisy. We introduced the *GLPE family*, which ties intrinsic reward to two complementary signals computed from on-policy transitions: feature-space impact and region-local learning progress, estimated from short- and long-horizon prediction-error averages within an online latent-space partition. The gated variant adds a simple region-wise gate that suppresses intrinsic shaping when prediction error remains high while learning progress stalls, providing a targeted guardrail against unproductive curiosity. GLPE (no gate) uses the same base score without this additional thresholding.

Across five Gymnasium control benchmarks and a shared PPO backbone, GLPE (no gate) is the most consistent default, remaining competitive with strong intrinsic baselines under both step- and wall-clock-normalized views. The gated variant is most valuable in sparse-reward regimes such as MOUNTAINCAR-V0, where it improves reliability and reduces instability relative to ungated shaping, but it can be unnecessarily conservative in high-variance settings such as HUMANOID-V5. Finally, the cosine taper schedule makes intrinsic shaping intentionally transient, smoothly reducing its contribution late in training so that final performance reflects primarily the learned task policy.

This study is limited to vector-observation benchmarks and on-policy PPO; as with most intrinsic bonuses, the shaping signal we apply is not policy-invariant. Future work can extend GLPE to richer observation modalities and alternative training regimes and make the gated variant more computationally efficient and adaptive without changing the underlying intrinsic objective.



**Figure 7: Timing breakdown per PPO update.** Each panel shows a stacked decomposition of wall-clock seconds per training update for one task. For each method, we take the final quarter of recorded updates in each run, compute per-component medians, and then average these medians across runs (stacked bars). The black error bar on each stack is a 95% confidence interval for the total seconds per update across runs.

## References

- [1] Adrià Puigdomènech Badia, Pablo Sprechmann, Alex Vitvitskyi, Daniel Guo, Bilal Piot, Steven Kapturowski, Olivier Tieleman, Martin Arjovsky, Alexander Pritzel, Andrew Bolt, and Charles Blundell. 2020. Never Give Up: Learning Directed Exploration Strategies. *arXiv (Cornell University)* (2 2020). doi:10.48550/arxiv.2002.06038
- [2] A. Baranes and P.-Y. Oudeyer. 2009. R-IAC: Robust Intrinsically Motivated Exploration and Active Learning. *IEEE Transactions on Autonomous Mental Development* 1, 3 (10 2009), 155–169. doi:10.1109/tamd.2009.2037513
- [3] Andrew G. Barto. 2012. *Intrinsic Motivation and Reinforcement Learning*. 17–47 pages. doi:10.1007/978-3-642-32375-1\_2
- [4] Marc G. Bellemare, Sriram Srinivasan, Georg Ostrovski, Tom Schaul, David Saxton, and Remi Munos. 2016. Unifying Count-Based Exploration and Intrinsic Motivation. *arXiv (Cornell University)* (6 2016). doi:10.48550/arxiv.1606.01868
- [5] Yuri Burda, Harri Edwards, Deepak Pathak, Amos Storkey, Trevor Darrell, and Alexei A. Efros. 2018. Large-Scale Study of Curiosity-Driven Learning. *arXiv (Cornell University)* (8 2018). doi:10.48550/arxiv.1808.04355
- [6] Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. 2018. Exploration by Random Network Distillation. *arXiv (Cornell University)* (10 2018). doi:10.48550/arxiv.1810.12894
- [7] Rein Houthoofd, Xi Chen, Yan Duan, John Schulman, Filip De Turck, and Pieter Abbeel. 2016. VIME: Variational Information Maximizing Exploration. *arXiv (Cornell University)* (5 2016). doi:10.48550/arxiv.1605.09674
- [8] Augustine N. Mavor-Parker, Kimberly A. Young, Caswell Barry, and Lewis D. Griffin. 2021. How to Stay Curious While Avoiding Noisy TVs Using Aleatoric Uncertainty Estimation. *arXiv (Cornell University)* (2 2021). doi:10.48550/arxiv.2102.04399
- [9] Andrew Ng, Daishi Harada, and Stuart Russell. 1999. Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping. Morgan Kaufmann, San Francisco, us, 278–287. doi:10.5555/645528.657613
- [10] Georg Ostrovski, Marc G. Bellemare, Aaron van den Oord, and Remi Munos. 2017. Count-Based Exploration with Neural Density Models. *arXiv (Cornell University)* (3 2017). doi:10.48550/arxiv.1703.01310
- [11] Pierre-Yves Oudeyer and Frederic Kaplan. 2007. What Is Intrinsic Motivation? A Typology of Computational Approaches. *Frontiers in Neurobotics* 1 (1 2007), 6. doi:10.3389/neuro.12.006.2007
- [12] Pierre-Yves Oudeyer, Frederic Kaplan, and Verena V. Hafner. 2007. Intrinsic Motivation Systems for Autonomous Mental Development. *IEEE Transactions on Evolutionary Computation* 11, 2 (4 2007), 265–286. doi:10.1109/tevc.2006.890271
- [13] Deepak Pathak, Pulkit Agrawal, Alexei Efros, and Trevor Darrell. 2017. Curiosity-Driven Exploration by Self-Supervised Prediction. *arXiv* (2017). doi:10.48550/arxiv.1705.05363
- [14] Deepak Pathak, Dhiraj Gandhi, and Abhinav Gupta. 2019. Self-Supervised Exploration via Disagreement. *arXiv (Cornell University)* (6 2019). doi:10.48550/arxiv.1906.04161
- [15] Roberta Raileanu and Tim Rocktäschel. 2020. RIDE: Rewarding Impact-Driven Exploration for Procedurally-Generated Environments. *arXiv (Cornell University)* (2 2020). doi:10.48550/arxiv.2002.12292
- [16] Nikolay Savinov, Anton Raichuk, Raphaël Marinier, Damien Vincent, Marc Pollefeys, Timothy Lillicrap, and Sylvain Gelly. 2018. Episodic Curiosity through Reachability. *arXiv (Cornell University)* (10 2018). doi:10.48550/arxiv.1810.02274
- [17] Juergen Schmidhuber. 1991. *A Possibility for Implementing Curiosity and Boredom in Model-Building Neural Controllers*. 222–228 pages. doi:10.7551/mitpress/3115.003.0030
- [18] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. 2015. High-Dimensional Continuous Control Using Generalized Advantage Estimation. *arXiv (Cornell University)* (6 2015). doi:10.48550/arxiv.1506.02438
- [19] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. *arXiv* (2017). doi:10.48550/arxiv.1707.06347
- [20] Satinder Singh, Andrew Barto, and Nuttapon Chentanez. 2004. Intrinsically Motivated Reinforcement Learning. MIT Press, Cambridge, us.
- [21] Bradley C. Stadie, Sergey Levine, and Pieter Abbeel. 2015. Incentivizing Exploration in Reinforcement Learning with Deep Predictive Models. *arXiv (Cornell University)* (7 2015). doi:10.48550/arxiv.1507.00814
- [22] Haoran Tang, Rein Houthoofd, Davis Foote, Adam Stooke, Xi Chen, Yan Duan, John Schulman, Filip De Turck, and Pieter Abbeel. 2016. #Exploration: A Study of Count-Based Exploration for Deep Reinforcement Learning. *arXiv (Cornell University)* 30 (11 2016), 2750–2759. doi:10.48550/arxiv.1611.04717

## Peer Review Materials

Reviewer #1

---

### Questions

**1. Clarity / Writing Style**

Mostly understandable.

**2. Originality / Innovativeness** Note that a paper could score high for originality even if the results do not show a convincing benefit.

Somewhat conventional.

**3. Impact of Ideas and/or Results**

Some of the ideas/results will substantially help other people's ongoing research.

**4. Overall Recommendation** This is the score that will be used for the ranking of the paper.

Very good paper (only the 25% of the papers should have this ranking or higher).

**5. Reviewer Confidence (How accurate are your reviews?)**

Low. I have low confidence on my review.

**6. Summarize shortly the contributions of the paper in your own words. Summary should explain shortly (in 1-3 sentences) the main technical contribution. Do not discuss strengths and weaknesses here.**

The paper proposed two methods that generate intrinsic rewards on learning explorations. These methods differ on intrinsic rewards forwarded in transition.

**7. Strengths** What are the strengths of the paper? Itemised list of max 5 strongest points (just list shortly; do not discuss here)

The algorithm is well designed, equipped with bounding constraints on the learning process.

**8. Weaknesses** What are the weaknesses of the paper? Itemised list of max 5 weakest points (just list shortly; do not discuss here).

There are some elements which are presented but whose utility in the algorithm or in the generation of the rewards is not defined or described.

**9. Main Discussion of Paper** This is the most important part of your review. Discuss all weaknesses and strengths of the paper in detail and everything else that you consider important and that influences your review scores in some way. The authors must be provided with all sufficient feedback to understand why their paper is scored the way it is. \*\*\* Your review should have the depth and the thoroughness.

1. How is  $g_{xi}$  used in the model? How is it different from  $g_r$  used in the gate?

2. What's the purpose  $script(L)_{dyn}$ ?

3. Is there an initial value for the accumulator  $nu$ ?

4. The use of  $r$  as notation for region and for reward can be confusing, even with the indices present.

5. The weights  $\beta_{long}$  and  $\beta_{short}$  are high at .99 and .9. Are these values standard? What happens when lesser values are used?

6. In the training progress, how is  $p$  determined?

7. How is  $w(p)$  incorporated in the model?



## Questions

### 1. Clarity / Writing Style

Understandable.

### 2. Originality / Innovativeness **Note that a paper could score high for originality even if the results do not show a convincing benefit.**

Creative: Relatively few people in our community would have put these ideas together

### 3. Impact of Ideas and/or Results

Some of the ideas/results will substantially help other people's ongoing research.

### 4. Overall Recommendation **This is the score that will be used for the ranking of the paper.**

Very good paper (only the 25% of the papers should have this ranking or higher).

### 5. Reviewer Confidence (How accurate are your reviews?)

High. I have high confidence on my review.

### 6. Summarize shortly the contributions of the paper in your own words. Summary should explain shortly (in 1-3 sentences) the main technical contribution. Do not discuss strengths and weaknesses here.

This paper explores GLPE (Gated Learning-Progress Exploration) in reinforcement learning and subjects the method to several benchmark environments. The authors observe comparable performance against baselines and noticeable advantages for the ungated GLPE version.

### 7. Strengths **What are the strengths of the paper? Itemised list of max 5 strongest points (just list shortly; do not discuss here)**

- well written and generally understandable
- with substantial contribution to reinforcement learning literature

### 8. Weaknesses **What are the weaknesses of the paper? Itemised list of max 5 weakest points (just list shortly; do not discuss here).**

- limited to gymnasium benchmarks

### 9. Main Discussion of Paper **This is the most important part of your review. Discuss all weaknesses and strengths of the paper in detail and everything else that you consider important and that influences your review scores in some way. The authors must be provided with all sufficient feedback to understand why their paper is scored the way it is. \*\*\* Your review should have the depth and the thoroughness.**

The paper introduces gated learning progress exploration (GLPE) and evaluates variants of reinforcement learning algorithms employing GLPE. It subjects the algorithms to standard vector-observation benchmarks and concludes that the GLPE(no gate)-variant provides competitive results. The gated GLPE variant, on the other hand, showed promise in sparse reward regimes. The results appear correct and the paper is well written.

The research itself is very interesting and contributes to reinforcement learning literature. It has a robust experimental design as it subjected the algorithms to benchmarks with varying difficulty and complexity, within its limitation of gymnasium-based environments, which enabled deeper analysis of its results.

I recommend that this paper be accepted to the conference.

## Presentation Certification

